

IIR

**INGÉNIERIE
INFORMATIQUE
& RÉSEAUX**

ANNÉE UNIVERSITAIRE

2024-2025

OPTION

**MÉTHODES INFORMATIQUES APPLIQUÉES
À LA GESTION DES ENTREPRISES (MIAGE)**

PROJET DE FIN D'ÉTUDES

THÈME

**Refonte d'une plateforme web de prospection
B2B-B2C basée sur l'extraction, le traitement et la
visualisation intelligente de données
« DataPull »**

RÉALISÉ PAR :

ALOUAN Ayoub

ENCADRÉ PAR :

M. CHAREF Ayoub (EMSI)

M. El Kharaoua Rabie (IAWeb.dev)

SOUTENU LE :

..... Juillet 2025

Dédicace

À mes chers parents,

Pour votre affection, votre amour sans limite, votre confiance et votre patience. Pour les innombrables sacrifices que vous avez consentis afin de m'offrir une éducation et une formation dignes. Ce travail est l'aboutissement de vos prières et de vos vœux les plus chers. Que Dieu vous protège et vous garde à mes côtés.

À mes frères et sœurs,

Merci pour votre amour et votre soutien indéfectible tout au long de ces années d'études. Que ce travail soit le reflet d'une fraternité sincère et éternelle.

À mes professeurs,

Je vous rends un profond hommage pour votre engagement, votre rigueur et la richesse de votre enseignement. Grâce à vous, j'ai pu acquérir des connaissances solides et forger mon parcours avec fierté.

À mes camarades de promotion,

Merci pour l'esprit d'entraide, de partage et de convivialité qui a marqué ces années. Les souvenirs que nous avons construits ensemble resteront gravés dans ma mémoire. Vous êtes plus que des collègues, vous êtes de véritables amis.

À tous ceux qui ont contribué de près ou de loin à la réalisation de ce projet,

Recevez à travers ce modeste travail l'expression de ma sincère reconnaissance.

À mes proches et à mes amis les plus chers,

Je vous dédie humblement ce travail, en témoignage de mon attachement et de ma gratitude.

Remerciements

*Au terme de ce travail, nous tenons à exprimer nos sincères remerciements à toutes les personnes dont l'intervention, de près ou de loin au cours de ce projet, a favorisé son aboutissement. Nous tenons à exprimer notre immense gratitude, et à présenter nos remerciements les plus sincères à **M. CHAREF Ayoub**, qui a eu l'amabilité d'assurer notre encadrement durant la période de réalisation de notre travail, en nous fournissant un soutien fort précieux. Nous souhaitons remercier tout particulièrement nos encadrants externes, **M. El Kharaoua Rabie**, qui nous ont accueilli au sein de l'organisme et qui n'ont ménagé ni leur temps ni leur énergie pour nous aider à élaborer ce travail dans les meilleures conditions. Nous sommes très reconnaissants des conseils fructueux qu'ils n'ont cessé de nous prodiguer. Nos sincères remerciements vont également à tout le personnel de la société LAweb.dev qui nous ont offert l'occasion de travailler dans un cadre professionnel aussi agréable. Nous remercions chaleureusement **M. SAFSOUF Yassine**, chef du département Informatique, ainsi que l'ensemble du corps professoral de la section Informatique qui ont contribué à notre formation le long de notre parcours à l'école marocaine des sciences de l'ingénieur de Marrakech, et qui ont veillé à notre confort durant toute cette période. Et finalement, nous tenons à remercier vivement tous les membres du jury d'avoir accepté d'évaluer notre travail.*

Abstract

This report documents the comprehensive revamping and optimization of the DataPull platform during my end-of-studies internship at laweb.dev. The project addressed critical challenges in sourcing and delivering reliable business data by implementing a complete system overhaul. The solution encompasses automated data collection, cleaning, and enrichment processes designed to support effective lead generation for businesses.

The platform exploration covered multiple data acquisition methods including public APIs, web scraping techniques, and specialized tools deployed on cloud infrastructure. A robust ETL pipeline processes large-scale datasets with intelligent data cleaning and email discovery algorithms. The system features dual interfaces serving both business markets through interactive mapping, advanced search filters, and real-time data visualization.

Technical implementation involved developing two parallel applications using different technology stacks to compare rapid prototyping capabilities against enterprise scalability requirements. The ETL component utilizes modern frameworks for asynchronous task processing, handling large files with optimized memory management. Deployment strategies incorporated containerization and cloud infrastructure management, ensuring reliable data processing workflows and comprehensive system monitoring capabilities.

Key words: DataPull platform, Lead generation, Data extraction and processing, B2B/B2C prospecting, ETL pipeline.

Résumé

Ce rapport documente la refonte et l'optimisation de la plateforme DataPull lors de mon stage de fin d'études au sein de l'agence laweb.dev. Le projet a adressé des défis critiques dans l'approvisionnement et la livraison de données commerciales fiables en implémentant une refonte système complète. La solution propose des processus automatisés de collecte, de nettoyage et d'enrichissement des données, conçus pour optimiser la génération de leads pour les entreprises.

L'exploration de la plateforme a couvert plusieurs méthodes d'acquisition de données incluant des APIs publiques, des techniques de web scraping et des outils spécialisés déployés sur une infrastructure cloud. Un pipeline ETL robuste traite des jeux de données à grande échelle avec des algorithmes intelligents de nettoyage de données et de découverte d'emails. Le système propose des interfaces duales servant les deux marchés d'entreprise à travers une cartographie interactive, des filtres de recherche avancés et une visualisation de données en temps réel.

L'implémentation technique a impliqué le développement de deux applications parallèles utilisant différentes piles technologiques pour comparer les capacités de prototypage rapide contre les exigences de scalabilité entreprise. Le composant ETL utilise des frameworks modernes pour le traitement de tâches asynchrones, gérant de gros fichiers avec une gestion mémoire optimisée. Les stratégies de déploiement ont incorporé la conteneurisation et la gestion d'infrastructure cloud, assurant des flux de traitement de données fiables et des capacités complètes de monitoring système.

Mots clés : Plateforme DataPull, Génération de leads, Extraction et traitement de données, Prospection B2B/B2C, Pipeline ETL.

Glossaire

Abréviation	Définition
ACID	Atomicity, Consistency, Isolation, and Durability
AI	Artificial Intelligence
AMI	Amazon Machine Image
API	Application Programming Interface
AWS	Amazon Web Services
B2B	Business to Business
B2C	Business to Consumer
BS4	BeautifulSoup version 4
CFO	Chief Financial Officer
CIO	Chief Information Officer
CLI	Command Line Interface
CMS	Content Management System
CPU	Central Processing Unit
CRM	Customer Relationship Management
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CSV	Comma Separated Value
DB	Database
DevOps	Development and Operations
DOM	Document Object Model
EC2	Elastic Compute Cloud (AWS)
EMSI	École Marocaine des Sciences de l'Ingénieur
ENSAH	École Nationale des Sciences Appliquées d'Al Hoceima
ETL	Extract, Transform, Load
GDPR	General Data Protection Regulation
GIL	Global Interpreter Lock
HTML	HyperText Markup Language

HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
INPI	Institut National de la Propriété Industrielle
INSEE	Institut National de la Statistique et des Études Économiques
JPA	Java Persistence API
JS	JavaScript
JSON	JavaScript Object Notation
JSONB	JavaScript Object Notation Binary
JWT	JSON Web Token
ML	Machine Learning
NgRx	Angular Reactive Extensions
PaaS	Platform as a Service
PHP	Hypertext Preprocessor
RDC	Remote Desktop Connection
REST	Representational State Transfer
RxJS	Reactive Extensions for JavaScript
S3	Simple Storage Service (AWS)
SaaS	Software as a Service
SEO	Search Engine Optimization
SQL	Structured Query Language
SSL	Secure Sockets Layer
SSO	Single Sign-On
UC	Undetected Chrome
UML	Unified Modeling Language
UI/UX	User Interface / User Experience
VS Code	Visual Studio Code

Liste des figures

<i>Figure 1.1. Création de Iaweb.Dev.....</i>	<i>3</i>
<i>Figure 1.2. Havet Digital logo</i>	<i>4</i>
<i>Figure 1.3. Iaweb.dev logo.....</i>	<i>4</i>
<i>Figure 1.4. Organigramme de Iaweb.dev.....</i>	<i>6</i>
<i>Figure 1.5. Processus de Développement Agile avec Scrum</i>	<i>10</i>
<i>Figure 1.6. Diagramme du processus Agile Scrum</i>	<i>11</i>
<i>Figure 1.7. Diagramme de GANTT.....</i>	<i>14</i>
<i>Figure 2.1. Cycle de nettoyage des données.....</i>	<i>16</i>
<i>Figure 2.2. Concepts clés du backend.....</i>	<i>17</i>
<i>Figure 2.3. Principes UI/UX.....</i>	<i>18</i>
<i>Figure 2.4. Web scraping.....</i>	<i>19</i>
<i>Figure 2.5. RESTful API.....</i>	<i>20</i>
<i>Figure 2.6. ETL process.....</i>	<i>21</i>
<i>Figure 2.7. Infrastructure cloud - AWS</i>	<i>21</i>
<i>Figure 2.8. Tech stack de l'application web 1</i>	<i>28</i>
<i>Figure 2.9. Tech stack de l'application web 2</i>	<i>29</i>
<i>Figure 2.10. application ETL</i>	<i>30</i>
<i>Figure 3.1. Diagramme de cas d'utilisation de gestion du site</i>	<i>35</i>
<i>Figure 3.2. Diagramme de cas d'utilisation de gestion des données</i>	<i>36</i>
<i>Figure 3.3. Diagramme de cas d'utilisation de recherche et exportation de données</i>	<i>37</i>
<i>Figure 3.4. Diagramme de classe ShadowPilot</i>	<i>38</i>
<i>Figure 3.5. Diagramme de classe B2B.....</i>	<i>39</i>
<i>Figure 3.6. Diagramme de classe B2C.....</i>	<i>40</i>
<i>Figure 3.7. Diagramme de séquence de Recherche B2B avec Authentification.....</i>	<i>41</i>
<i>Figure 3.8. Diagramme de séquence de Gestion et Monitoring du Site</i>	<i>42</i>
<i>Figure 3.9. Diagramme de séquence de Collecte et Mise à Jour des Données</i>	<i>43</i>
<i>Figure 3.10. Diagramme de séquence de Gestion du Nettoyage des Données</i>	<i>44</i>
<i>Figure 5.1. test des APIs Publiques sur Postman</i>	<i>60</i>
<i>Figure 5.2. Page principale de DataPull.....</i>	<i>78</i>
<i>Figure 5.3. Page de tableau de bord.....</i>	<i>79</i>
<i>Figure 5.4. Page de Recherche personnalisée B2C.....</i>	<i>80</i>
<i>Figure 5.5. Page de recherche géographique B2C.....</i>	<i>81</i>

<i>Figure 5.6. Page de recherche d'emails professionnels</i>	<i>82</i>
<i>Figure 5.7. Page de recherche d'emails d'entreprise</i>	<i>82</i>
<i>Figure 5.8. Page dev Recherche d'entreprises B2B</i>	<i>83</i>
<i>Figure 5.9. détaillée d'entreprise B2B.....</i>	<i>83</i>
<i>Figure 5.10. Page pour trouver les entreprises les mieux notées.....</i>	<i>84</i>
<i>Figure 5.11. Page d'analyse de la réputation et de la fiabilité d'une entreprise</i>	<i>85</i>
<i>Figure 5.12. ETL Dashboard.....</i>	<i>86</i>
<i>Figure 5.13. Visualiseur de données récentes ETL</i>	<i>86</i>
<i>Figure 5.14. Interface de test APIs de ETL</i>	<i>87</i>
<i>Figure 5.15. Gestion et surveillance des tâches ETL</i>	<i>87</i>

Liste des tableaux

<i>Tableau 1.1. Planning prévisionnel.....</i>	<i>12</i>
<i>Tableau 2.1. Benchmark des frameworks backend FastAPI vs Spring boot vs Laravel.....</i>	<i>23</i>
<i>Tableau 2.2. Benchmark des bases de données Postgres vs Elasticsearch vs MongoDB</i>	<i>24</i>
<i>Tableau 2.3. Benchmark frontend.....</i>	<i>25</i>
<i>Tableau 2.4. Comparaison des stacks de technologie.....</i>	<i>26</i>
<i>Tableau 2.5. Benchmark des pipelines ETL Airflow vs NiFi vs Celery/Flask vs AWS Glue – Fonctionnalités.....</i>	<i>27</i>
<i>Tableau 2.6 . Benchmark des pipelines ETL Airflow vs NiFi vs Celery/Flask vs AWS Glue – Métriques.....</i>	<i>27</i>
<i>Tableau 5.1. Types d’instances pour le serveur Windows</i>	<i>66</i>

Table des matières

Introduction Générale.....	1
1. CONTEXTE GÉNÉRAL DU PROJET	2
1.1. Introduction.....	3
1.2. Présentation de l'organisme d'accueil	3
1.2.1. Présentation de Iaweb.dev	3
1.2.2. Clients d'Iaweb.Dev	4
1.2.3. Organigramme d'Iaweb.Dev.....	6
1.3. Vue d'ensemble du projet	6
1.3.1. Problématique	6
1.3.2. Leads, prospects et données clients/entreprises	7
1.3.3. Solution.....	7
1.3.4. Pourquoi choisir DataPull ?	7
1.3.5. Description du projet.....	7
1.4. Méthodologie de travail	8
1.4.1. La méthode SCRUM :	9
1.4.2. Pourquoi SCRUM ?	9
1.4.3. Répartition des rôles	10
1.4.4. L'équipe et Rôles	11
1.4.5. Planning prévisionnel.....	11
1.4.6. Planification du projet :.....	13
1.5. Conclusion	14
2. ÉTUDE DES FONDEMENTS TECHNIQUES ET CHOIX TECHNOLOGIQUES	15
2.1. Introduction.....	16
2.2. Architecture conceptuelle et fondations techniques	16

2.2.1.	Nettoyage et enrichissement des données.....	16
2.2.2.	Développement Backend	17
2.2.3.	Principes UI/UX.....	18
2.2.4.	Web Scraping	19
2.2.5.	Intégration d'API	20
2.2.6.	Processus ETL	21
2.2.7.	Infrastructure Cloud (AWS – EC2, S3)	21
2.3.	Benchmarks technologiques	23
2.3.1.	Benchmark des frameworks backend	23
2.3.2.	Benchmark des bases de données	24
2.3.3.	Benchmark frontend.....	25
2.3.4.	Comparaison des stacks complètes.....	26
2.3.5.	Benchmark des pipelines ETL	26
2.4.	Stacks sélectionnés.....	28
2.4.1.	Première Tech stack de l'application web	28
2.4.2.	Deuxième Tech stack de l'application web	29
2.4.3.	Stack ETL	30
2.5.	Conclusion	30
3.	ANALYSE ET CONCEPTION.....	31
3.1.	Introduction.....	32
3.2.	Langage de modélisation UML	32
3.2.1.	UML :.....	32
3.2.2.	Le rôle d'UML dans la modélisation et la conception orientées objet :...32	
3.2.3.	Résumé :.....	33
3.3.	Diagrammes UML :	34
3.3.1.	Diagramme de cas d'utilisation	34
3.3.2.	Diagramme de classe	38

3.3.3.	Diagramme de séquence	41
3.4.	Conclusion	45
4.	TECHNOLOGIES ET OUTILS TECHNIQUES	46
4.1.	Introduction.....	47
4.2.	Les Framework	47
4.2.1.	Qu'est-ce qu'un Framework ?.....	47
4.3.	Frontend Framework :.....	48
4.3.1.	ReactJs :	48
4.3.2.	CSS Personnalisé :	49
4.4.	Backend Framework :.....	51
4.4.1.	Spring Boot :.....	51
4.4.2.	Maven :	52
4.5.	Base de Données et Recherche	53
4.5.1.	Elasticsearch	53
4.5.2.	PostgreSQL	53
4.6.	Outils Complémentaires.....	54
4.7.	Conclusion	57
5.	IMPLEMENTATION ET MISE EN ŒUVRE.....	58
5.1.	Introduction.....	59
5.2.	Scraping et nettoyage des données : sources et méthodes	59
5.2.1.	APIs publiques (INPI, INSEE)	59
5.2.2.	Jeux de données publics ouverts (DataSoft, INPI)	61
5.2.3.	Scraping web (Selenium, BeautifulSoup).....	62
5.2.4.	Outil de scraping et instances EC2	65
5.2.5.	Nettoyage et enrichissement des données.....	67
5.3.	Développement de l'application web.....	68
5.3.1.	Application Python	68

5.3.2.	Application Java.....	69
5.3.3.	Application Front-end React.....	74
5.3.4.	Développement de l'application ETL	75
5.4.	Résultats et mises à jour futures.....	78
5.4.1.	Application web	78
5.4.2.	Application ETL	86
5.5.	Points à améliorer	88
5.6.	Leçons apprises	88
5.7.	Perspectives d'avenir de DataPull	89
5.8.	Conclusion	90
Conclusion Générale et Perspectives.....		91
Références.....		92

Introduction Générale

Dans un contexte économique marqué par une transformation numérique rapide, la donnée est devenue un actif stratégique central. Les entreprises doivent désormais être capables de collecter, traiter, analyser et valoriser efficacement de grands volumes d'informations provenant de sources diverses. Cette capacité à exploiter des données fiables constitue un levier fondamental pour améliorer la prise de décision, optimiser les processus, anticiper les besoins des clients et renforcer la compétitivité. Pour répondre à ces enjeux, les organisations se tournent vers des plateformes technologiques avancées, capables de gérer des flux complexes et de transformer la donnée brute en information exploitable.

C'est dans cette dynamique que s'inscrit le projet présenté dans ce rapport, réalisé dans le cadre d'un stage de fin d'études au sein de l'agence technologique Iaweb.dev, spécialisée dans les solutions numériques orientées données et intelligence artificielle. Il a porté sur la refonte complète de la plateforme DataPull, un outil web de prospection commerciale automatisée, permettant l'extraction, le nettoyage, l'enrichissement et la visualisation de données d'entreprises issues de sources publiques ou privées. L'objectif était de moderniser l'architecture existante pour améliorer la qualité des données, renforcer les performances, assurer la scalabilité du système et proposer une interface utilisateur plus fluide. Le projet a été mené en suivant la méthodologie agile Scrum et a mobilisé des technologies modernes telles que ReactJS, Spring Boot, Elasticsearch et des outils ETL.

Ce rapport est structuré en cinq chapitres :

- **Chapitre 1 :** Présentation de l'entreprise Iaweb.dev, de ses activités, de son écosystème technologique et du projet DataPull dans son contexte initial.
- **Chapitre 2 :** Étude théorique des concepts techniques liés au projet (traitement de données, scraping, ETL, cloud, UX/UI) et comparaison des technologies envisagées.
- **Chapitre 3 :** Analyse fonctionnelle et modélisation UML à travers les cas d'utilisation, entités métier et interactions entre composants.
- **Chapitre 4 :** Description de l'architecture technique et des outils adoptés pour garantir performance, sécurité et évolutivité.
- **Chapitre 5 :** Présentation de l'implémentation, des résultats obtenus, axes d'amélioration, leçons apprises et perspectives d'évolution de la plateforme.

Chapitre 1 :

CONTEXTE GÉNÉRAL DU PROJET

1.1. Introduction

Ce chapitre présente l'organisation d'accueil, en détaillant ses solutions, ses clients, ses partenaires ainsi que son siège social à l'échelle mondiale. Il expose également les défis liés au projet ainsi que les stratégies proposées pour y faire face.

1.2. Présentation de l'organisme d'accueil

1.2.1. Présentation de laweb.dev

Iaweb.dev, une agence numérique spécialisée rattachée à Havet Digital, a été fondée en 2024 afin de se concentrer sur les solutions orientées données et les applications web basées sur l'intelligence artificielle. Basée à Lille et à Paris, en France, elle s'appuie sur l'expertise de Havet Digital pour fournir des outils innovants tels que la plateforme DataPull, à destination des entreprises en quête de transformation digitale avancée.

Iaweb.dev est animée par un engagement fort en faveur de l'innovation, de la satisfaction client et de l'excellence technique. L'agence cultive un environnement collaboratif axé sur la fourniture de solutions personnalisées et centrées sur les données, afin de renforcer la compétitivité des entreprises. Sa mission repose sur la transparence, l'agilité et une passion pour les technologies de pointe, dans le but de créer des outils numériques percutants générant des résultats concrets et mesurables.

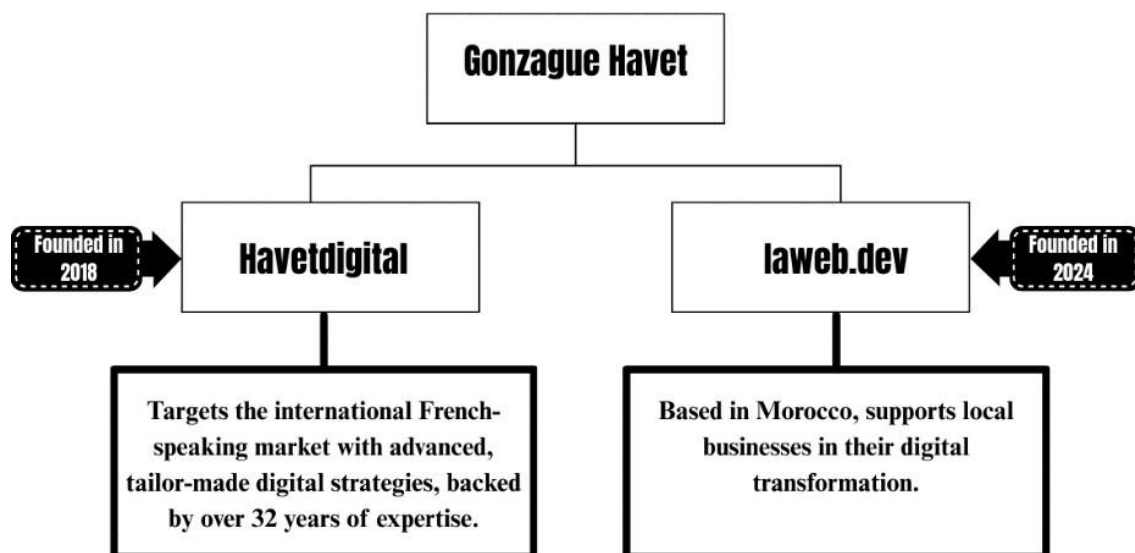


Figure 1.1. Création de laweb.Dev

- **2018** : Havet Digital a été fondée par M. Havet Gonzague, avec des activités basées à Lille et à Paris, en France. L'entreprise se concentre sur la stratégie digitale, le développement web et le marketing. [1]



Figure 1.2. Havet Digital logo

- **2024** : Iaweb.dev a été créé par M. Havet Gonzague en tant que division de Havet Digital, avec une implantation à Marrakech, au Maroc. Elle se spécialise dans les solutions web orientées données et propulsées par l'intelligence artificielle. [2]



Figure 1.3. Iaweb.dev logo

1.2.2. Clients d'Iaweb.Dev

a. Secteurs d'activité ciblés par Iaweb.dev

Iaweb.dev est une agence digitale spécialisée dans le développement web et les solutions intelligentes propulsées par l'IA. Grâce à sa grande flexibilité technologique, l'entreprise est capable d'opérer dans un large éventail de secteurs, tant publics que privés. Positionnée comme un partenaire digital transversal, Iaweb.dev accompagne des organisations de taille moyenne à grande, avec des effectifs allant de 11 à 5 000 employés, en proposant des outils digitaux personnalisés, évolutifs et orientés performance.

- Les principaux secteurs ciblés sont :
 - Institutions publiques et organisations gouvernementales
 - Sociétés de capital-investissement, fonds de capital-risque, incubateurs
 - Bâtiment, immobilier, logistique
 - Finance, assurances, services juridiques
 - Télécommunications et technologies de l'information
 - Commerce de détail, biens de consommation
 - Loisirs, voyages, hôtellerie, salons de beauté, riads
 - Événementiel et divertissement
 - Santé, bien-être, laboratoires pharmaceutiques (par exemple, Sanofi)
 - Médias, communication et agences marketing
 - Services généraux et sociétés de location de voitures

Cette diversité illustre la capacité d'Iaweb.dev à s'adapter aux défis spécifiques de chaque secteur, en proposant des portails projets, des interfaces sur-mesure et des automatisations intelligentes qui améliorent à la fois la visibilité digitale et l'efficacité opérationnelle.

b. Rôles ciblés et profils clients

Les solutions d'Iaweb.dev sont principalement conçues pour les décideurs et les responsables opérationnels chargés de piloter la stratégie digitale, les systèmes informatiques et la communication d'entreprise. L'agence agit comme un partenaire technologique stratégique auprès de ces acteurs clés, les aidant à optimiser les outils digitaux et à accélérer la performance commerciale.

- Profils clés ciblés :
 - **Direction exécutive :**
 - CEO / Fondateur
 - Directeur Général
 - Propriétaire d'entreprise
 - **Rôles en stratégie et innovation :**
 - Responsable de la transformation digitale
 - Directeur digital
 - Chef de projet IT
 - CTO (Chief Technology Officer)
 - **Rôles techniques et informatiques :**
 - CIO (Chief Information Officer)
 - Lead Tech
 - Responsable qualité et processus
 - Responsable des systèmes d'information
 - **Rôles marketing et communication :**
 - Responsable marketing
 - Responsable communication digitale
 - **Rôles financiers :**
 - Directeur financier (CFO)
 - Contrôleur de gestion

Ces acteurs évoluent au sein de PME, d'entreprises de taille moyenne et de grandes entreprises, confrontés à des défis de digitalisation, d'automatisation des processus, de génération de leads et de présence de marque en ligne. Iaweb.dev se positionne comme un

facilitateur stratégique, offrant des solutions personnalisées et à fort impact, adaptées à chaque besoin organisationnel.

1.2.3. Organigramme d'Iaweb.Dev

L'organigramme de notre entreprise d'accueil pour ce stage illustre la structure et la hiérarchie au sein des équipes et des départements dans lesquels nous travaillons. Il met en avant les rôles clés, des superviseurs et mentors aux membres des équipes, et montre le flux de communication ainsi que les responsabilités. Cette structure claire nous aide à comprendre les lignes hiérarchiques, à collaborer efficacement et à aligner nos tâches avec les objectifs globaux de l'organisation.

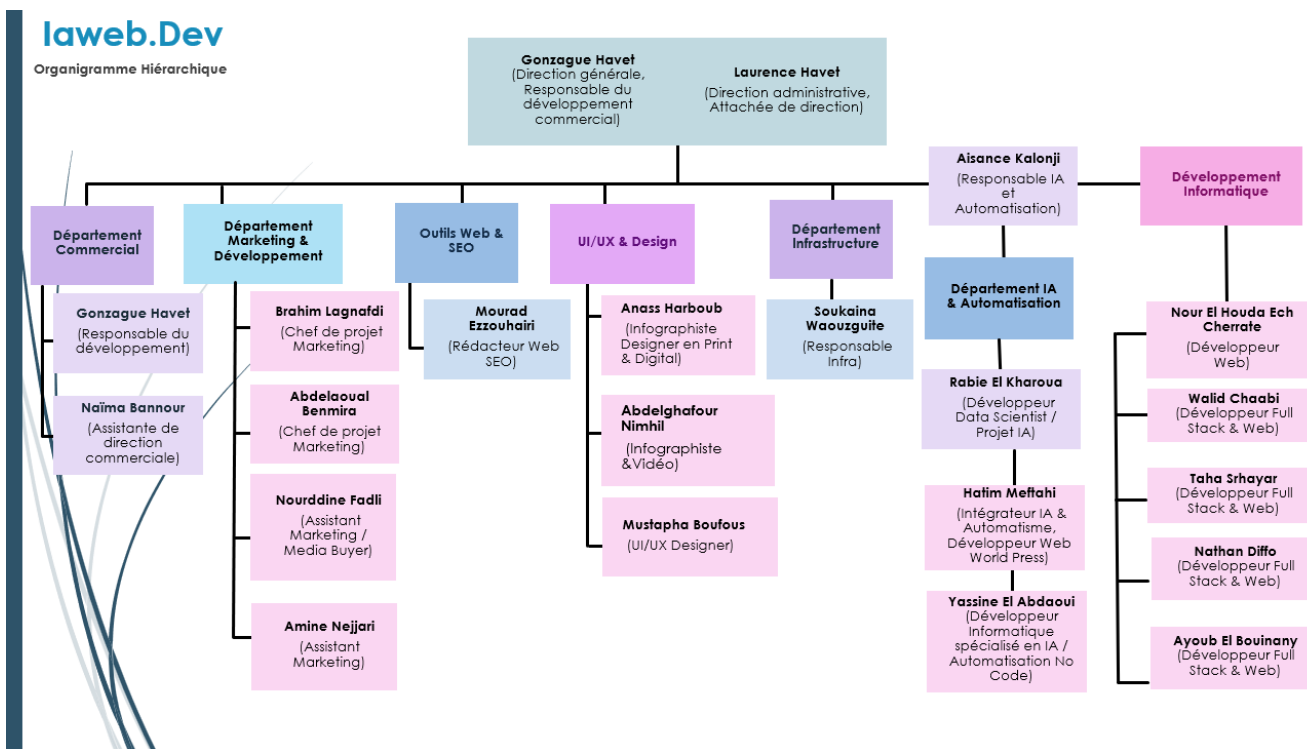


Figure 1.4. Organigramme de Iaweb.dev

1.3. Vue d'ensemble du projet

1.3.1. Problématique

Dans le marché actuel, rapide et compétitif, les entreprises font face à un défi majeur : transformer les données brutes en informations exploitables. Souvent, elles s'appuient sur des bases de données obsolètes, incomplètes ou inexactes, ce qui entraîne une prospection inefficace et un ciblage médiocre. Les méthodes traditionnelles de génération de leads sont non seulement chronophages, mais aussi peu évolutives et manquent de précision. De plus,

l'absence d'outils intégrés pour nettoyer, enrichir et organiser les données clients limite la capacité à lancer des campagnes marketing et commerciales efficaces. Cette fragmentation se traduit par des taux de conversion faibles, des opportunités perdues et des ressources gaspillées

1.3.2. Leads, prospects et données clients/entreprises

Au cœur de toute stratégie d'acquisition client réussie se trouve la capacité à comprendre et gérer les données relatives aux leads et prospects. Un lead représente un client potentiel ayant manifesté de l'intérêt, tandis qu'un prospect est un lead qualifié selon des critères précis. La gestion de ce pipeline nécessite un accès à des informations de qualité, structurées et à jour : profils d'entreprise, coordonnées, segmentation sectorielle et données comportementales. Une collecte, un nettoyage et un enrichissement rigoureux de ces données permettent aux entreprises de mieux cibler leur prospection, personnaliser leurs messages et prendre des décisions éclairées pour stimuler leur croissance. À l'inverse, des données inexacts ou non structurées nuisent fortement à ces efforts.

1.3.3. Solution

Dans un monde où les données sont omniprésentes, les transformer en informations exploitables est essentiel pour rester compétitif. DataPull incarne cette approche en tant qu'expert en analyse de données. Notre mission est d'extraire, traiter et analyser les vastes ensembles de données de DataPull afin de générer des données précieuses et améliorer la prise de décision stratégique.

1.3.4. Pourquoi choisir DataPull ?

- Expertise inégalée : En tant que spécialistes de l'analyse de données, nous apportons un savoir-faire unique pour transformer les données en opportunités concrètes.
- Approche personnalisée : Chaque solution que nous proposons est adaptée aux besoins spécifiques de DataPull, garantissant une valeur ajoutée maximale.
- Résultats concrets : Nos analyses approfondies fournissent des résultats tangibles et exploitables, permettant à DataPull de prendre des décisions éclairées pour l'avenir.

1.3.5. Description du projet

Notre projet est une plateforme web en mode Platform-as-a-Service (PaaS) conçue pour aider les entreprises à identifier et gérer leurs clients potentiels. La plateforme se spécialise dans la génération de leads en fournissant des informations précises sur les prospects, y compris des adresses email vérifiées et des données détaillées sur les entreprises.

Équipé d'outils puissants de recherche et de filtrage, le service permet aux utilisateurs d'explorer et segmenter efficacement leur audience cible. Il intègre également des fonctionnalités robustes pour organiser et gérer les leads et prospects, assurant un processus de contact client fluide et efficace.

Pour améliorer la prise de décision et la performance, la plateforme intègre des capacités avancées d'IA et de machine learning pour l'analyse des données, le scoring des leads et la prédiction des tendances. Des visualisations de données interactives permettent aux utilisateurs d'obtenir des données claires et de suivre facilement l'évolution de leurs campagnes.

En plus de ses fonctionnalités de base, la plateforme propose des options avancées qui augmentent significativement la qualité des leads et les taux de conversion. Les utilisateurs bénéficient d'options de ciblage précises pour identifier et prioriser les prospects à fort potentiel selon des critères spécifiques. Cette segmentation fine permet aux entreprises de concentrer leurs efforts là où ils sont les plus efficaces, optimisant ainsi l'allocation des ressources et la performance des campagnes.

Outre le nettoyage et l'enrichissement des datasets, la plateforme permet aux utilisateurs de structurer leurs données grâce à des filtres interactifs et personnalisables. Cela facilite la navigation et améliore la capacité à extraire des informations exploitables. Grâce à une automatisation intégrée, elle supporte des séquences de prospection personnalisées et des messages ciblés, augmentant encore les chances d'engagement et de conversion.

En combinant des données précises et actualisées avec une segmentation intelligente et des outils de communication, la plateforme permet aux entreprises de faire évoluer leur stratégie d'acquisition client. Elle rationalise l'ensemble du processus, de l'identification du suspect à la maturation du lead, ce qui se traduit par des pipelines plus efficaces et une croissance mesurable des taux de conversion.

1.4. Méthodologie de travail

Respecter les délais du projet est une priorité absolue pour toute équipe de développement logiciel. L'un des défis les plus fréquents lors de la mise en œuvre est la survenue de changements imprévus dans les exigences. Ces changements peuvent perturber le flux de travail, créer une pression au sein de l'équipe et entraîner des retards dans la livraison du projet.

Comme dans tout processus de fabrication, une approche clairement définie et structurée est essentielle. Pour minimiser ces risques et conserver de la flexibilité, nous avons choisi d'adopter la méthodologie Agile pour la gestion de notre projet

1.4.1. La méthode SCRUM :

La méthode Scrum est un Framework agile utilisé dans le développement logiciel pour organiser et gérer les projets de manière itérative et collaborative. Basée sur des principes de transparence, d'inspection et d'adaptation, Scrum favorise une approche flexible et réactive qui permet aux équipes de répondre efficacement aux changements et aux défis tout au long du processus de développement.

1.4.2. Pourquoi SCRUM ?

Scrum est de très loin la méthodologie la plus utilisée parmi les méthodes Agile existantes. Elle est donc la plus éprouvée, documentée et supportée. Livres, blogs, formations, vidéos, associations, conférences traitant de Scrum ne manquent pas et bon nombre de ces ressources sont accessibles gratuitement. On pourrait pratiquement parler d'un standard Agile. Un autre atout important : **Scrum** est simple à comprendre. Sa maîtrise est en revanche difficile.

Les experts de Scrum, même ses fondateurs, le décrivent comme un « cadre de travail permettant de répondre à des problèmes complexes et changeants tout en livrant de manière productive et créative des produits de la plus grande valeur possible » Scrum propose un modèle de contrôle de processus basé sur l'empirisme. Il s'appuie sur trois piliers.

- **La transparence** : Scrum met l'accent sur le fait d'avoir un langage commun entre l'équipe et le management, qui doit permettre à tout observateur d'obtenir rapidement une bonne compréhension du projet.
- **L'inspection** : Scrum propose de faire le point sur les différents artéfacts produits à intervalle régulier, afin de détecter toute variation indésirable.
- **L'adaptation** : Si une dérive est constatée pendant l'inspection, le processus doit alors être adapté. Scrum fournit des rituels, durant lesquels cette adaptation est possible. Il s'agit de sprint planning, de Daily scrum, et de sprint review que va détailler dans la section suivante.

La figure suivante présente le cycle de vie de la méthodologie SCRUM :

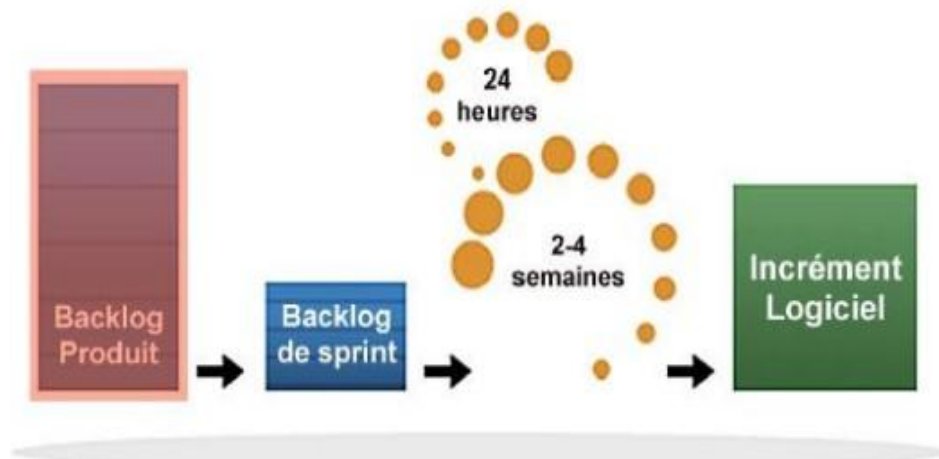


Figure 1.5. Processus de Développement Agile avec Scrum

1.4.3. Répartition des rôles

L'équipe SCRUM possède toutes les compétences nécessaires à la réalisation du projet, elle est donc auto-organisée et peut choisir la meilleure façon d'accomplir sa tâche.

En procédant ainsi, la productivité, la flexibilité et la créativité de l'équipe sont optimisées. L'équipe Scrum se compose de :

- Le **Product Owner** porte la vision du produit à réaliser. Il travaille en interaction avec l'équipe de développement qui suit ses instructions. Il est celui qui établit la priorité des fonctionnalités à développer ou à corriger, et il valide les fonctionnalités terminées. Il est le responsable de la gestion du **Product Backlog**. Dans notre projet, le **Chef du projet** a été désigné **Product Owner** du projet.
- Le **Scrum Master** est responsable de la compréhension, de l'adhésion et de la mise en œuvre de la méthode Scrum. Il veille à ce que les valeurs de la méthodologie et ses principes soient respectés. Il a un rôle de facilitateur aidant à l'amélioration de la communication au sein de l'équipe et cherchant à maximiser le savoir-faire et la productivité de celle-ci. C'est en quelque sorte le coach de l'équipe de développement.
- La **Scrum Team**, l'équipe de développement, est chargée de transformer les besoins définis par le **Product Owner** en fonctionnalités utilisables. Pluri-disciplinaire, elle possède en interne toutes les compétences nécessaires à réalisation du projet. La notion de hiérarchie est absente, les décisions sont prises collégialement.

1.4.4. L'équipe et Rôles

Agile scrum process diagram

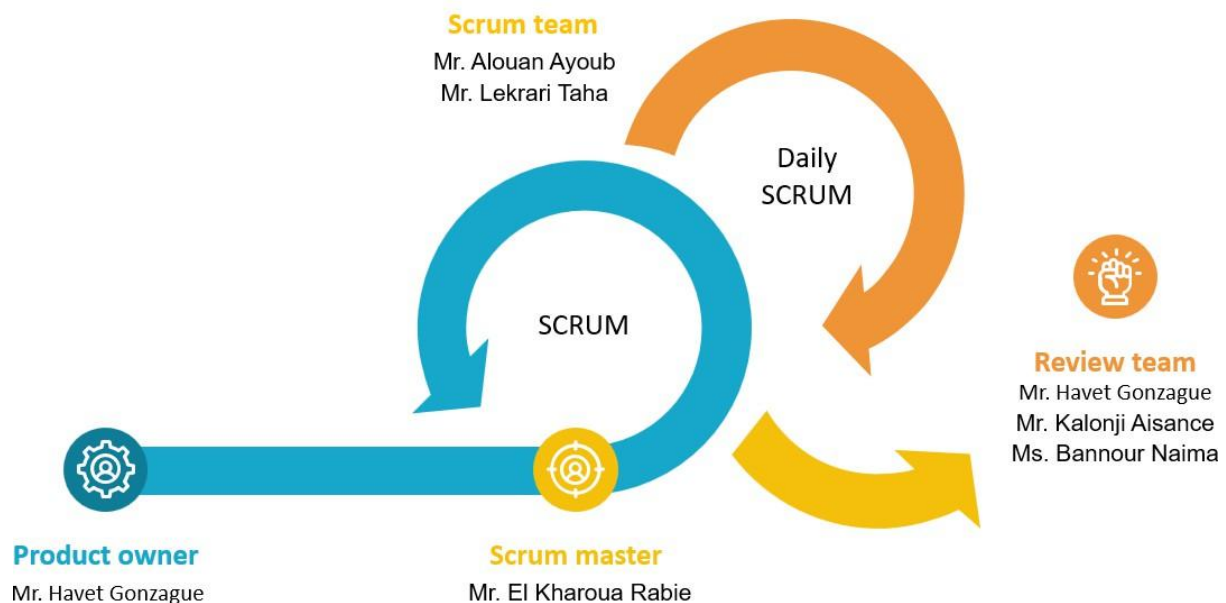


Figure 1.6. Diagramme du processus Agile Scrum

1.4.5. Planning prévisionnel

Pour assurer la bonne organisation et le suivi du travail, nous avons établi le planning prévisionnel suivant afin de garantir une gestion efficace du projet.

Le tableau 1.1 représente les délais correspondants pour chaque tâche du projet. Ce planning détaillé nous permettra de visualiser et de respecter les échéances fixées, en veillant à ce que chaque étape soit réalisée dans les délais impartis. Nous avons pris en compte les différentes contraintes et priorités pour établir ce calendrier, en mettant l'accent sur la coordination des ressources et des activités.

Grâce à ce planning, nous serons en mesure de suivre de près l'avancement du projet, d'identifier d'éventuels retards ou écarts par rapport aux objectifs initiaux et de prendre les mesures nécessaires pour les résoudre. Il constituera également un outil précieux pour la communication et la collaboration entre les membres de l'équipe, en assurant une compréhension commune des échéances et des attentes.

Tableau 1.1. Planning prévisionnel

SPRINT	OBJECTIFS	TÂCHES	Estimation
Sprint 0	Préparation et cadrage du projet	- Analyse de la problématique métier- Collecte des besoins fonctionnels- Étude des solutions existantes et des sources de données- Rédaction du cahier des charges- Mise en place de l'environnement de développement	15 jours
Sprint 1	Étude technique et choix d'architecture	- Benchmark des stacks techniques- Choix des frameworks frontend, backend, ETL, base de données- Configuration initiale des projets (ReactJS, Spring Boot, Python ETL) - Maquettage et préparation des modèles de données	15 jours
Sprint 2	Développement du module de scraping et ETL	- Collecte via APIs (INSEE, INPI, DataSoft) - Développement du scraping avec Selenium et BeautifulSoup- Nettoyage et enrichissement des données (normalisation, standardisation, suppression des doublons) - Déploiement EC2/S3	28 jours
Sprint 3	Backend Spring Boot et Intégration Elasticsearch	- Création des entités JPA- Développement des API RESTful pour B2B/B2C- Intégration avec PostgreSQL et Elasticsearch- Implémentation des endpoints de recherche, visualisation, scoring	25jours
Sprint 4	Développement de l'interface utilisateur	- Création des composants ReactJS pour les pages principales : tableau de bord, recherche B2B/B2C, visualisations- Intégration des API via axios/fetch- Design responsive avec Bootstrap et CSS personnalisée	25 jours

SPRINT	OBJECTIFS	TÂCHES	Estimation
Sprint 5	Mise en place des pipelines ETL	- Intégration de Celery + Flask pour les tâches asynchrones- Planification avec Celery Beat- Monitoring avec Flower- Test de robustesse sur grands volumes	20 jours
Sprint 6	Tests, sécurité et déploiement	- Tests unitaires avec JUnit, Mockito (backend), PyTest (ETL)- Implémentation de la sécurité avec JWT- Déploiement sur AWS (EC2 + S3) - Conteneurisation avec Docker	10 jours
Sprint 7	Documentation et présentation finale	- Rédaction de la documentation technique et utilisateur- Création de démonstrations Power BI- Présentation des résultats, axes d'amélioration et perspectives d'évolution	5 jours

1.4.6. Planification du projet :

La planification est un élément clé de l'avant-projet. Elle implique de prévoir la chronologie des étapes et d'estimer leurs délais respectifs.

Pour la modélisation du déroulement de mon projet, nous avons utilisé le diagramme de GANTT, qui est un outil de planification et d'ordonnancement des tâches sur une période donnée. Grâce à une visualisation facile et pratique des actions planifiées, il nous a permis de suivre et de contrôler le déroulement de chacune des étapes de notre projet et, en outre, de reproduire graphiquement les tâches, leur durée et leur séquence, ainsi que le calendrier général de ce projet.

Pour bien structurer le travail et suivre l'avancement du projet à toutes ses étapes, le projet a été séparé en plusieurs tâches, puis en plusieurs sous-tâches.

Le diagramme de Gantt ci-dessous présente la planification de toutes ces tâches pendant la période de stage.

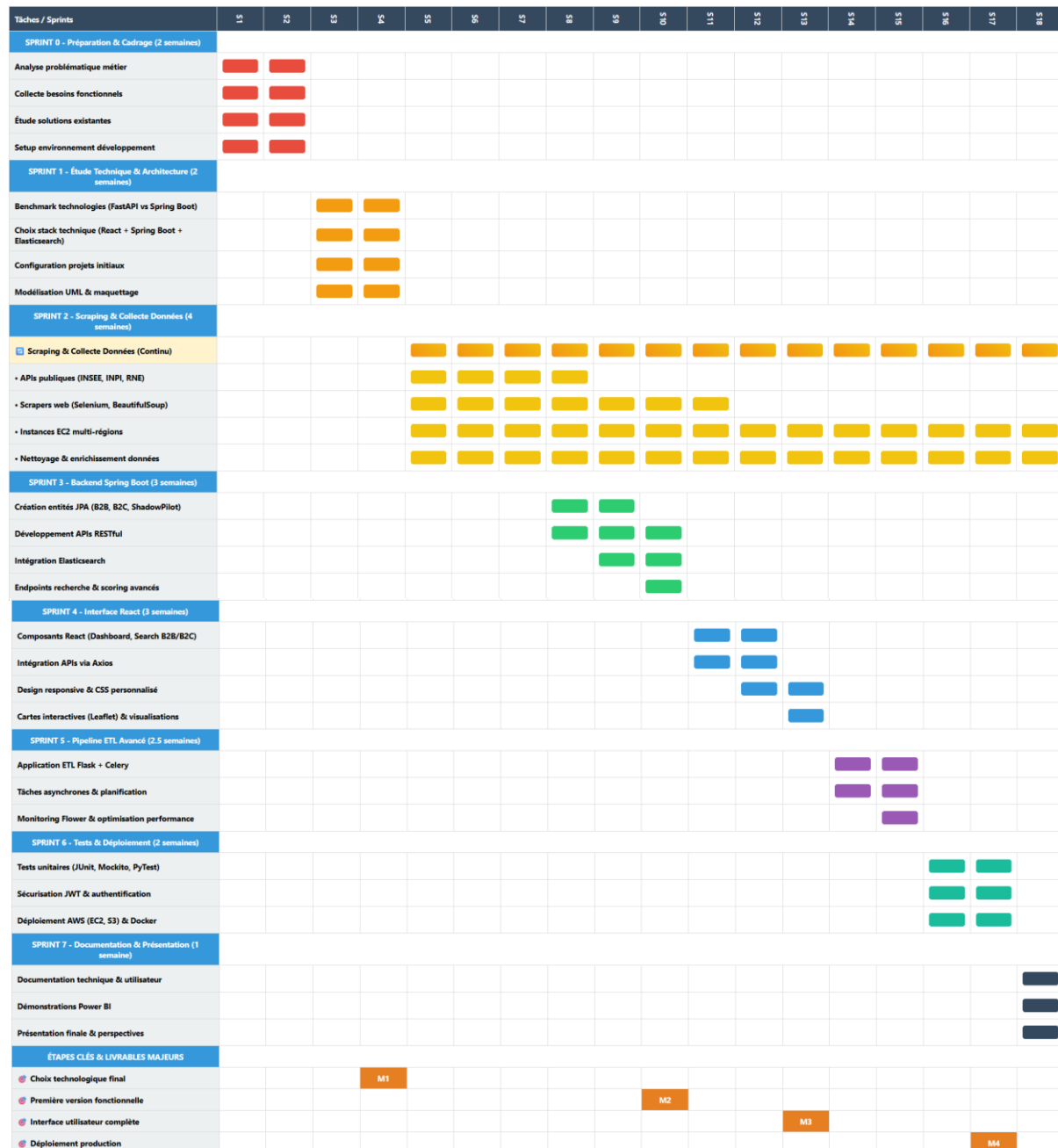


Figure 1.7. Diagramme de GANTT

1.5. Conclusion

Dans ce chapitre, nous avons découvert l'organisation d'accueil et ses clients. Nous avons également présenté le projet, sa problématique, sa description ainsi que la méthodologie de développement adoptée, à savoir Scrum

Chapitre 2 :

ÉTUDE DES FONDEMENTS TECHNIQUES ET CHOIX TECHNOLOGIQUES

2.1. Introduction

Dans cette seconde partie, nous proposons une étude théorique des concepts techniques sous-jacents au développement de la plateforme DataPull. Elle a pour objectif de présenter les fondements technologiques sur lesquels repose le projet, tels que les principes du traitement de données, le web scraping, les architectures backend modernes, l'expérience utilisateur (UX/UI), ou encore les services cloud.

Cette étude est accompagnée d'une analyse comparative de différentes solutions technologiques : frameworks frontend et backend, bases de données, moteurs de recherche, outils ETL, et stacks applicatifs. L'objectif est de justifier les choix retenus en s'appuyant sur des critères de performance, de scalabilité, de maintenabilité et de compatibilité avec les besoins métier.

Ces éléments fournissent une base solide pour la mise en œuvre de la solution, en garantissant que les orientations techniques du projet sont à la fois cohérentes et justifiées.

2.2. Architecture conceptuelle et fondations techniques

2.2.1. Nettoyage et enrichissement des données



Figure 2.1. Cycle de nettoyage des données

Le nettoyage et l'enrichissement des données sont des étapes essentielles dans tout projet axé sur les données. Ils constituent la colonne vertébrale du processus de transformation des données brutes en une ressource fiable et exploitable. La figure présentée ci-dessus illustre le « cycle de nettoyage des données », un flux de travail circulaire mettant en évidence les étapes interconnectées impliquées. Ce cycle commence par l'importation des données, suivie de la fusion des ensembles, la reconstruction des données manquantes, la standardisation, la

normalisation, la déduplication, la vérification et l'enrichissement, pour se terminer par l'exportation des données le tout formant une boucle continue d'amélioration.

Le nettoyage vise à détecter et corriger les erreurs ou incohérences au sein des jeux de données. Cela comprend le traitement des valeurs manquantes ou nulles, la correction des formats (par exemple : standardisation des dates ou normalisation des chaînes de caractères), ainsi que la suppression des doublons. Chaque étape du cycle, comme la déduplication ou la normalisation, garantit que les données sont précises et cohérentes, posant ainsi une base solide pour les futures analyses ou automatisations. Par exemple, la standardisation permet d'harmoniser les formats, tandis que la reconstruction des données manquantes comble les lacunes afin de préserver l'intégrité des jeux de données.

L'enrichissement, illustré dans la phase de « vérification et enrichissement » du cycle, consiste à améliorer les données en les complétant avec des informations supplémentaires. Cela peut inclure l'ajout de métadonnées sur les entreprises (par exemple : secteur d'activité ou localisation), la récupération d'adresses e-mail à partir de sources diverses, ou encore la mise en correspondance des données collectées avec des bases standardisées. Ce processus transforme des entrées non structurées en un jeu de données structuré et de grande valeur, facilitant ainsi l'extraction d'insights approfondis. [3]

2.2.2. Développement Backend

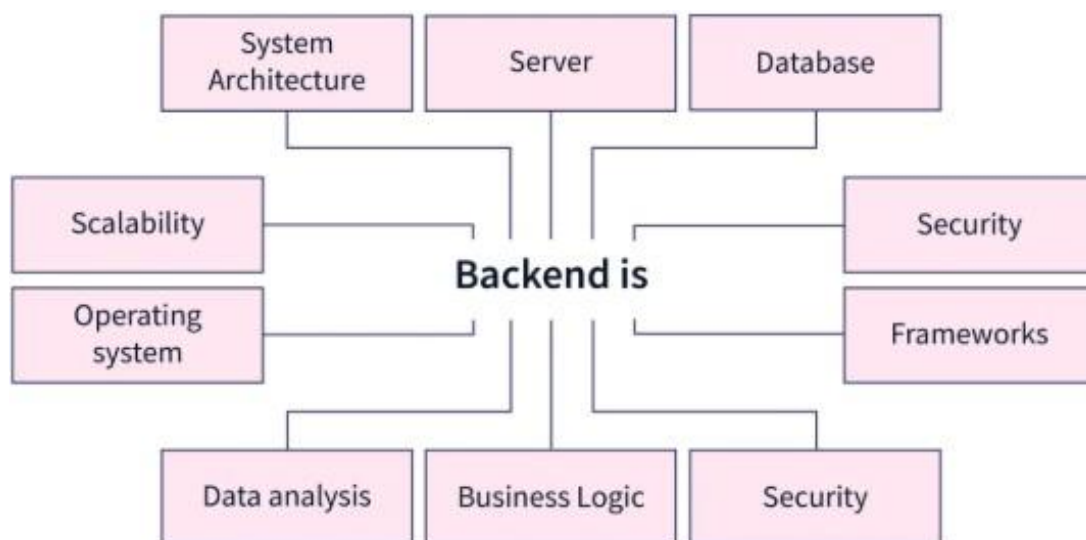


Figure 2.2. Concepts clés du backend

Le développement backend constitue le noyau logique d'un système, chargé de l'automatisation et de la gestion des fonctionnalités côté serveur. Il englobe la création et la maintenance de l'infrastructure serveur, la logique métier, les endpoints d'API, les pipelines de traitement des données, ainsi que les couches de sécurité.

Cela implique un ensemble d'éléments interconnectés : architecture système, serveurs, bases de données, scalabilité, systèmes d'exploitation, sécurité, frameworks, analyses de données, et logique métier, collaborant pour assurer une base robuste.

Dans notre cas, le travail backend a consisté à développer des APIs permettant de transmettre les données nettoyées et enrichies au frontend, orchestrer les scripts de scraping et les processus ETL, et gérer la communication entre microservices ou services cloud comme AWS EC2 et S3.

Un backend bien conçu est scalable, sécurisé et documenté, permettant une intégration fluide avec le frontend et les outils de monitoring. Les frameworks et systèmes d'exploitation fournissent les environnements nécessaires pour concrétiser cette logique.

Au final, le backend transforme les algorithmes théoriques et de scraping en une solution déployable et opérationnelle, comme l'a démontré DataPull. [4]

2.2.3. Principes UI/UX

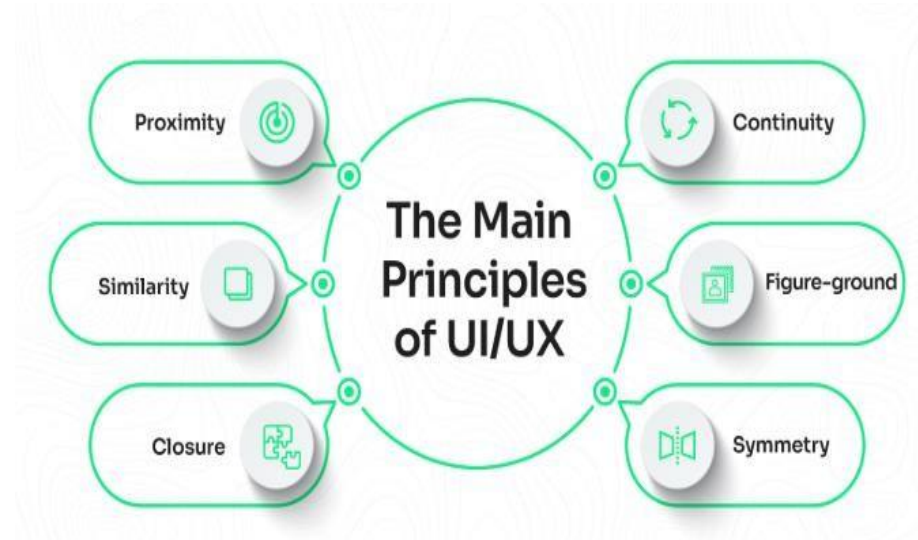


Figure 2.3. Principes UI/UX

Le design UI/UX joue un rôle crucial dans la création d'interfaces intuitives, accessibles et efficaces, garantissant une expérience utilisateur fluide et agréable.

Les principes clés tels que la proximité, la similarité, la clôture, la continuité, le contraste fond/forme et la symétrie sont mobilisés pour concevoir des interfaces à la fois esthétiques et

fonctionnelles. Ces principes guident la disposition logique des éléments, la cohérence visuelle, et aident l'utilisateur à se concentrer sur l'essentiel sans surcharge cognitive.

Une UI bien conçue permet une navigation fluide, une compréhension immédiate des fonctionnalités et une lecture claire des visualisations. L'UX, quant à elle, englobe des aspects plus larges : parcours utilisateur, accessibilité, réactivité, performance perçue, etc., assurant une expérience complète et agréable.

Dans le cadre de notre projet, ces principes ont été appliqués à la conception de tableaux de bord et d'outils facilitant l'interprétation des données extraites et enrichies. Avec DataPull, nous avons conçu des interfaces où l'utilisateur peut consulter les métadonnées d'entreprises ou les coordonnées de contacts, de manière simple et intuitive.

Au-delà de l'aspect visuel, l'optimisation des performances a été une priorité : réduction du temps de chargement, amélioration de la vitesse d'affichage, adaptation mobile, etc. Même si le projet était fortement orienté backend, l'intégration des bonnes pratiques UI/UX a permis de proposer une solution complète et agréable à utiliser. [5]

2.2.4. Web Scraping

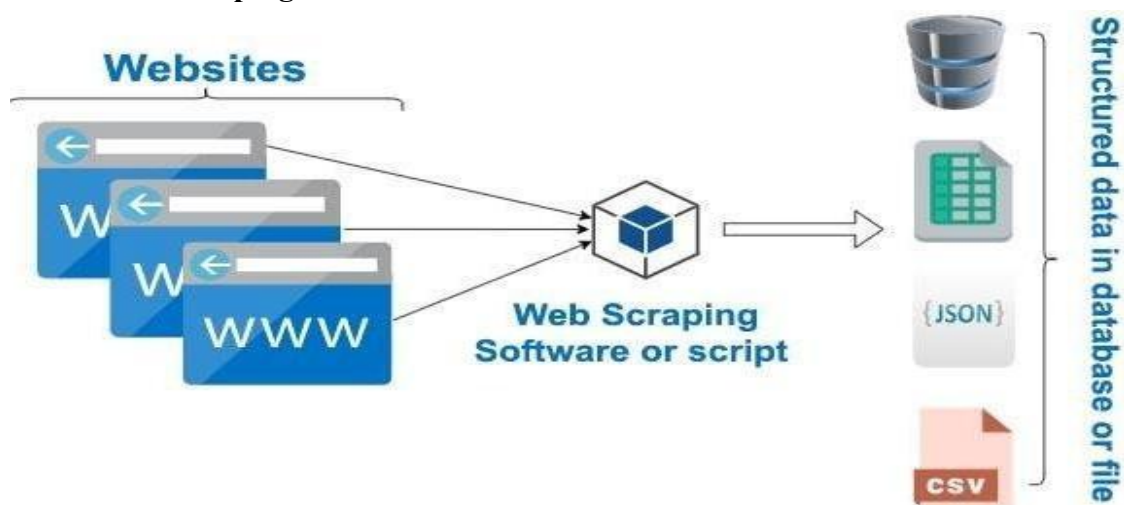


Figure 2.4. Web scraping

Le web scraping désigne le processus d'extraction automatisée de données structurées à partir de sites web à l'aide d'outils ou de scripts.

Dans notre projet, le scraping a été une technique centrale permettant de collecter de grandes quantités de données publiques d'entreprises : noms, descriptions, coordonnées, etc.

Il s'appuie sur des requêtes HTTP envoyées aux pages web, dont le contenu HTML est analysé via des bibliothèques comme BeautifulSoup, Selenium ou Scrapy.

Cependant, cette pratique implique des responsabilités légales et éthiques : respect du fichier robots.txt, limitation de la fréquence des requêtes pour éviter la surcharge des serveurs,

conformité au RGPD [6] (en particulier lors de la collecte d'e-mails ou de numéros de téléphone). Ces obligations ont guidé la conception de notre logique de scraping et l'instauration de limites de fréquence.

2.2.5. Intégration d'API

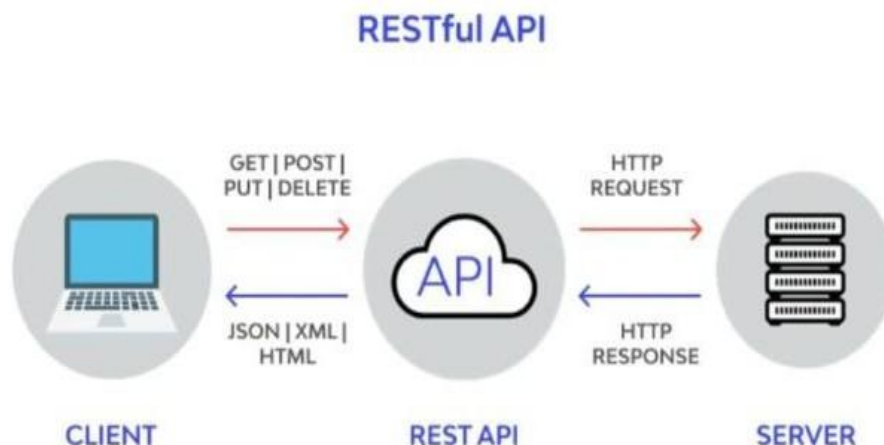


Figure 2.5. RESTful API

Les APIs (Interfaces de Programmation d'Applications) sont des outils clés du développement moderne, permettant aux systèmes d'échanger des données de manière structurée et sécurisée.

Dans notre projet, l'intégration d'APIs a complété le scraping en offrant un moyen d'obtenir des données structurées et fiables directement depuis des plateformes tierces.

Nous avons principalement utilisé des APIs REST, accédées via des méthodes HTTP (GET, POST...). Certaines nécessitaient une authentification par clé API ou OAuth token.

L'usage d'APIs a permis d'automatiser une partie du traitement, de limiter les risques de blocage par scraping, et d'accéder à des données de meilleure qualité. Cela nous a également permis de maîtriser la gestion des limites de requêtes, la pagination et le traitement des erreurs, tout en assurant la régularité des flux de données. [7]

2.2.6. Processus ETL



Figure 2.6. ETL process

Le processus ETL (Extract, Transform, Load) est un modèle standard de pipeline de données permettant de transférer des données depuis des sources vers des formats exploitables.

Dans notre projet, l'ETL a constitué le socle fondamental de la gestion des données brutes collectées via le scraping et les appels API.

- **Extract** : récupération des données depuis les pages web, APIs ou autres sources.
- **Transform** : nettoyage, formatage, déduplication et enrichissement des données pour garantir leur qualité.
- **Load** : enregistrement des données traitées dans des formats structurés (fichiers, bases de données, cloud...).

La mise en place de pipelines ETL nous a permis de traiter de gros volumes de données d'entreprises efficacement, d'automatiser nos workflows, et de garantir la cohérence des résultats. [8]

2.2.7. Infrastructure Cloud (AWS – EC2, S3)



Figure 2.7. Infrastructure cloud - AWS

L'infrastructure cloud désigne l'ensemble des composants matériels, logiciels, réseaux et services qui permettent la mise à disposition de ressources informatiques telles que le stockage, la puissance de traitement et les applications via Internet. Elle est généralement fournie par des

prestataires de services cloud (par exemple : AWS, Microsoft Azure, Google Cloud) et inclut des machines virtuelles, des serveurs, des bases de données, des équilibreurs de charge et des systèmes de stockage. Cette infrastructure est conçue pour être scalable (extensible), flexible et accessible à la demande, permettant aux utilisateurs de déployer et de gérer des applications ou des données sans posséder d'équipements physiques ni mettre en place des installations sur site.

Le déploiement et la gestion d'une infrastructure cloud ont été essentiels pour assurer la scalabilité et la fiabilité de notre solution. Nous nous sommes appuyés sur Amazon Web Services (AWS), en particulier sur EC2 (Elastic Compute Cloud) et S3 (Simple Storage Service).

Les instances EC2 ont hébergé nos scripts de scraping, nos services backend ainsi que nos environnements de test, offrant une plateforme serveur personnalisable et extensible. Le service S3 a été utilisé pour stocker les données brutes et les ressources, dans un format à la fois hautement durable et facilement accessible.

Le recours à l'infrastructure cloud a supprimé le besoin d'hébergement local, tout en nous permettant de planifier des tâches de scraping récurrentes, de surveiller la santé du système et d'effectuer des sauvegardes.

L'apprentissage de la gestion des services EC2 et S3 nous a permis de nous familiariser avec les concepts clés du DevOps et d'acquérir une expérience pratique dans le déploiement de systèmes réels dans des environnements proches de la production. [9]

2.3. Benchmarks technologiques

Avant de lancer tout projet de données ou projet informatique en général, il est essentiel de choisir les outils et frameworks qui répondent à nos besoins tout en garantissant un produit performant.

2.3.1. Benchmark des frameworks backend

Nous nous sommes appuyés sur les critères listés dans le tableau suivant pour comparer trois frameworks et identifier celui qui convient le mieux à notre cas d'usage. [10]

Tableau 2.1. Benchmark des frameworks backend FastAPI vs Spring boot vs Laravel

Fonctionnalité	FastAPI	Spring Boot	Laravel
Langage	Python	Java	PHP
Performance	Très bonne – asynchrone par défaut, utilise Starlette et Pydantic	Excellente – optimisations JVM matures, support de la programmation réactive	Bonne – suffisante pour la majorité des applications web
Courbe d'apprentissage	Basse à moyenne – lisibilité de Python, syntaxe intuitive	Moyenne à élevée – verbosité de Java, écosystème complexe	Basse – adapté aux débutants, excellente documentation
Vitesse de développement	Rapide	Moyenne	Rapide
Sûreté des types	Typage statique optionnel avec Pydantic	Typage statique strict	Faible – typage dynamique
Écosystème	En croissance, packages Python	Vaste, mature	Large, solide écosystème PHP
Scalabilité	Bonne – support asynchrone, mais limité par le GIL	Excellente – gestion des threads, modèles de scalabilité matures	Modérée – mieux adaptée aux architectures monolithiques
Adoption en entreprise	En croissance	Très élevée – standard industriel	Élevée – très utilisée pour les CMS et les backends
Support microservices	Disponible – nécessite des outils supplémentaires	Excellente – support intégré	Disponible – via Laravel Octane, Horizon, outils API

2.3.2. Benchmark des bases de données

Le tableau ci-dessous met en évidence les fonctionnalités que nous avons évaluées pour comparer les trois solutions de base de données et identifier celle qui correspond le mieux à nos besoins. [11]

Tableau 2.2. Benchmark des bases de données Postgres vs Elasticsearch vs MongoDB

Fonctionnalité	Postgres	Elasticsearch	MongoDB
Type	Base de données relationnelle SQL	Moteur distribué de recherche et d'analytique	Base de données NoSQL orientée documents
Cas d'usage principal	Stockage structuré généraliste	Recherche plein texte, analytique, logs	Stockage flexible de documents, scalabilité
Conformité ACID	Conforme ACID complet	Cohérence éventuelle par défaut	Non conforme ACID complet
Langage de requête	SQL	Requête en JSON	Requête en JSON (API MongoDB)
Performance en lecture	Très bonne – indexation, optimisation	Excellente pour recherche et récupération	Bonne – optimisé pour les documents
Performance en écriture	Excellente pour données structurées	Bonne – optimisée pour cohérence éventuelle	Bonne – optimisée pour débit élevé
Scalabilité	Scalabilité verticale, horizontale via extensions	Scalabilité horizontale native	Excellente – sharding, scalabilité horizontale
Flexibilité du schéma	Semi-rigide, JSONB pour flexibilité	Sans schéma, dynamique, mapping	Très flexible, schémas dynamiques
Support transactionnel	Excellent	Limité	Limité (transactions mono-document)
Recherche plein texte	Bonne	Excellente – cas d'usage principal	Bonne – via indexation de texte
Capacités analytiques	Bonnes avec extensions	Excellentes – agrégation, fonctions d'analyse	Bonnes – pipeline d'agrégation
Intégrité des données	Excellente – contraintes, intégrité référentielle	Limitée – pas d'intégrité référentielle native	Limitée – pas d'intégrité référentielle

2.3.3. Benchmark frontend

Afin de garantir le bon choix du Frontend, nous avons basé notre comparaison sur les fonctionnalités clés détaillées dans le tableau ci-dessous. [12]

Tableau 2.3. Benchmark frontend

Fonctionnalité	Vanilla JS/CSS/HTML	React	Angular	Vue.js
Courbe d'apprentissage	Faible	Moyenne – JSX et hooks	Élevée – TypeScript et concepts avancés	Faible à moyenne – syntaxe simple
Vitesse de développement	Lente	Rapide – composants réutilisables	Moyenne – structure imposée	Rapide – flexible et simple
Performance	Bonne – sans surcharge	Bonne – DOM virtuel	Bonne – peut se dégrader à grande échelle	Bonne – réactive, empreinte légère
Réutilisation de composants	Limitée	Excellente – concept fondamental	Bonne – architecture modulaire	Excellente – support intégré
Gestion d'état	Manuelle	Intégrée (useState, etc.)	Services + bibliothèques (NgRx)	Intégrée (VueX / Pinia)
Expérience développeur	Basique	Excellente – outils dev	Bonne	Excellente – simplicité, devtools
Maintenabilité	Difficile à grande échelle	Meilleure – hooks et patterns	Solide – structure par conception	Bonne – modulaire et intuitive
Communauté et support	Massive (standards web)	Massive – écosystème React	Large – support entreprise	Large et en croissance
Développement mobile	Non applicable	React Native	Ionic (intégration mature)	NativeScript, Quasar

2.3.4. Comparaison des stacks complètes

Nous avons utilisé les critères présentés dans le tableau ci-dessous pour évaluer et comparer quatre solutions de piles technologiques en fonction de nos besoins spécifiques, de nos ressources et de nos capacités. [13]

- **Stack 1** : Spring Boot + Elasticsearch + React
- **Stack 2**: FastAPI + PostgreSQL + JS/CSS/HTML
- **Stack 3** : Laravel + MySQL + Vue.js
- **Stack 4** : Express.js + MongoDB + Angular

Tableau 2.4. Comparaison des stacks de technologie

Métrique	Stack 1	Stack 2	Stack 3	Stack 4
Requêtes/sec	20K–30K	40K–50K	8K–15K	15K–25K
Mémoire utilisée	400–800MB	150–300MB	200–400MB	100–250MB
Vitesse des requêtes SQL	5–10ms	2–5ms	3–8ms	1–4ms
Temps de réponse de recherche	20–50ms	50–100ms	100–200ms	80–120ms
Temps de démarrage	15–40s	1–3s	2–6s	1–2s
Stars GitHub	65K / 55K / 195K	55K / 25K / N/A	29K / 65K / 200K	60K / 25K / 80K
Vulnérabilités	8–12	5–10	10–15	12–18
Prêt de l'équipe	70%	90%	50%	20%

2.3.5. Benchmark des pipelines ETL

Les tableaux suivants présentent les fonctionnalités et les métriques que nous avons jugées essentielles pour comparer toutes les architectures possibles de pipeline dans le cadre de la sélection de l'option la plus adaptée à notre cas d'usage. [14-15]

Fonctionnalités :

Tableau 2.5. Benchmark des pipelines ETL Airflow vs NiFi vs Celery/Flask vs AWS Glue – Fonctionnalités

Fonctionnalité	Apache Airflow	Apache NiFi	Celery + Flask	AWS Glue
Composants	Scheduler, Executor, Web UI, PostgreSQL, Redis	Application standalone	Redis, workers, API Flask	Service AWS entièrement managé
Modèle de dev	DAGs en Python	Interface graphique glisser-déposer	Code Python	Python/Scala
Ordonnancement	Intégré (type cron)	Déclencheurs logiques	Celery Beat	AWS EventBridge
Gestion d'erreurs	Politiques de retry, alertes email	Traçabilité, pression inverse	Retry basique, monitoring manuel	Retry automatique, alertes CloudWatch
Méthode de déploiement	Docker Compose	JAR autonome	Docker Compose	Console AWS ou IaC
Interface utilisateur	Tableau de bord complet	UI riche	Aucune (nécessite dev personnalisé)	Console AWS
Support communauté	Très fort	Fort	Fort (sur les composants)	Documentation AWS
Vitesse de traitement	Rapide	Rapide	Modérée	Rapide

Métriques :

Tableau 2.6 . Benchmark des pipelines ETL Airflow vs NiFi vs Celery/Flask vs AWS Glue – Métriques

Métrique	Apache Airflow	Apache NiFi	Celery + Flask	AWS Glue
Débit (records/sec)	5000	8500	4000	12000
Latence (sec)	45	30	60	25
Utilisation CPU (%)	65%	70%	50%	N/A
Utilisation mémoire (%)	75%	65%	50%	N/A
Taux d'échec (%)	1.2%	0.9%	1.8%	0.7%
Temps de récupération	5–8 min	3–5 min	8–12 min	2–4 min

2.4. Stacks sélectionnés

Deux Stack a été développées et utilisées à des fins de démonstration, mais finalement une a été choisie, voyons d'abord les premières piles mises en œuvre

2.4.1. Première Tech stack de l'application web

Ce premier stack a été développé pour démontrer rapidement les fonctionnalités de DataPull tout en assurant une bonne performance.

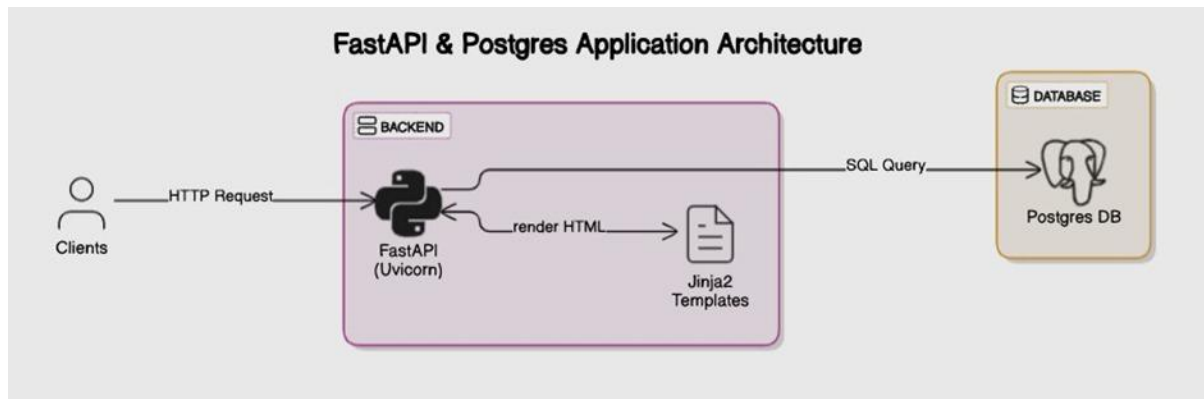


Figure 2.8. Tech stack de l'application web 1

a. Facteurs clés de sélection :

- **Développement rapide** : FastAPI + Python ont permis un prototypage efficace, plus rapide que Spring Boot.
- **Performance des données** : FastAPI (asynchrone) + PostgreSQL (ACID) a bien géré de gros volumes.
- **Stockage structuré** : PostgreSQL (relationnel avec JSONB) pour stocker les données enrichies.
- **Efficacité de l'équipe** : Équipe déjà formée sur Python et SQL.
- **Simplicité UI** : Jinja2 a permis une visualisation HTML simple sans complexité frontend.

2.4.2. Deuxième Tech stack de l'application web

Cette architecture mise à jour, que nous utilisons et perfectionnons actuellement, a remplacé la pile initiale basée sur FastAPI et Postgres afin de mieux répondre aux besoins évolutifs de notre projet DataPull. La décision d'abandonner la première pile au profit de celle-ci a été motivée par les arguments suivants, fondés sur des données de benchmarking.

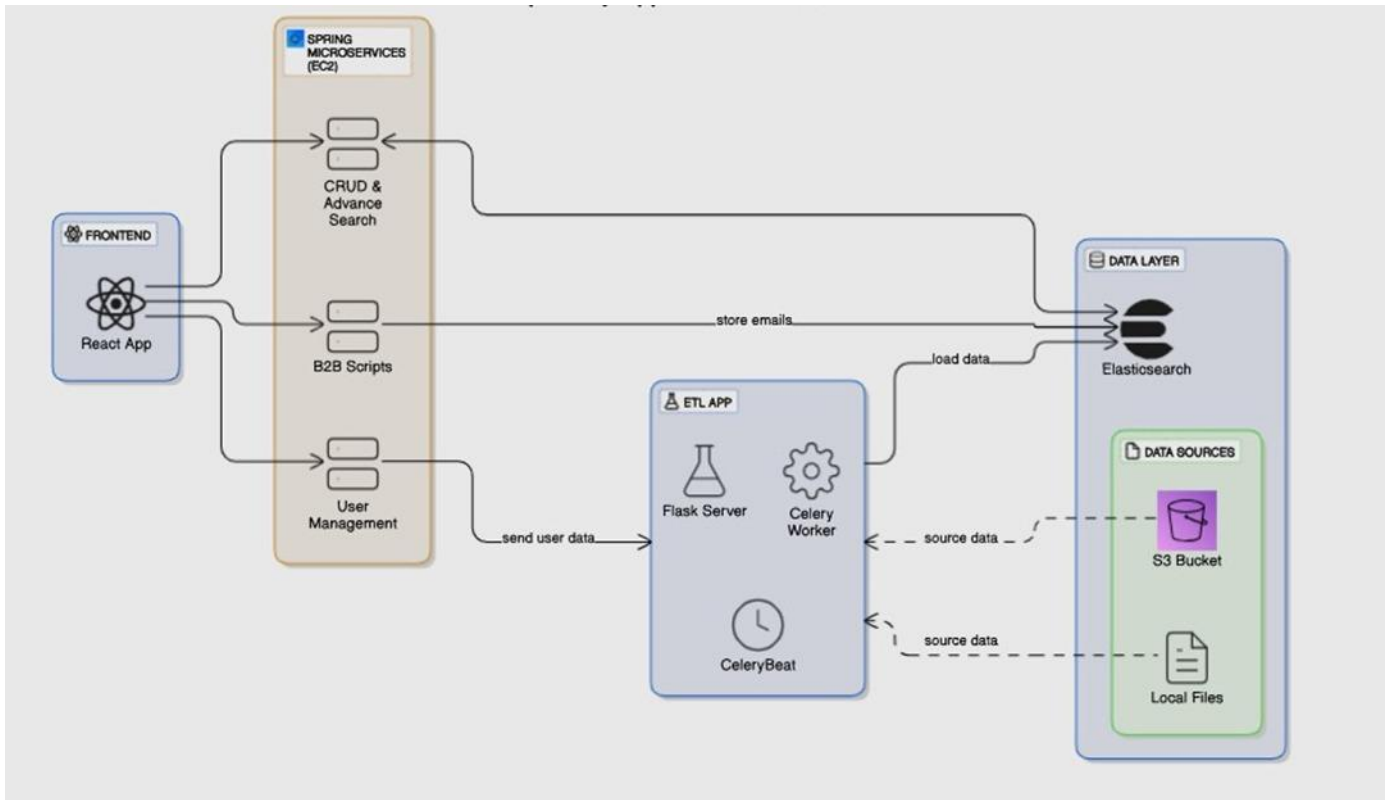


Figure 2.9. Tech stack de l'application web 2

Cette architecture, actuellement utilisée, a remplacé FastAPI/PostgreSQL afin de mieux répondre aux exigences évolutives du projet.

a. Motivations de migration :

- **Scalabilité améliorée** : Spring (microservices) + Elasticsearch (scalabilité horizontale)
- **Recherche avancée** : Elasticsearch (recherche plein texte, agrégations)
- **Préparation entreprise** : Spring (forte adoption), architecture distribuée
- **UI dynamique** : React fournit une interface interactive impossible avec Jinja2

Compromis : Une complexité accrue et une consommation de ressources plus élevée par rapport à la stack légère FastAPI, justifiées par la nécessité de disposer de capacités avancées de recherche, d'analyse et de traitement distribué.

2.4.3. Stack ETL

Sélectionné en fonction de la rapidité de développement, de la facilité de mise en œuvre et de la configurabilité pour les besoins initiaux d'ETL de DataPull.

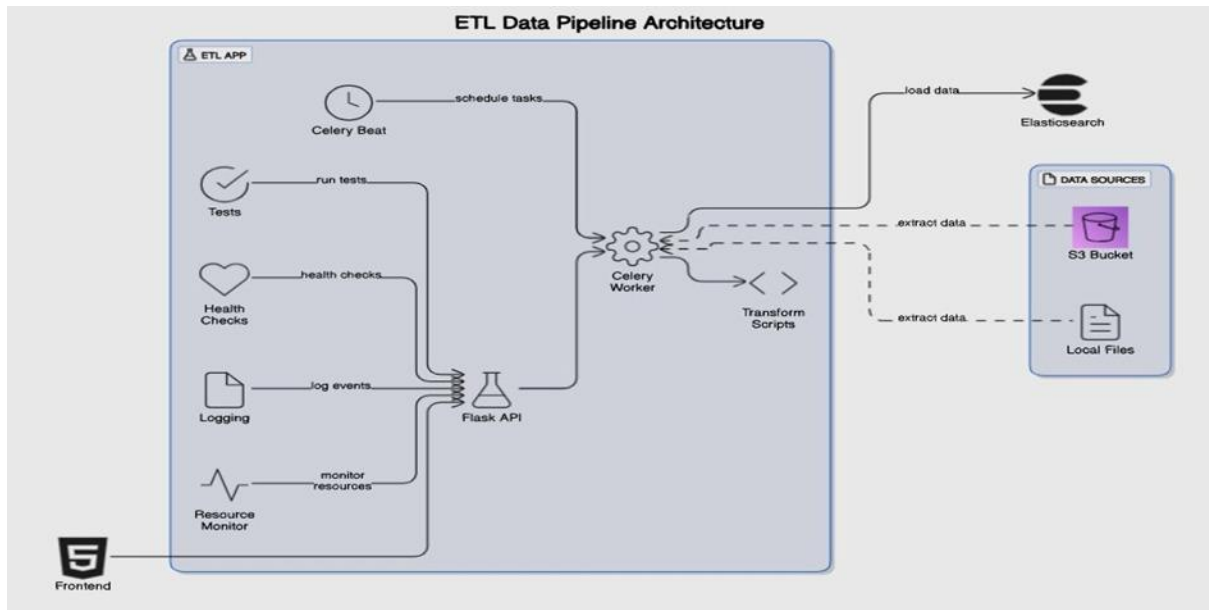


Figure 2.10. application ETL

a. Motivations du choix :

- **Vitesse de dev :** Python plus rapide qu’Airflow ou AWS Glue
- **Facilité d’implémentation :** Courbe d’apprentissage plus faible
- **Contrôle total :** Interface personnalisée et pipeline libre
- **Évolutivité :** Architecture modulaire et flexible
- **Performance acceptable :** Malgré un débit inférieur, Celery + Flask offrait un bon compromis avec un taux d’échec gérable et des temps de récupération raisonnables.

2.5. Conclusion

Ce chapitre a présenté les bases techniques de la plateforme DataPull, en abordant les principaux concepts liés au traitement de données, au backend, à l’UX/UI, au scraping, à l’ETL et au cloud. Une analyse comparative a permis d’évaluer plusieurs solutions technologiques, afin de justifier les choix retenus en termes de performance, scalabilité et maintenabilité. Ces éléments constituent une base fiable pour la mise en œuvre de la solution technique finale

Chapitre 3 :

ANALYSE ET CONCEPTION

3.1. Introduction

Dans ce chapitre, nous présentons l'analyse fonctionnelle du projet DataPull ainsi que la conception du système à l'aide des outils de modélisation UML. Cette étape joue un rôle fondamental dans le cadrage du projet, car elle permet de structurer les besoins, d'identifier les entités métier et de formaliser les principales interactions entre les composants du système.

À travers cette modélisation, nous décrirons les cas d'utilisation majeurs, la structure des objets métier et les flux d'informations, afin de fournir une vision claire du fonctionnement attendu de la plateforme. Les diagrammes produits, notamment les cas d'utilisation, de classes et de séquence, serviront de support pour valider les spécifications fonctionnelles avec les parties prenantes et orienter les choix d'implémentation technique.

3.2. Langage de modélisation UML

3.2.1. UML :

UML, ou Unified Modeling Language, est un langage graphique et visuel standardisé utilisé en génie logiciel pour modéliser, concevoir, spécifier et documenter des systèmes et des processus logiciels. Il fournit une large gamme de symboles graphiques et de diagrammes pour représenter différentes vues et aspects du système, facilitant ainsi la communication, la compréhension et la conception entre les différentes parties d'un projet logiciel. UML est largement utilisé pour la modélisation orientée objet, mais il peut également être appliqué à d'autres modèles de conception et de développement. [16]

3.2.2. Le rôle d'UML dans la modélisation et la conception orientées objet :

Le rôle d'UML dans la modélisation et la conception orientées objet est vaste et crucial. Voici quelques-uns de ses principaux rôles :

- **Communication** : UML fournit un langage commun pour les développeurs, concepteurs et autres parties prenantes impliquées dans le développement logiciel. Il facilite la communication en utilisant des diagrammes et des notations standardisées, ce qui réduit les ambiguïtés et améliore la compréhension entre les membres de l'équipe.
- **Modélisation conceptuelle** : UML permet de capturer les idées et les concepts clés d'un système avant de commencer à coder. Les diagrammes de cas d'utilisation aident à définir les fonctionnalités attendues du système du point de vue de l'utilisateur.
- **Conception structurelle** : UML propose des diagrammes de classes pour modéliser la structure du système en définissant ses classes, attributs, méthodes et relations entre les

classes. Les diagrammes de composants et de packages aident à organiser les éléments du système en modules logiques.

- Conception comportementale : Les diagrammes d'états, d'activités et de séquences aident à modéliser le comportement dynamique du système. Ils montrent comment les objets interagissent et évoluent dans différents scénarios.
- Modélisation des interactions : Les diagrammes de séquence et de communication permettent de représenter les interactions entre objets ou composants. Ils illustrent les messages échangés entre les parties du système.
- Documentation : UML offre une manière structurée et standardisée de documenter les décisions de conception, les choix architecturaux et les spécifications logicielles. Cela facilite la maintenance et la compréhension du code au fil du temps.
- Analyse et validation : UML peut être utilisé pour analyser et valider la conception avant d'entamer le développement. Les diagrammes peuvent aider à détecter des problèmes potentiels ou des incohérences dans le système à un stade précoce.

3.2.3. Résumé :

En somme, UML joue un rôle essentiel dans la planification, la conception, la communication et la documentation des systèmes logiciels orientés objet. Cependant, il est important de noter que UML est un outil et une méthodologie, et non une fin en soi. Son utilisation efficace dépend de la compréhension des concepts sous-jacents et de l'adaptation aux besoins spécifiques de chaque projet.

3.3. Diagrammes UML :

3.3.1. Diagramme de cas d'utilisation

Un cas d'utilisation définit une manière d'utiliser le système et permet d'en décrire les exigences fonctionnelles. Chaque cas d'utilisation contient un ou plusieurs scénarios qui définissent comment le système devrait interagir avec les utilisateurs (appelés acteurs) pour atteindre un but ou une fonction spécifique d'un travail. Un acteur d'un cas d'utilisation peut être un humain ou un autre système externe à celui que l'on tente de définir. Les cas d'utilisation tentent d'éviter tout jargon technique et essaient au contraire d'adopter le langage de l'utilisateur final ou de l'expert du domaine.

a. Identification des principaux acteurs

Dans cette étape, nous identifions les principaux acteurs qui interagissent avec le système d'information. Un acteur représente un rôle externe (humain ou système) ayant un comportement spécifique dans le cadre de l'application.

- **Site Administrator** : Il est responsable du bon fonctionnement global de la plateforme. Il surveille les performances du site (qualité des données, performances des APIs, erreurs), audite les activités des utilisateurs, planifie les opérations de maintenance et notifie les utilisateurs si nécessaire.
- **Performance Monitor** : Cet acteur supervise la stabilité technique du système. Il détecte les erreurs, analyse les journaux et génère des rapports de performance. Il est aussi chargé d'identifier les comportements utilisateurs inhabituels pouvant indiquer une anomalie ou un usage abusif.
- **Data Collector** : Le collecteur de données est chargé de récupérer des informations provenant de sources externes, telles que des APIs, des bases de données publiques/privées ou des techniques de scraping. Ces données brutes alimentent le système.
- **Data Cleaner** : Il intervient pour nettoyer, filtrer et évaluer la qualité des données collectées. Il effectue des contrôles qualité sur les données brutes et nettoyées, et génère des rapports pour améliorer la fiabilité des analyses.
- **Data Updater** : Il est chargé de mettre à jour les données, de les segmenter et de suivre l'historique des modifications. Il veille à la cohérence, à la fraîcheur et à la traçabilité des données dans le système.
- **User** : L'acteur central qui initie diverses actions telles que la recherche B2B, la recherche B2C, la recherche d'emails professionnels/personnels, la recherche d'informations sur les

entreprises, la recherche d'avis sur les entreprises, l'exportation des résultats, le paiement de la facture et l'authentification. L'Utilisateur interagit avec le système pour accéder et utiliser ses fonctionnalités.

b. Diagramme de cas d'utilisation de gestion du site

Ce diagramme de cas d'utilisation montre comment les acteurs **Site Administrator** et **Performance Monitor** interagissent avec le système pour surveiller la performance du site. Les fonctionnalités incluent la surveillance de la qualité des données, des API, des journaux, la détection d'erreurs et l'analyse des comportements utilisateurs. Les relations d'inclusion («include») indiquent que certaines tâches sont déclenchées automatiquement lors de processus plus globaux comme 'Monitor site performance'.

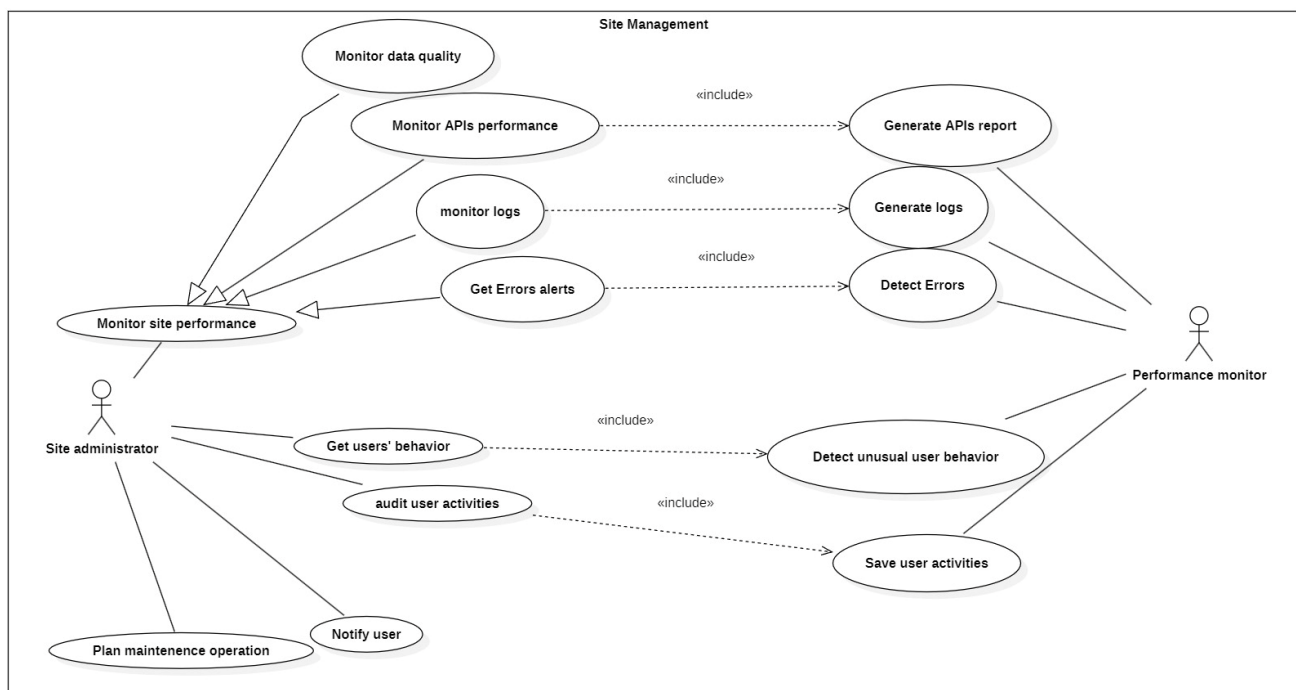


Figure 3.1. Diagramme de cas d'utilisation de gestion du site

c. Diagramme de cas d'utilisation de gestion des données

Ce diagramme illustre les processus liés à la collecte, au nettoyage et à la mise à jour des données dans le système. Les acteurs **Data Collector**, **Data Cleaner** et **Data Updater** exécutent des cas d'utilisation tels que collect data, filter data ou encore replicate and segment data. Les inclusions entre cas d'usage reflètent les dépendances fonctionnelles entre les tâches (ex. : Sauvegarde, évaluation, génération de rapports).

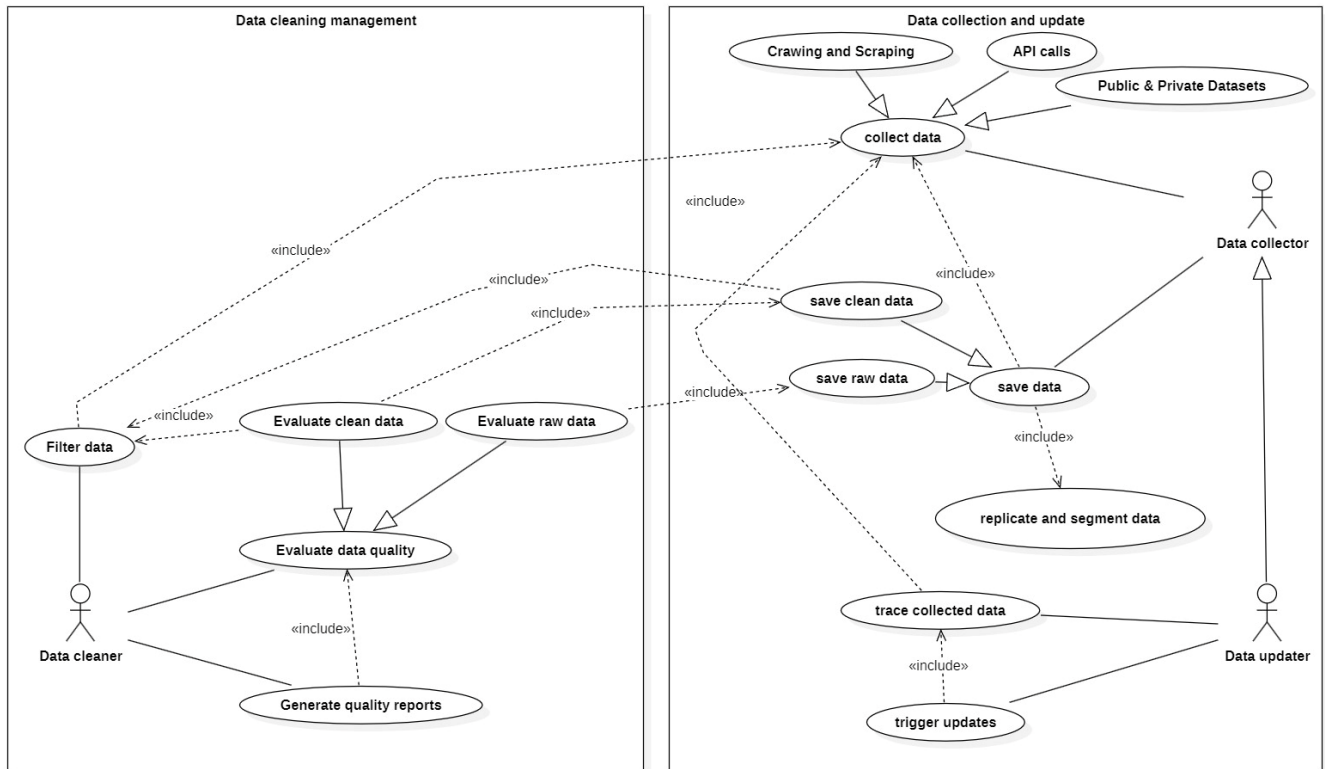


Figure 3.2. Diagramme de cas d'utilisation de gestion des données

d. Diagramme de cas d'utilisation de recherche et exportation de données

Ce diagramme de cas d'utilisation montre comment l'acteur User interagit avec le système pour effectuer diverses tâches liées à la recherche et à la gestion d'informations. Les fonctionnalités incluent la recherche B2B, la recherche B2C, la recherche d'emails professionnels ou personnels, la recherche d'informations sur les entreprises, la recherche d'avis sur les entreprises (y compris les scores de notation, les avis positifs et négatifs), l'exportation de résultats, le paiement de la facture et l'authentification.

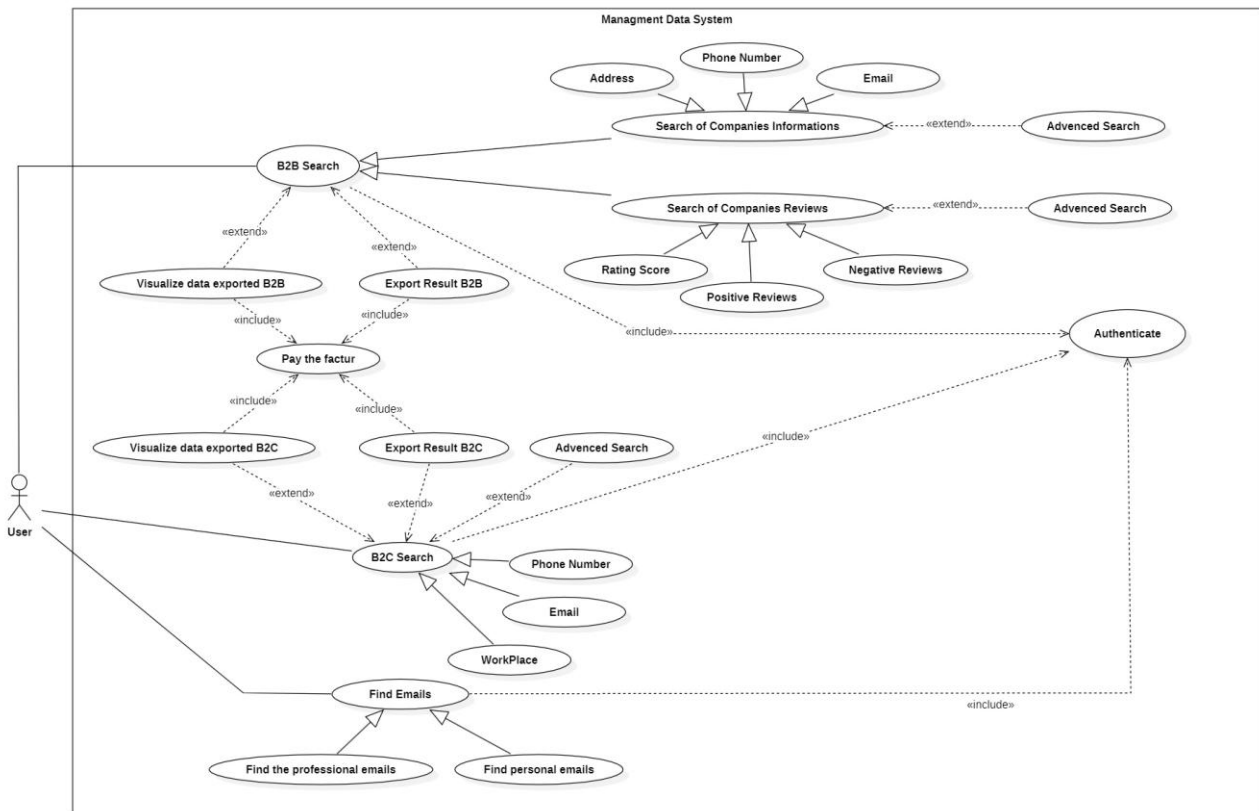


Figure 3.3. Diagramme de cas d'utilisation de recherche et exportation de données

3.3.2. Diagramme de classe

Dans le cadre de notre projet de développement d'un système d'information pour la gestion des données d'entreprises, nous avons conçu deux modèles de classes distincts répondant à des besoins spécifiques de collecte et d'analyse de données commerciales.

La modélisation UML (Unified Modeling Language) par diagrammes de classe nous permet de représenter la structure statique de notre système, en définissant clairement les entités métier, leurs propriétés et leurs relations. Cette approche garantit une architecture cohérente et facilite la compréhension du domaine par toutes les parties prenantes du projet.

Les trois classes principales que nous présentons - **ShadowPilot**, **B2B** et **B2C**- constituent les piliers de notre modèle de données et reflètent deux sources d'information complémentaires pour l'analyse des entreprises.

a. Diagramme de classe ShadowPilot

La classe **ShadowPilot** modélise un profil d'entreprise basé sur les données Trustpilot. Elle encapsule toutes les informations relatives à la réputation en ligne d'une entreprise, ses avis clients et ses métriques de confiance.

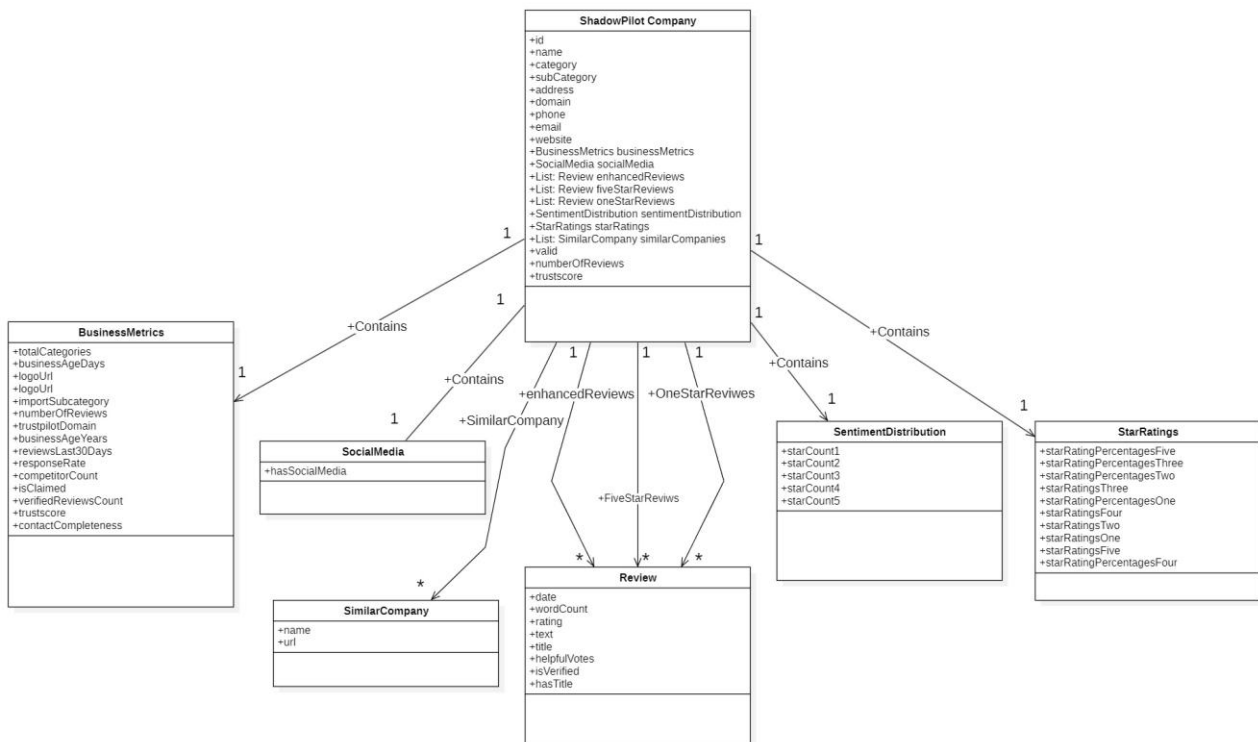


Figure 3.4. Diagramme de classe ShadowPilot

Responsabilités principales

- **Gestion des données d'entreprise** : Stockage des informations de base (nom, adresse, contact)
- **Analyse de réputation** : Calcul et suivi du score de confiance (trustscore)
- **Gestion des avis clients** : Organisation et classification des retours clients
- **Métriques business** : Suivi des performances et de l'évolution de l'entreprise
- **Analyse concurrentielle** : Comparaison avec des entreprises similaires

b. Diagramme de classe B2B

La classe **B2B** représente un établissement commercial avec ses données géographiques, ses informations pratiques et sa présence en ligne. Elle s'appuie sur les données Google Maps/Places pour fournir une vue complète d'un point d'intérêt commercial.

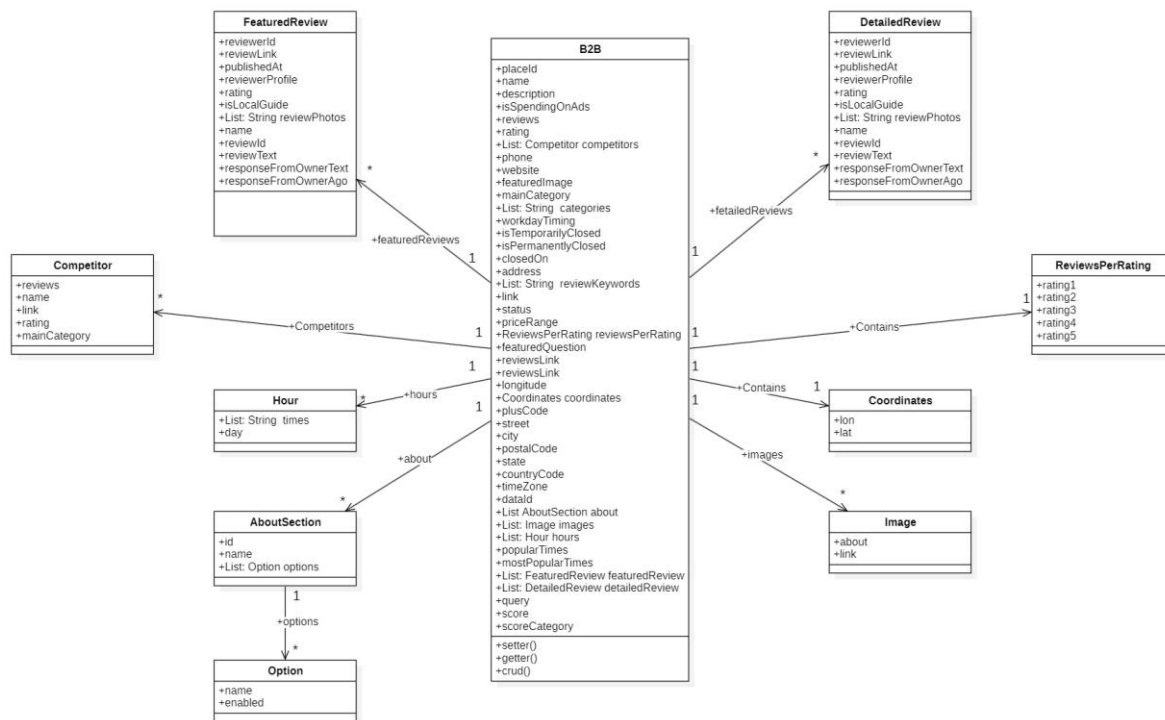


Figure 3.5. Diagramme de classe B2B

Responsabilités principales

- **Géolocalisation** : Gestion précise de la position géographique
- **Informations pratiques** : Horaires, contact, accessibilité
- **Réputation locale** : Avis Google et évaluations clients
- **Analyse concurrentielle** : Identification et comparaison avec la concurrence locale
- **Présence digitale** : Website, réseaux sociaux, visibilité en ligne

c. Diagramme de classe B2C

La classe **B2B** modélise un profil utilisateur individuel dans un contexte B2C (Business to Consumer). Elle encapsule toutes les informations personnelles, géographiques et professionnelles d'un utilisateur final, permettant une gestion complète des données de profil dans une application sociale ou de services personnalisés.

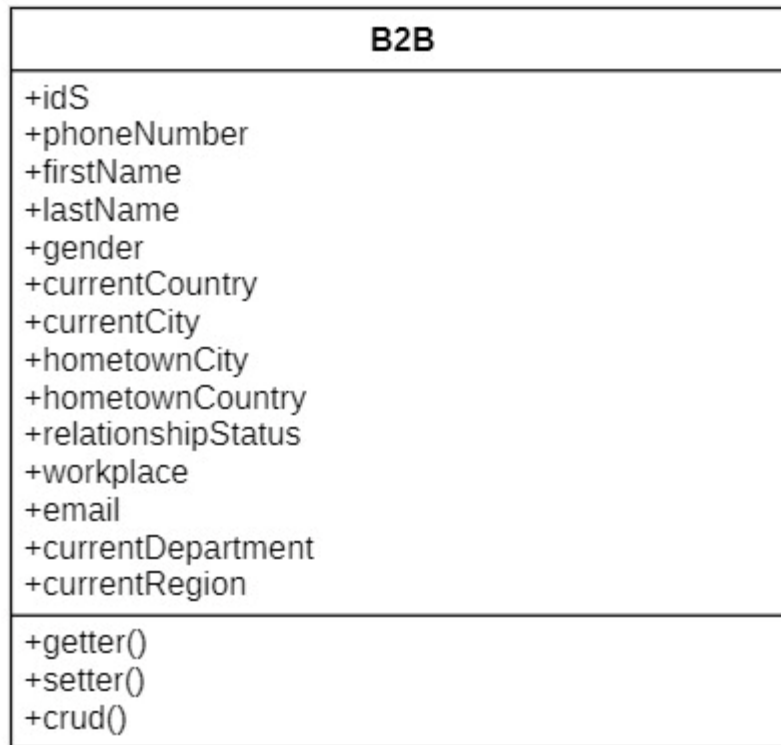


Figure 3.6. Diagramme de classe B2C

Responsabilités principales

- **Gestion de l'identité** : Stockage des informations d'identification (nom, prénom, identifiants)
- **Gestion des contacts** : Numéro de téléphone et email pour la communication
- **Géolocalisation personnelle** : Suivi de la localisation actuelle et ville d'origine
- **Profil démographique** : Informations sur le genre et statut relationnel
- **Informations professionnelles** : Lieu de travail et département actuel
- **Données administratives** : Gestion des régions et départements pour la segmentation

3.3.3. Diagramme de séquence

Afin de modéliser de manière plus précise les échanges entre les différents acteurs du système, l'application et la base de données, il est nécessaire de transposer nos scénarios d'usage sous forme de diagrammes de séquence.

Les diagrammes de séquence constituent un outil de représentation visuelle des cas d'utilisation. Ils offrent la possibilité de modéliser les interactions collaboratives entre les objets en adoptant une perspective temporelle, en mettant particulièrement l'accent sur l'ordre chronologique des communications et des échanges de messages.

a. Diagramme de Recherche B2B avec Authentification

Ce diagramme décrit un processus sécurisé où l'utilisateur doit s'authentifier avant d'accéder aux fonctions de recherche d'entreprises. Après validation des identifiants, il peut rechercher des entreprises via différents critères (email, téléphone, adresse). Des extensions comme la recherche avancée ou l'export sont également possibles après authentification.

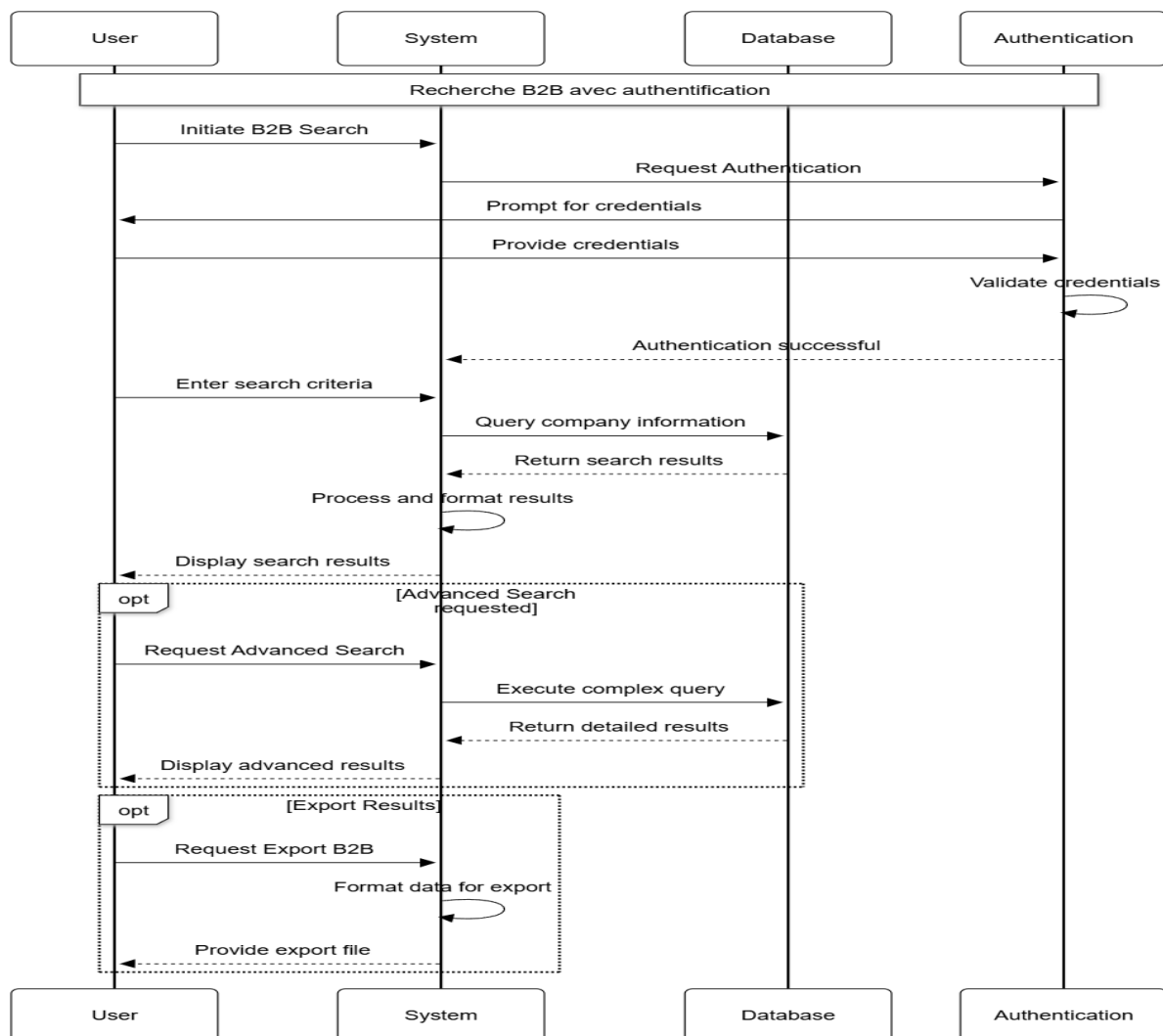


Figure 3.7. Diagramme de séquence de Recherche B2B avec Authentification

b. Diagramme de Gestion et Monitoring du Site

Ce diagramme illustre comment un administrateur surveille l'état du site en temps réel : performance des APIs, qualité des données, logs, alertes, détection d'anomalies, audit des actions utilisateur et planification de la maintenance. Il s'agit d'une approche proactive de supervision technique.

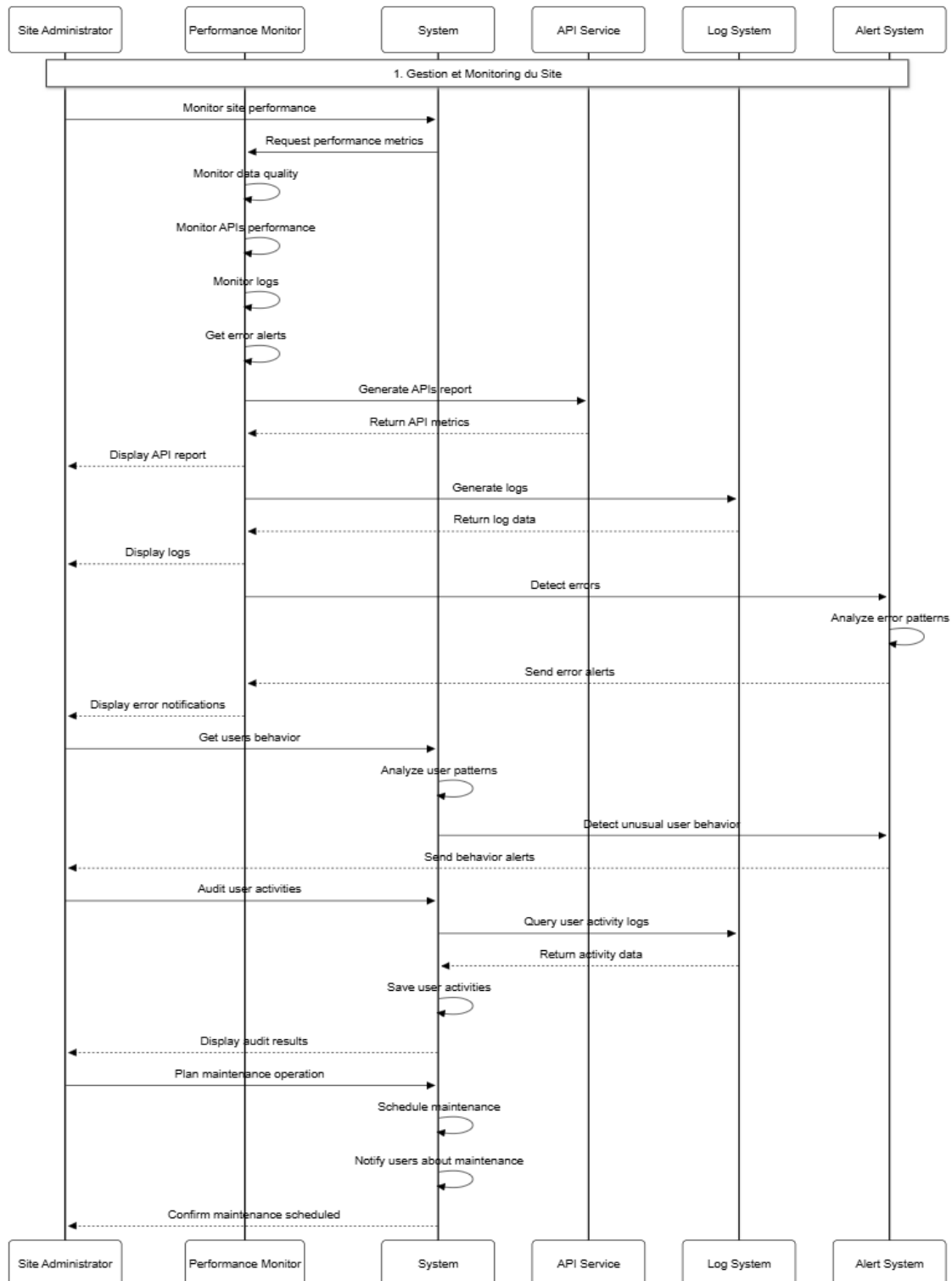


Figure 3.8. Diagramme de séquence de Gestion et Monitoring du Site

c. Diagramme de Collecte et Mise à Jour des Données

Ce diagramme montre un système de collecte multi-sources (scraping, APIs, jeux de données publics/privés), avec stockage des données brutes, nettoyées et enrichies. Il intègre des mécanismes de mise à jour automatique, segmentation et traçabilité pour garantir la fraîcheur et la qualité des données.

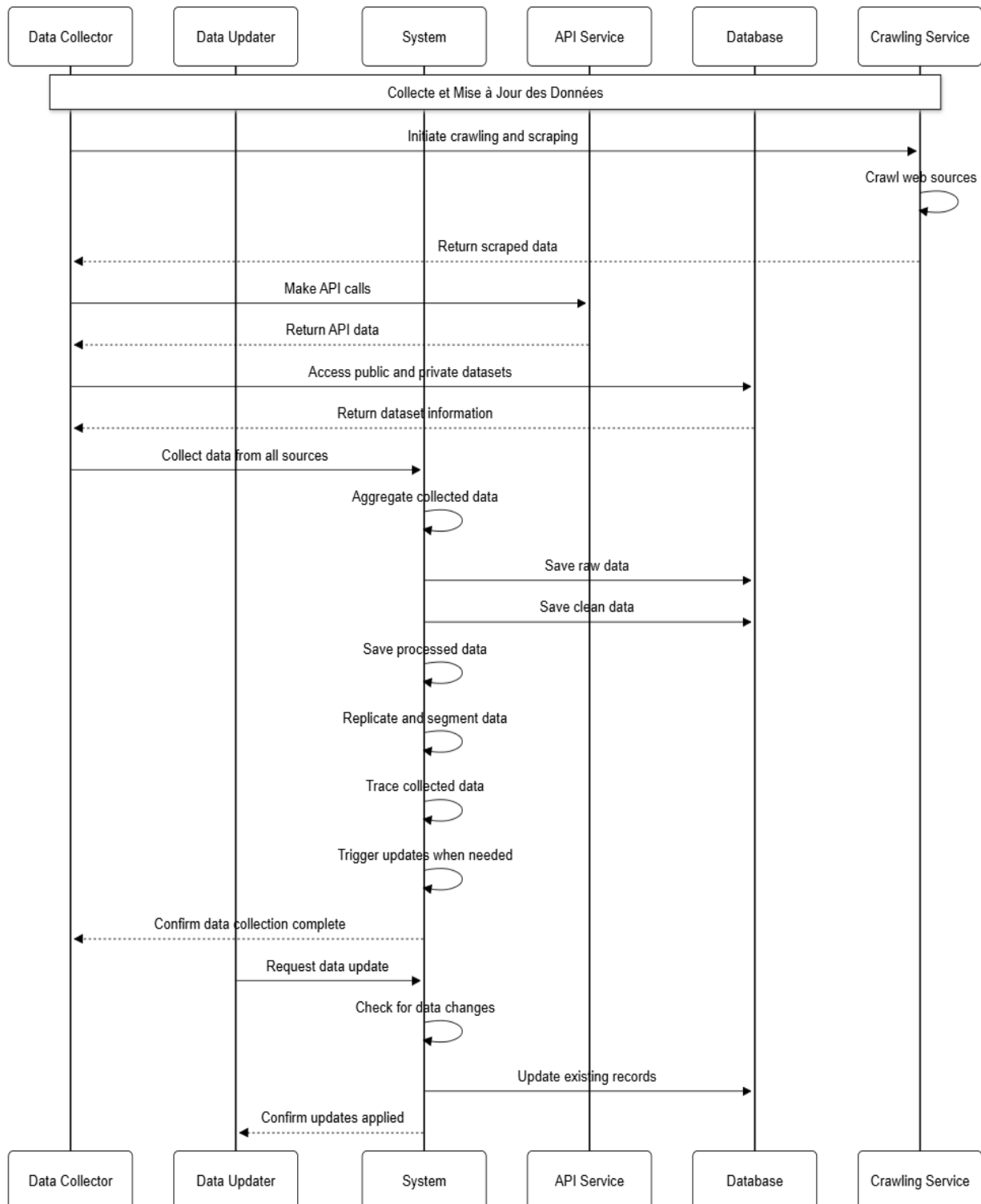


Figure 3.9. Diagramme de séquence de Collecte et Mise à Jour des Données

d. Diagramme de Gestion du Nettoyage des Données

Ce diagramme détaille un processus de nettoyage rigoureux : détection des anomalies, suppression des doublons, standardisation des formats et correction des erreurs. Une validation finale et un reporting qualité assurent l'exploitation fiable des données transformées.

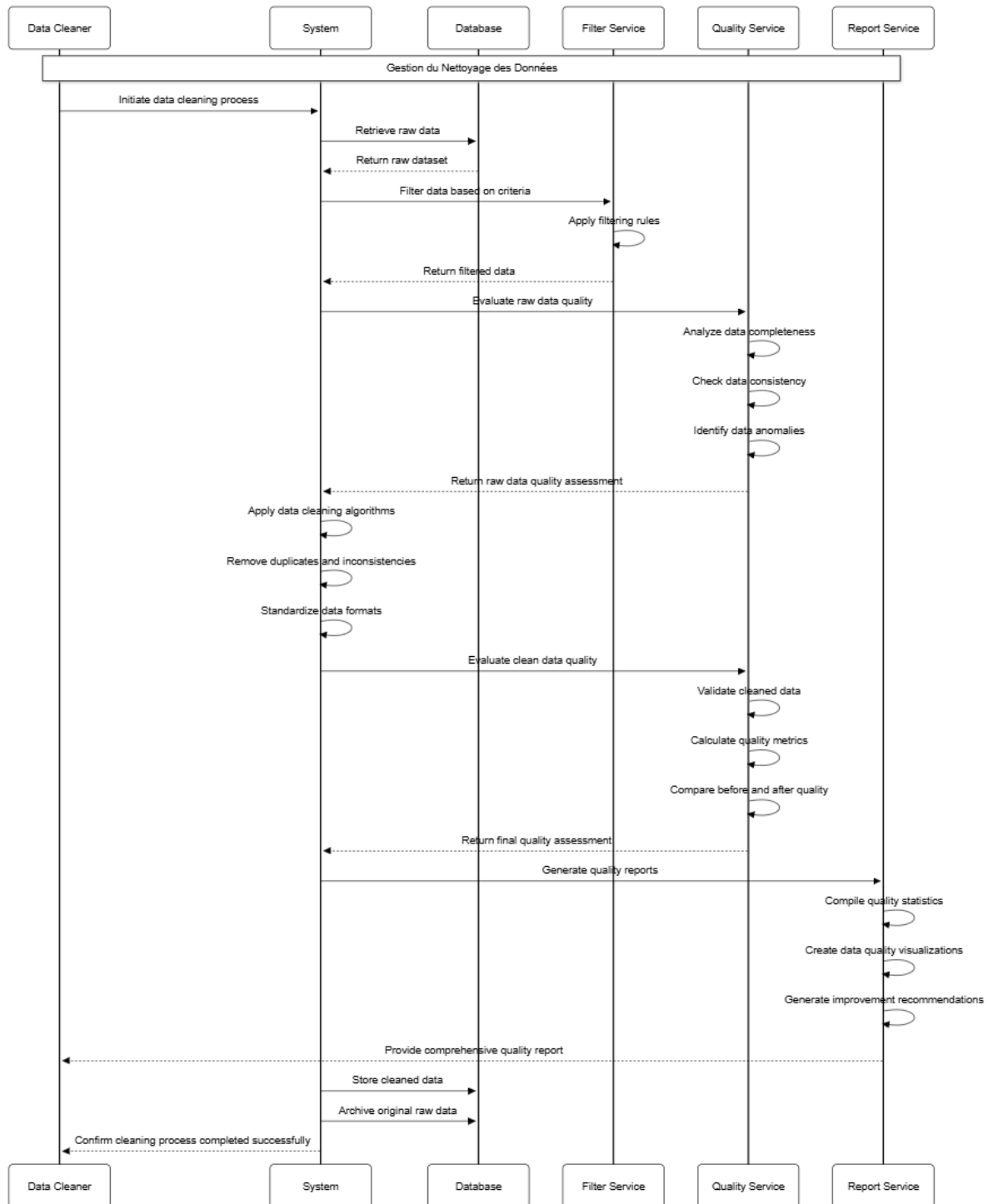


Figure 3.10. Diagramme de séquence de Gestion du Nettoyage des Données

3.4. Conclusion

Ce chapitre a présenté l'analyse fonctionnelle et la modélisation UML du projet DataPull. À travers différents diagrammes (cas d'utilisation, classes, séquences), nous avons défini les acteurs, les interactions et la structure du système. Cette modélisation a permis de clarifier les besoins, d'organiser les entités métier et de décrire les scénarios clés de manière dynamique. Elle constitue une base solide pour orienter les choix techniques et faciliter la mise en œuvre du projet dans les prochaines étapes.

Chapitre 4 :

TECHNOLOGIES ET OUTILS TECHNIQUES

4.1. Introduction

DataPull est une plateforme conçue pour l'extraction, la transformation et la visualisation de données provenant de multiples sources. La version actuelle de DataPull présente des limitations significatives en termes de performance (temps de réponse long pour les recherches volumineuses), ergonomie (interface utilisateur peu intuitive et non optimisée pour tous les appareils) et scalabilité (difficulté à gérer des volumes croissants de données). Pour surmonter ces défis, nous avons recréé l'application en adoptant des technologies modernes et performantes, sélectionnées pour leur robustesse, leur flexibilité et leur capacité à répondre aux exigences du projet.

Pour le back-end, Spring Boot a été choisi comme framework Java pour développer une application sécurisée, modulaire et évolutive. Pour le front-end, ReactJs permet de créer des interfaces utilisateur dynamiques et réactives, améliorant l'ergonomie. Elasticsearch a été intégré pour optimiser les recherches et l'analyse de données volumineuses, renforçant la performance et la scalabilité. Enfin, une suite d'outils complémentaires a été utilisée pour rationaliser le développement, les tests et le déploiement. Ce chapitre détaille chaque technologie, son rôle dans DataPull, ses avantages, et son intégration pour offrir une solution complète, performante et adaptée aux besoins futurs des clients.

4.2. Les Framework

Les frameworks sont au cœur du développement de DataPull, offrant une structure solide pour gérer la logique métier, les données et l'interface utilisateur. Nous avons utilisé Spring Boot pour le back-end, ReactJs pour le front-end, et Bootstrap pour le design, chacun jouant un rôle clé dans l'amélioration des performances, de l'ergonomie et de la scalabilité de l'application.

4.2.1. Qu'est-ce qu'un Framework ?

Un Framework désigne un ensemble de composants logiciels formant le squelette d'un programme, conçu pour aider les développeurs informatiques dans leur travail de programmation et leur offrir une base de travail réutilisable sur plusieurs projets.

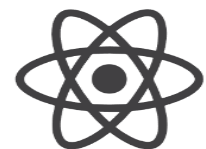
Un Framework s'apparente à un véritable espace de travail modulaire, réunissant les outils, les ressources et les conventions ou normes nécessaires à la création d'applications web sur mesure. Traduit littéralement, le mot anglais Framework signifie « cadre de travail ». Cette traduction est plutôt heureuse pour comprendre le concept qui se cache derrière le terme. Un

Framework de développement est donc :

- Un ensemble cohérent de composants éprouvés et réutilisables (bibliothèques, classes, helpers...).
- Un ensemble de préconisations pour la conception et le développement d'applications.

Il y a beaucoup de cadres pour chaque technologie. Ils sont là pour donner un cadre de travail au développeur. Grâce à ces outils, le développeur sera mieux organisé, son code sera maintenable, évolutif et clair. En effet, deux développeurs utilisant le même Framework, n'auront donc aucun mal à se comprendre. Dans le monde du web, il y a deux types de Framework principaux, ceux de la catégorie frontend, puis ceux de la catégorie back-end.

4.3. Frontend Framework :



React JS

4.3.1. ReactJs :

ReactJs est une bibliothèque JavaScript open-source, développée par Facebook, conçue pour créer des interfaces utilisateur dynamiques, modulaires et performantes. Elle repose sur deux concepts clés [17] :

- **Composants Réutilisables** : L'interface est décomposée en blocs indépendants (boutons, formulaires, tableaux) qui peuvent être réutilisés.
- **Virtual DOM** : Une représentation légère du DOM réel, permettant des mises à jour rapides et efficaces de l'interface utilisateur.

Dans **DataPull**, ReactJs a été utilisé pour adresser les limitations en ergonomie et performance de la manière suivante :

- **Composants Réutilisables** :
 - Nous avons développé des composants pour les tableaux de bord (affichage des statistiques), les formulaires de recherche (filtres personnalisés) et les visualisations de données (graphiques interactifs).
 - Cette modularité a réduit le temps de développement et simplifié la maintenance, tout en offrant une interface cohérente et intuitive, répondant aux besoins d'ergonomie.
- **Gestion Performante de l'Interface** :
 - Le Virtual DOM optimise les mises à jour en ne modifiant que les éléments nécessaires, par exemple lors de l'affichage de nouveaux résultats de recherche ou de l'actualisation des graphiques, améliorant les performances.
 - Cela réduit les temps de chargement des pages, un problème clé dans la version actuelle de **DataPull**.

- **Intégration avec Spring Boot :**

- ReactJs consomme les API RESTful fournies par Spring Boot pour récupérer les données transformées ou les résultats de recherche, assurant une communication fluide entre le front-end et le back-end.
- Par exemple, les requêtes de recherche sont envoyées via des appels HTTP (GET/POST), et les réponses JSON sont rendues dans des composants React.

- **Routage :**

- **React Router** gère la navigation dans l'application à page unique (SPA), permettant des transitions fluides entre les vues, comme du tableau de bord aux pages de résultats ou aux paramètres utilisateur.
- Cette approche améliore l'ergonomie en évitant les rechargements complets de page.

- **Gestion d'État :**

- **Redux** est utilisé pour gérer l'état global, synchronisant les données entre les composants (par exemple, les filtres de recherche appliqués ou les préférences utilisateur).
- Cela garantit une interface réactive et cohérente, même avec des interactions complexes.

- **Amélioration de l'Ergonomie :**

- ReactJs permet de créer des interfaces dynamiques avec des animations subtiles (par exemple, des transitions lors de l'ouverture d'un modal) et des retours visuels immédiats (par exemple, des messages de validation dans les formulaires), rendant l'application plus intuitive.

4.3.2. CSS Personnalisé :

Pour répondre aux besoins spécifiques de **DataPull** et surmonter les limitations ergonomiques, nous avons créé des feuilles de style CSS personnalisées. Ces styles ont été développés pour surmonter les limitations ergonomiques de la version actuelle de DataPull, en améliorant l'intuitivité, la lisibilité et l'adaptabilité de l'interface utilisateur pour une clientèle variée.



➤ **Personnalisation Visuelle**

- **Adaptation à l'Identité Client :**

- Les couleurs, polices et mises en page ont été conçues pour refléter une esthétique professionnelle et moderne, adaptée aux attentes des clients d'iaweb. Par exemple, nous avons utilisé des tons sobres comme le bleu et le gris, associés à des polices lisibles telles que Roboto ou Open Sans, pour garantir une apparence soignée et universellement attrayante.

- Cette personnalisation permet à DataPull de s'intégrer harmonieusement dans les environnements professionnels des clients, tout en renforçant la cohérence visuelle.
- **Clarté des Données :**
 - Les formulaires (par exemple, pour saisir des filtres de recherche) et les tableaux (pour afficher les résultats ou les statistiques) sont stylisés pour maximiser la lisibilité des données complexes.
 - Exemple : Les tableaux utilisent des bordures subtiles et des couleurs alternées pour différencier les lignes, facilitant l'analyse des résultats de recherche ou des visualisations de données.
- **Amélioration de l'Accessibilité**
 - **Conformité aux Normes :**
 - Les contrastes de couleurs respectent les normes WCAG 2.1 (niveau AA), garantissant une lisibilité optimale pour les utilisateurs ayant des déficiences visuelles, comme le daltonisme ou une basse vision.
 - Exemple : Un contraste minimum de 4.5:1 est appliqué entre le texte et l'arrière-plan pour les éléments clés comme les labels de formulaires.
 - **Navigation Intuitive :**
 - Les éléments interactifs, tels que les boutons et les liens, disposent d'états visuels distincts (hover, focus, active) pour guider les utilisateurs, y compris ceux utilisant des technologies d'assistance comme les lecteurs d'écran.
 - Exemple : Un bouton de soumission de recherche change de couleur au survol et affiche un contour clair lorsqu'il est sélectionné via le clavier.
- **Optimisation de l'Ergonomie**
 - **Interface Épurée :**
 - Les marges, espacements et tailles de police sont ajustés pour éviter la surcharge visuelle, rendant l'interface plus aérée et facile à naviguer par rapport à la version actuelle de DataPull, qui souffre d'une présentation encombrée.
 - Exemple : Les sections de visualisation de données utilisent des espacements généreux pour séparer les graphiques des filtres, améliorant la clarté pour les clients.
 - **Adaptabilité Multi-Client :**
 - Les styles sont conçus pour être flexibles, permettant une personnalisation rapide pour différents clients d'iaweb tout en maintenant une base cohérente.

- Exemple : Les thèmes CSS peuvent être modifiés via des variables pour adapter l'interface à l'identité visuelle spécifique d'un client.

➤ **Intégration avec Bootstrap**

• **Complémentarité avec Bootstrap :**

- Les styles CSS personnalisés s'appuient sur les classes de Bootstrap pour des ajustements spécifiques, comme la personnalisation des modals (par exemple, pour afficher des détails de résultats) ou des composants de navigation (barre latérale ou menus déroulants).
- Exemple : Une modal Bootstrap est stylisée avec des bordures arrondies et des couleurs spécifiques pour correspondre au thème de DataPull, tout en conservant la réactivité de Bootstrap.

• **Optimisation Responsive :**

- Les styles personnalisés renforcent la grille responsive de Bootstrap, garantissant que l'interface s'adapte parfaitement aux écrans des clients, qu'ils utilisent des ordinateurs de bureau, des tablettes ou des smartphones.
- Cela répond directement à la limitation ergonomique de la version actuelle, qui n'est pas optimisée pour les appareils mobiles.

4.4. Backend Framework :



4.4.1. Spring Boot :

Spring Boot est un framework Java basé sur le projet Spring, conçu pour simplifier le développement d'applications robustes et évolutives. Dans DataPull, Spring Boot a été utilisé pour gérer la logique métier, les API RESTful, et l'intégration avec les bases de données et Elasticsearch, répondant aux besoins de scalabilité.[18]

Les principales fonctionnalités de Spring Boot utilisées incluent :

- **Configuration Automatique** : Les starters de Spring Boot ont réduit les configurations manuelles, accélérant le développement.
- **Serveur Embarqué** : Tomcat embarqué permet d'exécuter l'application de manière autonome, simplifiant le déploiement.
- **Gestion des Dépendances** : Les **Spring Boot Starters** (par exemple, spring-boot-starter-web, spring-boot-starter-data-jpa, spring-boot-starter-data-elasticsearch) ont facilité l'intégration des bibliothèques nécessaires.

- **API RESTful** : Les contrôleurs REST, annotés avec `@RestController` et `@RequestMapping`, gèrent les requêtes HTTP pour l'extraction, la transformation et la visualisation des données.
- **Architecture MVC** : L'application suit le modèle Modèle-Vue-Contrôleur :
 - **Modèles (Entities)** : Les entités JPA définissent la structure des données, comme les métadonnées ou les informations sur les congés.
 - **Contrôleurs** : Les contrôleurs orchestrent la logique métier et les requêtes HTTP.
 - **Vues** : Les vues sont gérées par ReactJs, avec Spring Boot servant les API.
- **Gestion de la Base de Données** : **Spring Data JPA** gère les données structurées dans PostgreSQL, tandis que **Spring Data Elasticsearch** facilite les interactions avec Elasticsearch pour les recherches.
- **Sécurité** : **Spring Security** implémente l'authentification (via OAuth2 ou JWT) et l'autorisation, sécurisant les endpoints selon les rôles des utilisateurs.
- **Injection de Dépendances** : L'inversion de contrôle (IoC) rend le code modulaire et testable.
- **Gestion des Erreurs** : Les exceptions sont gérées via `@ControllerAdvice`, fournissant des réponses cohérentes aux erreurs.

4.4.2. Maven :

Apache Maven, également appelé Maven, est un outil de gestion et d'automatisation de la production de projets logiciels en Java, en particulier pour Java EE. Il est utilisé pour faciliter l'intégration continue lors du développement de logiciels. Maven est géré par l'organisation Apache Software Foundation et était initialement une branche du projet Jakarta. L'objectif principal de Maven est de simplifier le processus de construction d'un logiciel à partir de son code source, en optimisant les différentes tâches impliquées et en garantissant un ordre correct de construction. On peut le comparer au système make sous Unix ou à l'outil Ant. Maven utilise un modèle d'objet de projet (Project Object Model ou POM) pour décrire un projet logiciel, ses dépendances avec d'autres modules externes et l'ordre dans lequel les différentes étapes de production doivent être exécutées. Il fournit un large éventail de tâches prédéfinies, telles que la compilation du code Java et la modularisation du projet. Une caractéristique importante de Maven est sa capacité à fonctionner en réseau. Il permet de synchroniser des projets indépendants en offrant des fonctionnalités de publication standardisée d'informations et de distribution automatique



de modules JAR. En tant que tel, Maven peut télécharger dynamiquement les dépendances à partir de dépôts logiciels connus, offrant une synchronisation transparente des modules requis.

4.5. Base de Données et Recherche

4.5.1. Elasticsearch

Elasticsearch est un moteur de recherche et d'analyse distribué basé sur Lucene, utilisé dans DataPull pour [19] :

- Recherche Full-Text : Permettre des recherches rapides et pertinentes dans les données et métadonnées, avec prise en charge des recherches floues et des filtres, améliorant les performances.
- Analyse de Données : Générer des agrégations pour des statistiques (par exemple, nombre de congés par période ou département).
- Scalabilité : Gérer de grands volumes de données grâce à son architecture distribuée, répondant aux limitations de scalabilité.
- Intégration avec Spring Boot : Spring Data Elasticsearch simplifie les opérations de recherche et d'indexation via des repositories.



4.5.2. PostgreSQL

PostgreSQL, une base de données relationnelle open-source, est utilisée pour les données structurées [20]

- Stockage Structuré : Gestion des informations sur les utilisateurs, rôles et congés.
- Intégration avec Spring Data JPA : Simplification des opérations CRUD via des repositories.
- Performances : Optimisation des requêtes complexes pour répondre aux besoins de performance.



4.6. Outils Complémentaires

Pour optimiser le développement, les tests, le déploiement et la maintenance de **DataPull**, nous avons utilisé une suite d'outils complémentaires. Ces outils répondent aux limitations actuelles en matière de performance, ergonomie et scalabilité, tout en facilitant la collaboration et l'évolutivité de l'application.

- **Docker :**



- **Rôle** : Conteneurisation de l'application (Spring Boot, PostgreSQL, Elasticsearch) pour assurer une portabilité et une cohérence entre les environnements de développement, test et production.
- **Contribution** : Simplifie le déploiement et améliore la scalabilité en permettant l'orchestration via des outils comme Kubernetes, répondant aux besoins des clients pour des déploiements flexibles.[21]

- **Git et GitHub :**



- **Rôle** : Gestion de version du code source avec Git, hébergé sur GitHub, pour faciliter la collaboration entre développeurs.
- **Contribution** : Permet un suivi des modifications et une intégration continue, essentiel pour maintenir un code évolutif et collaboratif.[22]

- **Postman :**



- **Rôle** : Test des endpoints API RESTful pour valider les requêtes et réponses avant l'intégration avec le front-end.
- **Contribution** : Accélère le débogage et garantit la fiabilité des API, améliorant la performance des interactions client-serveur.[23]

- **Jenkins :**



- **Rôle** : Automatisation des pipelines CI/CD pour la compilation, les tests et le déploiement.
- **Contribution** : Réduit les erreurs manuelles et accélère les mises à jour, renforçant la scalabilité. [24]

- **Lombok :**



- **Rôle** : Réduction du code répétitif dans les entités Java (getters, setters) via des annotations comme `@Data`.
- **Contribution** : Améliore la lisibilité et la maintenabilité du code back-end.[25]

- **Swagger :**

- **Rôle :** Documentation automatique des API RESTful via Springdoc OpenAPI, accessible via une interface web.
- **Contribution :** Facilite l'utilisation des endpoints par les développeurs et les clients, améliorant l'ergonomie de l'intégration.[26]



- **JUnit et Mockito :**

- **Rôle :** Tests unitaires pour valider les services et contrôleurs Spring Boot.
- **Contribution :** Garantit la fiabilité du code, essentielle pour une application critique.[27]



- **Python :**

- **Rôle :** Utilisé pour des scripts de transformation de données et l'intégration avec des outils comme FastAPI.[28]
- **Contribution :** Sa simplicité syntaxique accélère le développement de fonctionnalités spécifiques, comme le prétraitement des données.



- **FastAPI :**

- **Rôle :** Développement d'API secondaires pour des tâches spécifiques, comme l'extraction rapide de données.[39]
- **Contribution :** Améliore les performances grâce à son support asynchrone, répondant aux limitations de temps de réponse.



- **Amazon EC2 :**

- **Rôle :** Hébergement des instances de l'application sur des serveurs virtuels AWS.
- **Contribution :** Offre une scalabilité dynamique pour gérer les volumes croissants de données des clients.[30]



- **Amazon S3 :**

- **Rôle :** Stockage des fichiers statiques (documents, logs, exports de données).
- **Contribution :** Améliore la disponibilité et la gestion des ressources, répondant aux besoins de scalabilité.[31]



- **Uvicorn :**

- **Rôle :** Serveur ASGI pour exécuter les API FastAPI en production.
- **Contribution :** Optimise les performances des services asynchrones, réduisant les temps de réponse.[32]



- **Gunicorn :**



- **Rôle :** Serveur WSGI pour exécuter des applications Python (par exemple, Flask) en production.
- **Contribution :** Gère les requêtes concurrentes, renforçant la performance pour les clients.[33]

- **Flask :**



- **Rôle :** Développement de micro-services légers pour des fonctionnalités spécifiques, comme des endpoints de test.
- **Contribution :** Offre une alternative légère à FastAPI pour des tâches simples, améliorant la flexibilité.[34]

- **Colab Notebook :**



- **Rôle :** Prototypage et analyse des données à l'aide de Jupyter Notebooks dans Google Colab.
- **Contribution :** Facilite l'expérimentation avec des runtimes GPU/TPU pour les transformations complexes.[35]

- **VS Code :**



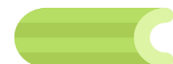
- **Rôle :** Environnement de développement principal avec extensions pour Python, Java, et React.
- **Contribution :** Améliore la productivité grâce à son interface intuitive et ses outils intégrés.[36]

- **Redis :**



- **Rôle :** Cache en mémoire pour stocker des résultats intermédiaires ou des tâches Celery.
- **Contribution :** Réduit les temps de réponse en accélérant l'accès aux données fréquemment utilisées.[37]

- **Celery, Celery Beat, Flower :**



- **Rôle :** Celery exécute des tâches asynchrones (par exemple, transformation de données en arrière-plan), Celery Beat planifie des tâches périodiques, et Flower monitore les tâches en temps réel.
- **Contribution :** Améliore les performances en déchargeant les tâches lourdes du cycle de requête/réponse.[38]

- **SQL :**



- **Rôle :** Langage pour interroger PostgreSQL et gérer les données structurées.
- **Contribution :** Permet des requêtes optimisées pour les analyses et rapports clients.[39]



- **Power BI :**

- **Rôle** : Création de visualisations interactives pour les clients, connectées aux données de **DataPull**.
- **Contribution** : Améliore l'ergonomie en offrant des tableaux de bord clairs et partageables.[40]



- **Shell/Bash :**

- **Rôle** : Scripts pour automatiser les tâches de déploiement, configuration des serveurs et gestion des conteneurs.
- **Contribution** : Réduit le temps de gestion des environnements, renforçant la scalabilité.[41]

4.7. Conclusion

Ce chapitre a détaillé les technologies et outils techniques utilisés pour recréer DataPull, répondant aux limitations de performance, ergonomie et scalabilité de la version actuelle. ReactJs et Bootstrap offrent une interface intuitive et responsive, Spring Boot et Java assurent un back-end robuste, tandis qu'Elasticsearch, PostgreSQL et MongoDB gèrent efficacement les données. Des outils comme Docker, Jenkins, FastAPI, Celery, Power BI et AWS (EC2, S3) optimisent le développement, le déploiement et la maintenance. Ces choix technologiques font de DataPull une plateforme performante, ergonomique et évolutive, adaptée aux besoins des clients.

Chapitre 5 :

IMPLEMENTATION ET MISE EN ŒUVRE

5.1. Introduction

Ce chapitre offre une vue d'ensemble complète des solutions mises en œuvre lors du développement de la plateforme DataPull. Il met l'accent sur les approches techniques et les outils utilisés pour répondre aux principaux défis du projet, notamment le scraping de données, les fonctionnalités B2C et B2B, ainsi que la création d'un outil ETL (Extract, Transform, Load) pour gérer le flux massif de données. Chaque section détaille les méthodes spécifiques, leur mise en œuvre et les raisons de leur choix, en montrant comment ces solutions ont contribué au succès global de la plateforme.

5.2. Scraping et nettoyage des données : sources et méthodes

Le scraping de données a constitué un élément essentiel de la plateforme DataPull, permettant de collecter un volume important de données relatives aux entreprises et aux clients à partir de sources variées. Nous avons adopté une approche multi-facette, en utilisant des API publiques, des jeux de données ouverts, des scrapers web personnalisés, ainsi qu'un outil de scraping spécialisé reposant sur une infrastructure cloud.

Ci-dessous, nous détaillons chaque méthode, sa mise en œuvre et les défis auxquels elle a permis de répondre.

5.2.1. APIs publiques (INPI, INSEE)

Pour initier le processus de collecte de données, nous avons exploité des APIs publiques provenant de sources officielles telles que l'INPI (Institut National de la Propriété Industrielle) [42], l'INSEE (Institut National de la Statistique et des Études Économiques) [43] et le RNE (Registre National des Entreprises). Ces APIs nous ont fourni des données structurées sur les entreprises, incluant les immatriculations, les indicateurs financiers et les statistiques économiques.

➤ Mise en œuvre

- Nous avons utilisé Swagger, un outil de documentation d'API, pour explorer les points d'accès disponibles et comprendre les paramètres nécessaires à la récupération des données.

- Des requêtes spécifiques ont été conçues afin de récupérer des données ciblées à l'aide de la documentation de chaque API, telles que les informations détaillées sur les entreprises et les classifications sectorielles.

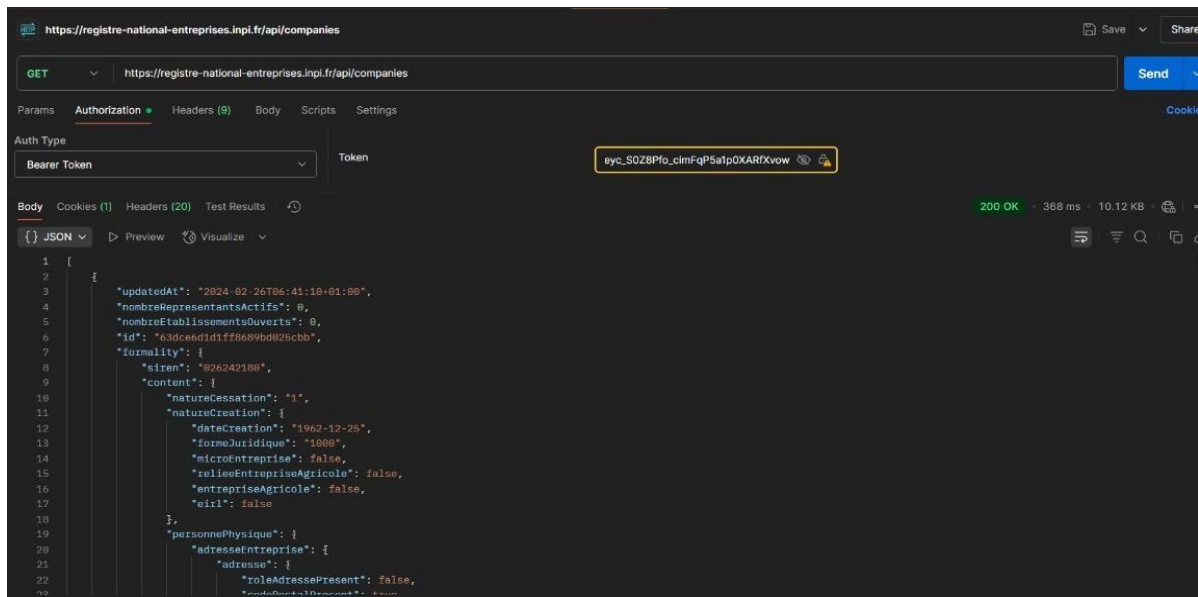


Figure 5.1. test des APIs Publiques sur Postman

- Des scripts Python, utilisant la bibliothèque requests, ont été développés afin d'automatiser les appels aux APIs et de gérer la pagination pour le traitement de grands ensembles de données.
- **Authentification** : Le script utilise un mécanisme d'authentification sécurisé, en obtenant un **JSON Web Token (JWT)** en envoyant les identifiants (nom d'utilisateur et mot de passe) stockés dans un fichier .env vers l'endpoint `"/sso/login"`. Ce token est ensuite inclus dans l'en-tête **Authorization** pour les requêtes suivantes, garantissant un accès sécurisé.
 - **Pagination basée sur les dates** : Afin de gérer de grands ensembles de données et de respecter les limites de requêtes imposées par les APIs, le script adopte une stratégie de pagination basée sur les dates. Il interroge les données dans des plages de dates spécifiques, allant du 1er janvier 2010 jusqu'à la date actuelle, avec des incréments de 30 jours. Cette approche permet une couverture complète des données tout en offrant la possibilité de reprendre l'exécution là où elle s'était arrêtée en cas d'interruption. Pour chaque plage de dates, le script effectue une pagination avec les paramètres `page` et `pageSize`, récupérant jusqu'à 100 enregistrements par page (la limite maximale autorisée par l'API).

- **Stockage des données** : Les données récupérées sont stockées dans une base de données MongoDB à l'aide de la bibliothèque pymongo.

La méthode « store_companies » utilise une opération upsert (update or insert) pour mettre à jour les enregistrements existants ou insérer de nouveaux documents, en évitant les doublons grâce à un index sur l'identifiant de l'entreprise. Cela garantit l'intégrité des données et un stockage efficace.

- **Gestion des erreurs et limitations** : Le script intègre une gestion des erreurs robuste, en réessayant jusqu'à trois fois en cas d'échec d'une requête, et en respectant les limites de requêtes par l'introduction d'un délai entre les appels. Si le token d'authentification expire, le script se réauthentifie automatiquement et relance la requête.
- **Journalisation (Logging)** : Une journalisation complète est mise en place à l'aide de la bibliothèque **logging**, enregistrant les événements clés tels que le succès de l'authentification, la récupération des pages ou encore les erreurs rencontrées.
- **Défis rencontrés et solutions apportées** :
 - **Limitations de requêtes (Rate Limits)** : Les APIs imposaient des restrictions sur le nombre de requêtes par heure, limitant ainsi le volume de données accessible. Pour y remédier, nous avons mis en place un système de requêtes échelonnées et un cache des résultats afin d'éviter les appels redondants.
 - **Portée des données** : Certaines APIs ne proposaient ni mises à jour en temps réel, ni des informations détaillées sur des secteurs d'activité de niche. Pour pallier cela, nous avons complété les données issues des APIs par d'autres méthodes de collecte.

5.2.2. Jeux de données publics ouverts (DataSoft, INPI)

Pour élargir la portée de nos données, nous avons intégré des jeux de données publics ouverts issus de plateformes comme DataSoft et INSEE, qui fournissent des informations statistiques et commerciales étendues.

- Pour le RNE, nous avons utilisé le protocole FTP (File Transfer Protocol) via FileZilla pour accéder aux jeux de données en masse. Nous avons aussi expérimenté des scripts Python utilisant la bibliothèque « ftplib » pour automatiser le téléchargement. [44]
- Les jeux de données Opendatasoft ont été récupérés via des téléchargements directs ou des interfaces de type API, selon leur disponibilité. [45]
- Actualisation des données : Les jeux de données n'étaient pas toujours à jour, avec des mises à jour peu fréquentes. Nous avons planifié des contrôles réguliers pour détecter les nouvelles versions et priorisé les sources en temps réel quand c'était possible.

- Incohérences de format : Les jeux de données étaient fournis dans divers formats (CSV, JSON, etc.) nécessitant un prétraitement pour les standardiser avant intégration. Nous avons développé des parseurs personnalisés pour gérer efficacement ces variations.

5.2.3. Scraping web (Selenium, BeautifulSoup)

Reconnaissant les limites des API et des jeux de données, nous avons développé des scrapers web personnalisés pour extraire des données dynamiques en temps réel depuis des sources en ligne telles que des annuaires d'entreprises et des listings publics.

➤ **Présentation de l'approche :**

L'objectif était de concevoir un cadre de scraping évolutif, résilient et modulaire pour extraire des données d'entreprises structurées. Notre approche mettait l'accent sur :

- La navigation dans des structures complexes avec plusieurs niveaux de catégories (ex. Villes, services).
- La gestion des contenus chargés dynamiquement et des systèmes anti-bots.
- L'optimisation de l'efficacité et la robustesse du scraping à grande échelle.
- La production de résultats propres, structurés, prêts pour analyse ou intégration. Cela a été réalisé en combinant des outils modernes de scraping, la programmation asynchrone, l'automatisation de navigateur, et des stratégies intelligentes de gestion des données.

➤ **Outils et technologies :**

Nous avons utilisé une pile complémentaire pour traiter différents aspects du scraping [46-47] :

- **Selenium et SeleniumBase :**

- Utilisés pour rendre les sites lourds en JavaScript et interagir avec des éléments dynamiques (boutons, menus déroulants).
- SeleniumBase a renforcé les capacités de furtivité en simulant un comportement plus réaliste du navigateur et en contournant des protections anti-bots comme les CAPTCHA et les bannières cookies.
- Une logique d'attente personnalisée (attentes explicites, pauses aléatoires) garantissait que le contenu soit entièrement chargé avant extraction.

- **Asyncio, Aiohttp et aiohttp socks :**

- Permettaient le scraping parallèle sur plusieurs URLs ou catégories.

- Les requêtes asynchrones autorisaient l'exécution simultanée de tâches (ex. validation de proxies, récupération des liens de sous-catégories), réduisant considérablement le temps total.
- Utilisés avec un sémaphore pour contrôler la concurrence et éviter la surcharge des serveurs.
- **BeautifulSoup (BS4) :**
 - Après rendu via Selenium, BS4 analysait le HTML brut pour extraire des champs comme le nom de l'entreprise, l'adresse, le numéro de téléphone, etc.
 - Offrait une méthode légère et efficace pour extraire le contenu du DOM statique.
- **Pandas :**
 - Servait de base pour la structuration et l'export des données.
 - Transformait les données brutes extraites en objets DataFrame propres.
 - Facilitaient l'export CSV, la nomination horodatée des fichiers et les analyses/visualisations préliminaires (via matplotlib).

➤ **Logique d'extraction des données :**

La logique d'extraction de données a été conçue avec souplesse et résilience, permettant aux scrapers de s'adapter à la fois aux structures statiques et dynamiques couramment rencontrées sur les annuaires d'entreprises en ligne. Les champs essentiels tels que le nom de l'entreprise, l'adresse, le numéro de téléphone, la description et le lien de la fiche étaient systématiquement ciblés.

Dans de nombreux cas, certains de ces champs notamment les numéros de téléphone étaient dissimulés derrière des éléments interactifs comme des boutons ou des modales. Pour gérer ce type de contenu dynamique, Selenium a été utilisé afin de simuler des interactions utilisateur, accompagnées d'attentes minutieusement calibrées, fixes ou aléatoires, garantissant que le contenu soit totalement chargé avant le début de l'extraction.

Une fois les pages rendues, BeautifulSoup était utilisé pour analyser le HTML et extraire les informations à partir du DOM. La logique intégrait des sélecteurs de secours et des conditions de traitement pour prendre en compte les variations structurelles entre les différentes fiches ou sous-catégories. Cette approche permettait de gérer les cas où certains éléments étaient absents ou affichés de manière incohérente.

Tout champ qui ne pouvait pas être extrait correctement était soit ignoré, soit consigné dans les logs, soit remplacé par une valeur par défaut ("N/A") afin de préserver la cohérence du jeu de données. Les fiches étaient traitées en boucle, chaque élément étant analysé

individuellement. Chaque itération était encapsulée dans un bloc try-except, garantissant qu'une fiche malformée ne perturbe pas l'ensemble du lot.

Cette conception modulaire et tolérante aux erreurs a permis aux scrapers de maintenir des performances robustes face à une grande diversité de structures et de comportements de contenu.

➤ **Gestion des proxies et User-Agent :**

Pour éviter le blocage et préserver l'anonymat, nous avons mis en place un système intelligent de rotation des adresses IP et des empreintes de navigateur.

- **Rotation des proxies :**

- Une liste prédéfinie de proxies était validée de manière asynchrone avant usage.
- Chaque requête ou session utilisait un proxy aléatoire.
- Les proxies défaillants étaient détectés via try-except et soit ignorés, soit réessayés avec backoff.

- **Falsification de l'User-Agent :**

- La bibliothèque « fake_useragent » randomisait les chaînes User-Agent des navigateurs (ex. variantes Chrome, Firefox).
- Chaque session navigateur utilisait un agent unique pour simuler différents utilisateurs et appareils.

- **Intégration :**

- Les proxies étaient intégrés dans aiohttp pour les requêtes rapides, et dans Selenium via des options Chrome personnalisées.
- Les headers (User-Agent et autres comme accept-language) étaient injectés directement dans les sessions HTTP ou profils Selenium.

- **Comportement humain simulé :**

- Les délais étaient aléatoires pour simuler le temps de réflexion des utilisateurs.
- Les paramètres du navigateur (ex. désactivation des flags d'automatisation) réduisaient l'empreinte bot.

➤ **Gestion des erreurs et stabilité :**

Une gestion robuste des erreurs était essentielle pour garantir un scraping ininterrompu sur de longues sessions et sur des centaines d'URLs.

Nous avons mis en place une tolérance multi-couches afin de minimiser la perte de données et d'éviter les plantages.

- Des blocs try-except entouraient les segments critiques :
 - Chargement des pages
 - Interactions avec les éléments (ex. clics)
 - Extraction des données d'annonces individuelles
- Les erreurs étaient consignées dans des fichiers ou affichées avec des infos contextuelles (URL, catégorie) pour faciliter le débogage sans interrompre le processus.
- Des timeouts et mécanismes de secours garantissaient que :
 - Les pages lentes ne figeaient pas le scraper (via timeouts Selenium et limites Asyncio).
 - Les pages sans éléments attendus (ex. bouton téléphone) n'entraînaient pas d'exception ; les champs manquants étaient marqués « N/A ».
- Sorties propres des pages paginées : si le bouton « suivant » était absent ou inactif, la boucle se terminait sans erreur.
- Pour éviter les scripts hors de contrôle, des limites de durée ou de nombre de tâches étaient fixées pour les longues exécutions asynchrones. Cette stratégie multi-niveau permettait au scraper de continuer à collecter les données même si certaines pages échouaient, améliorant la robustesse globale du flux de travail.

5.2.4. Outil de scraping et instances EC2

Pour dépasser les limites des méthodes précédentes et augmenter la capacité de collecte, nous avons adopté un outil de scraping spécialisé fourni par Iaweb.dev et déployé sur des instances Amazon EC2.

L'outil, conçu pour l'extraction de données en grand volume, a été déployé sur plusieurs instances EC2 pour paralléliser les opérations et accroître le débit.

Nous avons configuré des serveurs virtuels avec des configurations identiques, bien que l'usage de mots de passe statiques pose un risque de sécurité. Pour contourner la limite d'IP statiques, nous avons réparti les instances entre différentes régions AWS (ex. us-east-1, eu-west-1), chaque membre de l'équipe supervisant une région spécifique.

- **Choix du type d'instance :**

Nous avons adopté une stratégie d'allocation : 60 % t3.micro, 40 % t3.medium. [48]

Tableau 5.1. Types d'instances pour le serveur Windows

Type d'instance	vCPUs	RAM (Go)	Performances réseau (Gbps)	Coût à la demande (us-east-1, \$/h)
T3. micro	2	1	0,06 (pic à 5,09)	~0,0125
T3. small	2	2	0,13 (pic à 5,11)	~0,0250
T3. medium	2	4	0,25 (pic à 4,98)	~0,0500
T3. large	2	8	0,51 (pic à 5,11)	~0,1000

Pour équilibrer coût et performance, nous avons combiné majoritairement des t3.small avec des t3. medium, répondant aux besoins techniques et aux contraintes budgétaires.

- **Optimisation des coûts :**

Les t3. small sont environ 50 % moins coûteux que les t3.medium, ce qui permet d'exécuter plus de processus de scraping parallèles pour un budget donné. Cette priorisation réduit les coûts sans sacrifier la couverture des données.

- **Équilibre des performances :**

L'intégration des t3.medium garantit une puissance CPU et mémoire suffisante pour améliorer le taux de réussite des tâches, là où les t3.small peuvent être insuffisants.

- **Répartition de la charge :**

Cette configuration supporte une architecture évolutive et distribuée. Les t3.small assurent le gros du travail, tandis que les t3.medium prennent en charge les tâches complexes ou critiques.

- **Scalabilité et flexibilité :**

L'approche mixte facilite la scalabilité verticale et horizontale. Plus de t3.micro peuvent être ajoutés pour les volumes élevés, tandis que les t3.medium gèrent les opérations complexes ou centralisées.

- **Exclusion des t3.large :**

Bien que considérées, les t3.large ont été écartées à cause de leur coût élevé et de leur surcapacité par rapport à la charge typique. Dans une architecture distribuée, leurs ressources auraient été sous-utilisées.

- **Configuration des machines :**

- AMI : Windows Server 2025 Base (64 bits x86) fourni par AWS, personnalisé avec l'outil de scraping et ses dépendances.
- Configuration des mots de passe et IP : mot de passe administrateur statique configuré via un fichier YAML d'initialisation pour faciliter l'exploitation par l'équipe. Des IP statiques ont été attribuées pour éviter de devoir télécharger un nouveau fichier RDC à chaque redémarrage, limitées à 5 Elastic IP par région.

- **Suivi des ressources et facturation :**

Pour réduire les coûts, nous avons simplifié la gestion en évitant des services payants comme AWS CloudWatch. Chaque membre surveillait les instances dans sa région, vérifiant régulièrement l'état des tâches et des ressources consommées.

Le suivi des coûts a été fait via AWS Cost Explorer, et les instances étaient arrêtées après 8 heures pour limiter les dépenses. Nous avons aussi choisi les régions les moins chères pour nos types d'instances et IP.

- **Collecte et ingestion des données :**

Chaque membre récupérait les données de sa région, les compressait, puis les déposait sur un disque partagé. Nous n'utilisons pas S3 pour limiter les coûts durant la phase de test et production.

Un outil ETL ingérait les données locales dans le système applicatif. Des tableaux de suivi permettaient de tracer les données collectées.

5.2.5. Nettoyage et enrichissement des données

Le processus de nettoyage et d'enrichissement a été réalisé via trois modules spécialisés, chacun traitant des défis spécifiques de qualité des données :

Standardisation des données clients :

Utilisation d'un appariement flou avancé avec la bibliothèque RapidFuzz pour standardiser les contacts français. Correction des noms de villes, uniformisation des références pays, et complétion des informations géographiques manquantes via des algorithmes de cartographie.

Le système traitait les données par lots pour optimiser la mémoire et conservait des statistiques de correction tout au long du nettoyage.

Enrichissement des données entreprises :

Focalisé sur la découverte intelligente d'emails et la cartographie géographique. Plusieurs stratégies de recherche d'emails ont été mises en œuvre (recherches WHOIS, extraction du

contenu des sites web) avec validation pour garantir qu'ils étaient des contacts d'affaires légitimes et non des adresses sociales ou spam.

Le module enrichissait aussi les enregistrements d'entreprises avec des données de département et région françaises via des correspondances ville-géographie.

Protection des données sensibles :

Gestion de la confidentialité via un chiffrement des champs sensibles (informations personnelles). Utilisation de la bibliothèque cryptographique Fernet pour assurer un chiffrement/déchiffrement sécurisé tout en maintenant l'utilisabilité des données dans les flux de traitement.

Tous les modules incluaient des mécanismes de mise en cache, un traitement parallèle et une gestion complète des erreurs. Ils supportaient à la fois les structures DataFrame et dictionnaires, rendant leur intégration flexible dans divers scénarios de pipeline, tout en produisant des logs et statistiques détaillées pour monitorer les améliorations de qualité.

Pour analyser la qualité des données et communiquer les résultats du nettoyage, nous avons utilisé Power BI comme outil exploratoire et de reporting, complété par des visualisations intégrées dans notre application web via Recharts. Power BI nous a permis d'explorer les jeux de données bruts pour identifier les problèmes, lacunes et zones à traiter avant nettoyage. Des tableaux de bord suivaient la progression du nettoyage, les taux d'achèvement, la détection des doublons, et le succès de la standardisation selon les sources.

5.3. Développement de l'application web

Deux applications ont été développées simultanément au départ. L'objectif était de garantir qu'au moins une application fonctionnelle soit disponible pour des démonstrations, et de pouvoir comparer laquelle fonctionnait le mieux. Notre consensus a été que le choix de Spring Boot était plus mature et mieux aligné avec nos objectifs de montée en charge de la plateforme DataPull.

5.3.1. Application Python

Cette application a été développée en utilisant FastAPI, PostgreSQL, et une interface basée sur HTML/JS/CSS. Elle a très bien fonctionné avec les faibles exigences de la première phase du projet.

FastAPI, un framework web Python moderne, a été choisi pour ses hautes performances et ses capacités asynchrones. Il gère les points d'accès RESTful pour fournir les données et

l'interface. L'application utilisait SQLAlchemy comme ORM pour interagir avec une base de données PostgreSQL, où toutes nos données collectées étaient stockées.

Les données étaient ingérées grâce à un script Python optimisé afin de ne pas surcharger la machine. En surveillant les ressources, le script s'arrêtait temporairement jusqu'à ce que les ressources soient de nouveau disponibles. De plus, le nombre de workers était calculé automatiquement en fonction des ressources disponibles. Cela assurait une ingestion saine des données dans la base PostgreSQL.

Le frontend a été conçu pour être facile d'utilisation, communiquant et expliquant tous les services de manière fluide.

Cette application a été déployée avec Docker sur une instance EC2 pour l'hébergement, utilisant des scripts bash pour configurer automatiquement la machine hôte en exécutant les mises à jour et les téléchargements nécessaires.

5.3.2. Application Java

a. Backend

Nous avons choisi de poursuivre avec l'application Spring Boot et d'y intégrer des éléments de l'interface de l'application Python. Au départ, cette application était unique et gérant les opérations CRUD ainsi qu'une interface simple. Par la suite, nous avons créé plusieurs applications pour aboutir à une architecture microservices.

Le backend est composé de trois microservices principaux : l'application CRUD, l'application de gestion des utilisateurs, et l'application de gestion des scripts. Ces microservices sont interconnectés et communiquent via des API RESTful, avec le stockage et la recherche des données assurés par Elasticsearch pour des requêtes haute performance, et Amazon S3 pour un stockage de fichiers évolutif.

L'utilisation de Spring Boot facilite l'intégration avec Elasticsearch via Spring Data Elasticsearch, tandis que les microservices sont hébergés sur des instances AWS EC2, garantissant évolutivité et fiabilité.

L'architecture est conçue pour soutenir les objectifs de la plateforme DataPull, notamment la génération efficace de leads, le traitement des données et l'interaction utilisateur, en mettant l'accent sur la maintenabilité et l'évolutivité future.

b. Application CRUD/Recherche

❖ Objectif et fonctionnalité :

L'application CRUD (Créer, Lire, Mettre à jour, Supprimer) constitue l'interface principale pour la gestion et la recherche de données au sein de la plateforme DataPull. Elle

prend en charge l'ingestion, la récupération, la mise à jour et la suppression de données B2C et B2B, telles que les profils d'utilisateurs individuels et les informations sur les entreprises, stockées dans un cluster Elasticsearch. Cette microservice est essentielle pour permettre des fonctionnalités avancées de recherche et de gestion de données, prenant en charge à la fois les opérations CRUD de base et des requêtes complexes pour la génération de leads. La fonctionnalité de recherche avancée permet des requêtes dynamiques sur plusieurs champs, comme le prénom, le nom, l'email, la ville actuelle ou le secteur d'activité, offrant des résultats pertinents et paginés pour répondre aux besoins de recherche simples ou sophistiqués.

❖ Framework et dépendances :

L'application est construite avec Spring Boot, en s'appuyant sur Spring Data Elasticsearch pour interagir de manière transparente avec le cluster Elasticsearch. Les entités pour les profils B2C et B2B sont annotées avec `@Document` pour être mappées aux indices Elasticsearch, garantissant une persistance et une recherche efficaces des données. Le client Java ElasticsearchClient améliore les performances de recherche en permettant la construction dynamique de requêtes, telles que MatchQuery avec tolérance aux fautes de frappe, PrefixQuery et BoolQuery, pour des recherches flexibles et robustes. Les dépendances incluent des configurations SSL sécurisées et des identifiants d'authentification définis dans ElasticsearchConfig, assurant une communication chiffrée avec le cluster lors des opérations de recherche et CRUD.

❖ Opérations clés :

- **Créer/Mettre à jour :**

Les méthodes `saveUser()` et `saveEntreprise()`, implémentées respectivement dans `B2CService` et `B2BService`, utilisent la méthode `save()` de `ElasticsearchRepository` pour indexer ou mettre à jour les documents. Cela garantit que les nouveaux profils ou ceux modifiés sont immédiatement disponibles pour la recherche.

- **Lire :**

L'application permet la récupération par ID (`findById()`) et la récupération en masse (`findAll()`). La méthode `searchByAttributes(Map<String, String>)` prend en charge des recherches dynamiques sur plusieurs champs en utilisant les types de requêtes Elasticsearch, tels que MatchQuery (avec tolérance aux fautes de frappe), PrefixQuery, WildcardQuery et BoolQuery pour combiner des conditions. Par exemple, une recherche pour "Ali" à "Paris" construit une BoolQuery combinant des correspondances floues sur `firstName` et `currentCity`, triées par pertinence (`_score`) et paginées via `NativeQuery`.

- **Supprimer :**

Les documents sont supprimés par ID avec `deleteById()`, et la suppression en masse est prise en charge via `deleteAll()`, annotée avec `@Async` pour une exécution non bloquante.

- **Compter :**

La méthode `countUsers()` fournit des statistiques pour les tableaux de bord, exploitant les capacités de comptage efficaces d'Elasticsearch.

❖ **Recherche avancée :**

La fonctionnalité de recherche avancée est un atout majeur de l'application, permettant des requêtes dynamiques et intelligentes sur les profils B2C et B2B stockés dans le cluster Elasticsearch. La méthode `searchByAttributes(Map<String, String>)` prend en charge des recherches sur plusieurs champs, comme le prénom, le nom, l'email, la ville actuelle, l'entreprise actuelle, le secteur d'activité et le genre, offrant des résultats hautement pertinents pour la génération de leads.

Construction dynamique des requêtes : Les champs fournis par l'utilisateur sont transformés en sous-requêtes Elasticsearch, utilisant :

- **MatchQuery** avec tolérance aux fautes de frappe pour gérer les erreurs (par exemple, "Parls" correspond à "Paris").
- **PrefixQuery** pour les correspondances partielles, idéal pour les emails ou les noms d'entreprises.
- **WildcardQuery** pour des correspondances flexibles sur des champs comme la ville.
- **TermQuery** pour des correspondances exactes sur des champs comme `email.keyword`.
- **BoolQuery** pour combiner les sous-requêtes avec des conditions `must` ou `should`.

Par exemple, une recherche pour `firstName = "Ali"` et `currentCity = "Paris"` génère une **BoolQuery** combinant des correspondances floues sur ces champs, garantissant une tolérance aux fautes et aux correspondances partielles.

❖ **Flux de recherche :**

- L'utilisateur soumet une requête via l'interface frontend basée sur React (par exemple, "Ali" à "Paris").
- Le `@RestController` transmet la requête à la méthode `searchByAttributes()` dans `B2CService` ou `B2BService`.
- Le service construit une **BoolQuery** avec les sous-requêtes appropriées.
- Une **NativeQuery** est créée, triée par pertinence (`_score`) et paginée.
- La requête est exécutée via `elasticsearchOperations.search()`.

- Les résultats sont renvoyés sous forme de liste d'objets B2C ou B2B au format JSON.

❖ **Performance et sécurité :**

Les recherches sont optimisées pour la performance grâce à Spring Data Elasticsearch et le client Java ElasticsearchClient. Les résultats sont paginés et triés par pertinence, améliorant l'expérience utilisateur. L'application se connecte de manière sécurisée au cluster Elasticsearch via ElasticsearchConfig, utilisant des certificats SSL et des identifiants d'authentification pour garantir une communication chiffrée.

❖ **Intégration :**

L'application CRUD expose des points de terminaison RESTful via un @RestController, permettant à l'interface frontend basée sur React d'effectuer des opérations telles que la recherche de prospects, la mise à jour de profils ou la récupération de données. La fonctionnalité de recherche avancée est accessible via des points de terminaison dédiés qui acceptent des paramètres de requête multi-champs, renvoyant des réponses JSON paginées pour un rendu fluide dans le frontend. L'application s'intègre de manière sécurisée avec le cluster Elasticsearch via ElasticsearchConfig, en utilisant des certificats SSL et des identifiants d'authentification pour garantir une communication chiffrée lors des opérations de recherche et de gestion de données. Cette intégration prend en charge à la fois les requêtes interactives des utilisateurs et les processus automatisés, faisant de l'application un composant polyvalent de la plateforme DataPull.

c. **Application Scripts**

❖ **Objectif et Fonctionnalité :**

L'application Scripts permet d'exécuter des scripts personnalisés pour la recherche, la validation et l'enrichissement d'adresses email. Son rôle est de trouver ou de vérifier dynamiquement des adresses email grâce à des algorithmes spécialisés ou des services tiers. Elle fait le lien entre les entrées utilisateurs du frontend et la couche de données gérée par l'application CRUD. Cette application améliore la génération de leads en automatisant la découverte des contacts manquants ou non vérifiés. Elle fonctionne en deux phases principales :

- **Génération d'emails :** À partir des informations fournies par l'utilisateur (par exemple nom de domaine, prénom et nom), le service crée intelligemment des combinaisons d'emails probables selon des modèles prédéfinis.
- **Validation et enrichissement** des emails générés.

❖ **Framework et dépendances :**

Développée avec Spring Boot, cette application intègre des scripts Java personnalisés ou des API externes.

❖ **Intégration :**

L'application Scripts communique avec le frontend via des API REST pour recevoir les requêtes de recherche d'email ou de profil et retourner les résultats. Elle échange aussi avec l'application CRUD via des appels REST inter-services pour vérifier l'existence d'un email dans la base Elasticsearch, récupérer des entrées existantes ou stocker de nouvelles données après validation.

d. Application de gestion des utilisateurs

❖ **Objectif et Fonctionnalité :**

Ce microservice gère les comptes utilisateurs, l'authentification et les accès basés sur les rôles pour la plateforme en utilisant Keycloak [49]. Il gère l'inscription, la connexion, la gestion des profils, l'attribution des rôles et l'autorisation sécurisée. Ce service est fondamental pour toute plateforme nécessitant une gestion sécurisée du cycle de vie des utilisateurs et un contrôle administratif.

❖ **Framework et dépendances :**

Construit avec Spring Boot, intégré à Keycloak pour l'authentification et l'autorisation. Utilise JpaRepository pour la persistance des entités Utilisateur et Rôle, et implémente OpenID Connect et des tokens JWT pour sécuriser l'accès aux API.

❖ **Fonctionnalités clés :**

- **Création :** Les utilisateurs s'inscrivent via Keycloak ou sont créés par API admin. La synchronisation des rôles est gérée entre Keycloak et la base locale.
- **Lecture :** Les utilisateurs accèdent à leurs profils, les admins peuvent lister, rechercher par nom d'utilisateur/email ou filtrer par rôle.
- **Mise à jour :** Les utilisateurs modifient leurs données personnelles et mots de passe via Keycloak, les admins peuvent modifier les rôles ou désactiver des comptes.
- **Suppression :** Les admins peuvent désactiver ou supprimer des utilisateurs à la fois localement et dans Keycloak.

❖ **Authentification et sécurité :**

- Authentification par token JWT délivrée par Keycloak.
- Contrôle d'accès via annotations `@PreAuthorize` pour gérer les rôles (ROLE_USER, ROLE_ADMIN).

- Gestion complète des mots de passe et identifiants par Keycloak, assurant une protection robuste.

❖ **Intégration :**

Des API REST permettent une interaction fluide avec les frontends ou panneaux d'administration. Ce service agit comme un serveur de ressources validant les tokens Keycloak et appliquant les politiques d'accès. Il supporte une intégration modulaire avec des systèmes SaaS ou RH plus larges.

5.3.3. Application Front-end React

Le frontend de l'application DataPull est développé avec React, une bibliothèque JavaScript puissante, largement adoptée pour construire des interfaces utilisateurs dynamiques et réactives. L'architecture basée sur les composants permet une construction modulaire de l'interface, favorisant la réutilisabilité du code et la facilité de maintenance. Cela garantit que différentes parties de l'interface (formulaires, éléments de recherche, cartes, affichage des résultats) peuvent être développées indépendamment puis intégrées de façon fluide.

❖ **UI/UX :**

Application respecte les principes modernes de design UI/UX : simplicité, cohérence, accessibilité et réactivité. Ces principes se traduisent par une interface épurée, une typographie lisible, une palette de couleurs harmonieuse et une navigation fluide.

L'interface utilise une combinaison de Bootstrap et Tailwind CSS pour offrir une expérience utilisateur à la fois élégante et cohérente. Bootstrap est utilisé principalement pour sa grille responsive, ses composants interactifs prêts à l'emploi (modals, tooltips, carrousels, etc.) Et sa structure HTML standardisée. Tailwind CSS, quant à lui, apporte une grande flexibilité grâce à son approche utilitaire, permettant de personnaliser finement les espacements, les couleurs (teal, gris, blanc), la typographie ou encore les boutons, sans écrire de CSS personnalisé. L'utilisation conjointe de ces deux frameworks permet de combiner rapidité de développement, personnalisation avancée et uniformité visuelle sur toute la plateforme.

❖ **Bibliothèques et fonctionnalités :**

- **Axios** est utilisé pour les requêtes HTTP vers l'API backend, simplifiant l'envoi/récupération des données, la gestion des erreurs et des configurations de requêtes. Cela maintient la synchronisation en temps réel entre frontend et backend.

- **React Router** gère la navigation côté client, permettant de passer d'une section à l'autre (page d'accueil, recherche, tarifs, fonctionnalités) sans rechargement complet, offrant une expérience fluide de type application monopage (SPA).

❖ **Analyse des composants :**

- **Barre de recherche** : Permet aux utilisateurs d'entrer des critères (prénom, nom, ville, genre) avec option de recherche exacte ou approximative (fuzzy).
- **Grille de résultats** : Affiche les résultats de recherche sous forme structurée (tableau ou cartes), facilitant la comparaison, le filtrage et l'interprétation.
- **Carte interactive** : Réalisée avec la bibliothèque Leaflet, centrée sur la France, elle propose regroupement (clustering) des marqueurs, zoom, déplacement, et des marqueurs cliquables avec détails.
- **Cartes fonctionnelles** : Sur la page d'accueil et les pages fonctionnalités, des cartes résument les services clés avec icônes et descriptions courtes.
- **Tableaux tarifaires** : Présentent clairement les abonnements, avantages et coûts associés.
- **Tableaux de bord analytiques** : Incluent graphiques et indicateurs clés pour offrir aux utilisateurs une vision sur leur utilisation ou résultats.

5.3.4. Développement de l'application ETL

❖ **Vue d'ensemble**

L'application ETL (Extract, Transform, Load) est une plateforme robuste de traitement de données conçue pour gérer efficacement des opérations à grande échelle. Elle supporte divers formats (CSV, JSON, APIs) et intègre des fonctionnalités avancées telles que la découverte intelligente d'emails, la supervision en temps réel, et l'optimisation de performance pour des fichiers jusqu'à 10 Go.

La solution repose sur un stack technologique moderne comprenant Flask, Celery, Redis, MongoDB, SQL, Elasticsearch, et une architecture modulaire containerisée avec Docker garantissant scalabilité et facilité de déploiement.

❖ **Architecture système**

L'architecture est modulaire et scalable, composée de :

- **Framework web et API** : Flask 2.0.1 pour fournir l'interface utilisateur et les endpoints RESTful pour soumission, suivi et récupération des résultats des jobs.
- **Traitement des tâches** : Celery pour la gestion asynchrone, Redis comme broker de messages, Celery Beat pour les jobs planifiés.

- **Stockage des données** : Approche polyglotte avec MongoDB pour les données semi-structurées, bases SQL via SQLAlchemy pour les données relationnelles, et Elasticsearch pour la recherche textuelle et l'analyse.
- **Pipeline ETL modulaire** :
 - **Extract** : Extraction de données depuis API, bases, fichiers (CSV, JSON, Excel).
 - **Transform** : Mappage, dérivations, agrégations, filtrage.
 - **Load** : Chargement dans bases, fichiers, APIs.

Cette architecture est conçue pour être hautement configurable et scalable, capable de traiter des datasets de toutes tailles via des composants adaptables et un déploiement Dockerisé.

❖ **Fonctionnalités et workflow**

- **Interaction utilisateur** : Interface web pour upload de fichiers, configuration des jobs ETL, tableau de bord santé système, pages dédiées aux processus ETL clients/entreprises, suivi des jobs, tests API, uploads rapides.
- **Soumission des jobs** : Fichiers stockés sous `/app/uploads/streaming/`. Tâches envoyées à Celery, assignées à des queues standard ou segmentées selon la taille. Options configurables (destination, taille des lots, méthode de recherche d'email).
- **Traitement des tâches** : Workers Celery traite les tâches, les gros fichiers sont découpés en segments mémoire-limités. Les transformations incluent nettoyage, enrichissement, découverte d'emails via 4 méthodes spécialisées. Statistiques temps réel stockées dans Redis.
- **Chargement des données** : Données traitées chargées vers destinations configurées, avec traitement par lots, gestion des transactions et opérations idempotentes pour garantir intégrité.
- **Supervision** : Monitoring temps réel via gestion des tâches, API et outils comme Flower pour un suivi avancé.

❖ **Configuration et installation**

- **Installation** : Déploiement via Docker (`docker-compose -f docker-compose.dev.yml up -d`) ou installation manuelle avec dépendances puis lancement (`python app/main.py`).
- **Variables d'environnement** : Gestion via `config.py`, `docker-compose.yml` et `.env` pour config API, Redis, et paramètres de traitement.
- **Configuration workers** : Gunicorn pour requêtes web, Celery pour tâches de fond, nombre de workers ajustable selon ressources et charge.

- **Options de traitement** : Configuration via interface web ou API, paramétrage de destination, taille de batch, méthodes de recherche d'email.
- ❖ **Optimisation des performances**
 - **Réglages** : Taille dynamique des lots adaptée aux données, optimisation des workers pour tâches I/O, cache intelligent et pooling HTTP pour accélérer le traitement.
 - **Gestion mémoire** : Traitement en segments pour gros fichiers, limite mémoire à 2 Go par worker enfant, timeouts et restrictions sur la taille du contenu.
 - **Montée en charge** : Supporte montée horizontale (ajout de machines/conteneurs) et verticale (augmentation ressources), configurations Docker Compose pour scalabilité facile.
 - **Supervision pour optimisation** : Dashboards temps réel et outils comme Flower fournissent des métriques pour ajuster workers, mémoire et autres paramètres.

5.4. Résultats et mises à jour futures

Le développement de la plateforme DataPull a abouti à des réalisations significatives sur plusieurs plans, démontrant la mise en œuvre réussie d'une solution complète axée sur les données pour la génération de leads et l'intelligence économique.

5.4.1. Application web

a. Page principale

Cette page constitue la vitrine commerciale de la plateforme DataPull, mettant en avant l'automatisation de la prospection de leads. Elle présente les fonctionnalités clés sous forme de modules interactifs, tels que la recherche personnelle, la recherche géographique, la détection d'e-mails professionnels et le traçage d'entreprises. L'interface est épurée et intuitive, orientée vers une expérience utilisateur fluide. Elle permet également un accès rapide aux services grâce à des boutons d'appel à l'action clairs.

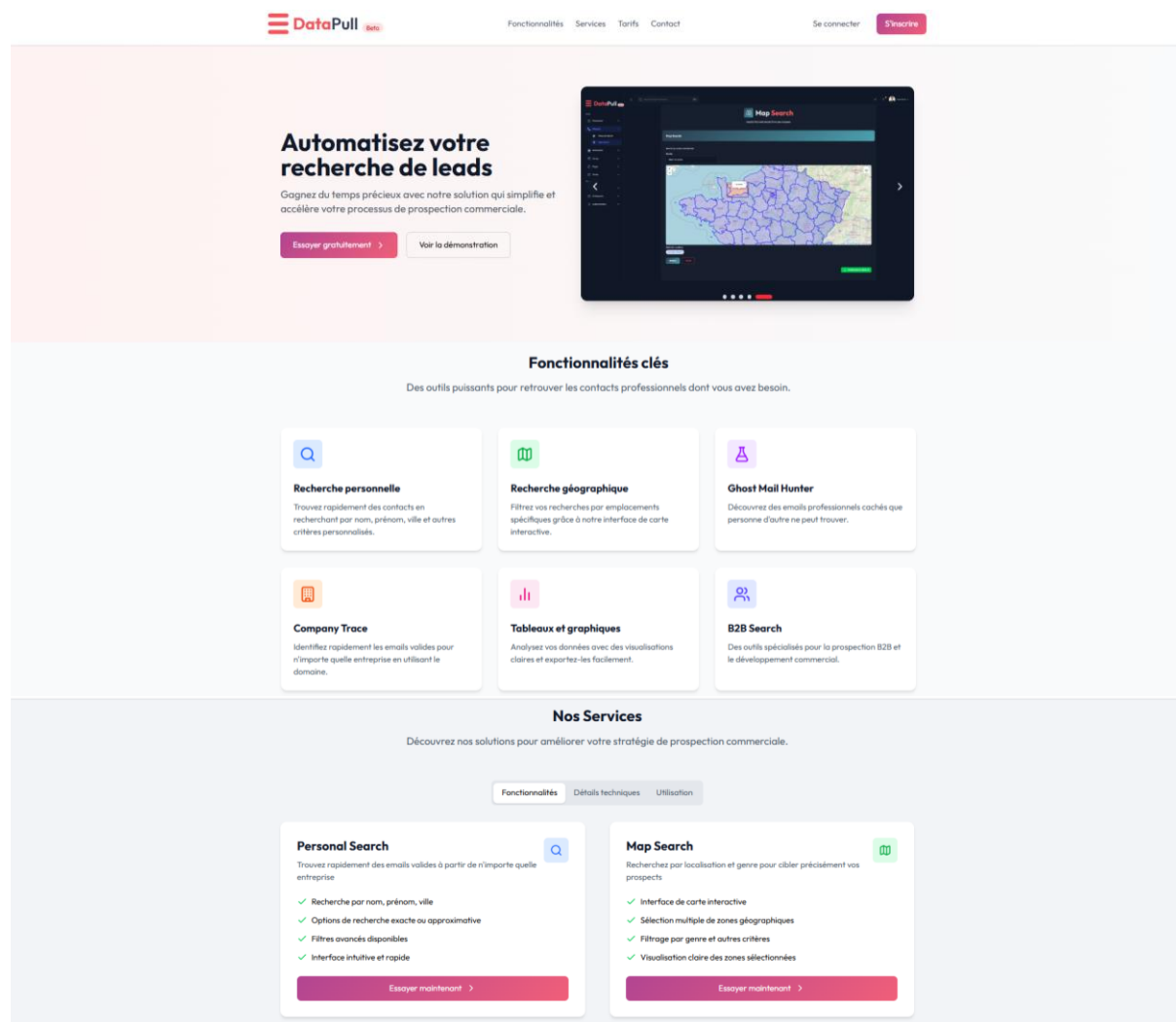


Figure 5.2. Page principale de DataPull

b. Page de tableau de bord

Cette page propose une interface claire pour orienter l'utilisateur vers la solution la plus adaptée à ses besoins : B2B pour les entreprises ou B2C pour les particuliers. Elle met en avant la simplicité de gestion des données grâce à la plateforme DataPull, qui permet d'organiser, rechercher et visualiser facilement les contacts. L'utilisateur peut ainsi choisir rapidement une solution et démarrer avec un essai gratuit. L'approche centrée sur l'utilisateur favorise une navigation intuitive et une prise de décision rapide.

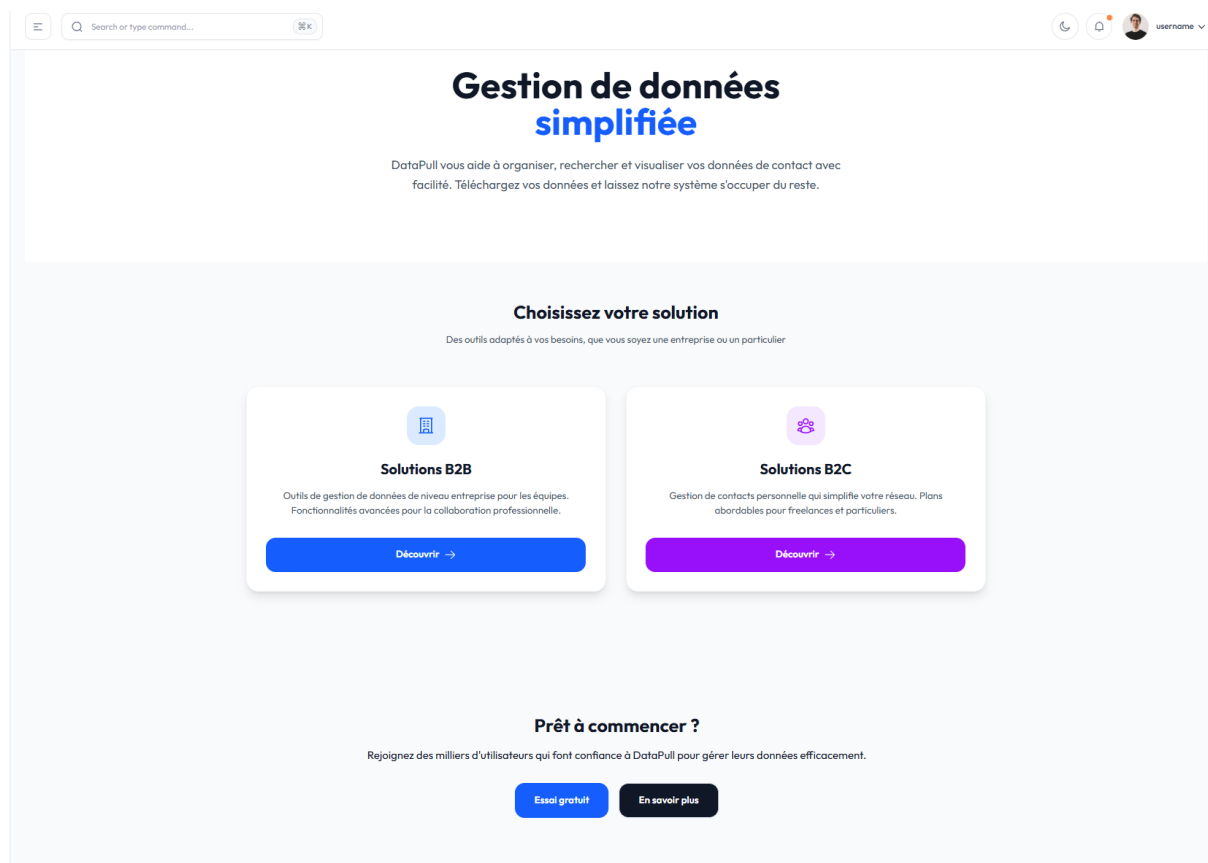


Figure 5.3. Page de tableau de bord

c. Page de recherche personnalisée B2C

Les pages Personal Search de DataPull présentent un outil de recherche personnalisée avec un formulaire comprenant les champs Prénom, Ville, Pays et Nom de famille, offrant des options de recherche exacte ou approximative via des boutons "Rechercher" et "Effacer", puis affichent les résultats sous forme de tableau détaillé contenant environ 15 contacts avec des colonnes complètes (Utilisateur, Genre, Localisation, Département, Région, Numéro de téléphone, Adresse e-mail, Lieu de travail, Statut relationnel, Localisation complète) où certaines données sont spécifiées (noms, villes comme Paris, statuts "Marié") tandis que d'autres affichent "Not specified", le tout avec des options d'export Excel/PDF pour faciliter l'exploitation des données de prospection collectées.

UTILISATEURS	GENRE	LOCALISATION	DÉPARTEMENT	RÉGION	NUMÉRO DE TÉLÉPHONE	ADRESSE E-MAIL	LIEU DE TRAVAIL	STATUT RELATIONNEL	LOCALISATION DOMICILE
Okenson Destiné	Male	Paris (Paris)			...	...	Not specified		Fra
Salah Mtb	Male	Paris (Paris)			...	...	Not specified		France)
Zopia Kemite Deh	Female	Paris (Paris)			...	...	Not specified		France)
Urmil Deb	Female	Paris (Paris)			...	...	Not specified		France) Montreuil.
Mourad Reikik	Male	Paris (Paris)			...	...	Not specified		France)
Mymy Bel	Female	Paris (Paris)			...	...	Not specified		France)
Retha Sim	Male	Paris (Paris)			...	...	Not specified		France)
Annie Coché-Brantonne	Female	Paris (Paris)			...	...	Not specified		France)
Thomas StrikeTeam	Male	Paris France	Paris	Île-de-France	...	...	the publisher's corporation		Mulhouse France
Jean Paul		Paris France	Paris	Île-de-France	...	...	california		Los Angeles France
Jean Michel Force	Male	Paris France	Paris	Île-de-France	...	...	Not specified		
Elnalem Poap	Female	Paris France	Paris	Île-de-France	...	...	Not specified		
Charlotte Poujol		Paris France	Paris	Île-de-France	...	...	sarawak		Toulouse France
Véronique Kurtz	Female	Paris France	Paris	Île-de-France	...	...	Not specified		Paris France
Louis Aubert	Male	Paris France	Paris	Île-de-France	...	...	mairie de paris		Lyon France
Hassan Khelifi	Male	Paris France	Paris	Île-de-France	...	...	Not specified		Tunis Tunisia
Jean Claude Denis	Male	Paris France	Paris	Île-de-France	...	...	Not specified		Nugododa France
.....		Paris	Paris	Île-de-France	...	...			Paris

Figure 5.4. Page de Recherche personnalisée B2C

d. Page de recherche géographique B2C

Les pages Map Search de DataPull présentent un outil de recherche géographique avec une carte interactive de l'Europe où la France est divisée en zones colorées, un formulaire de recherche par localisation et genre, puis affichent les résultats sous forme de tableau montrant 28008 résultats avec 50 contacts affichés incluant leurs informations personnelles (noms, genre, localisation, téléphone, email, lieu de travail) pour les régions sélectionnées comme Limoges et Nouvelle-Aquitaine, avec des options d'export CSV disponibles.

Map Search
Trouvez rapidement les numéros de téléphone de toute personne en France.

Rechercher par localisation et genre

Genre
Select an option

Emplacements sélectionnés:
Limoges Centre-Val de Loire (Region) Nouvelle-Aquitaine (Region)

RECHERCHER EFFACER

TÉLÉCHARGER CSV

Résultats de la recherche 28008 Résultats 50 Affichés

UTILISATEURS	GENRE	LOCALISATION	DÉPARTEMENT	RÉGION	NUMÉRO DE TÉLÉPHONE	E-MAIL	LIEU DE TRAVAIL	STATUT RELATIONNEL	LOCALISATION DOMICILE
Muriel Martinet	Female	Limoges France	Haute-Vienne	Nouvelle-Aquitaine	...	...	Not specified		
Grégoire Joliet	Male	Limoges France	Haute-Vienne	Nouvelle-Aquitaine	...	...	srfs		Barbenfaine France
Pierre-Alain Bernard	Male	Limoges France	Haute-Vienne	Nouvelle-Aquitaine	...	...	bull		Caillon-En-Médoc France
Dardillac Yolande	Female	Limoges France	Haute-Vienne	Nouvelle-Aquitaine	...	...	la poste (France)		Limoges France
Noëlle Depierre Thérénin	Female	Limoges France	Haute-Vienne	Nouvelle-Aquitaine	...	...	Not specified		Limoges France
Gilles Gauthier	Male	Limoges France	Haute-Vienne	Nouvelle-Aquitaine	...	...	Not specified		Limoges France
Amani Chatli	Female	Limoges France	Haute-Vienne	Nouvelle-Aquitaine	...	...	collège anatole France		Limoges France

Figure 5.5. Page de recherche géographique B2C

e. Page de recherche d'emails professionnels

La page Ghost Mail Hunter présente un outil de détection d'emails professionnels cachés. L'interface comprend un formulaire de recherche avec des champs Prénom, Nom et Domaine (exemple.com), suivi d'un bouton "Hunt Ghost Emails" pour lancer la recherche, et une section "E-mails trouvés" pour afficher les emails trouvés.

f. Page de recherche d'emails d'entreprise

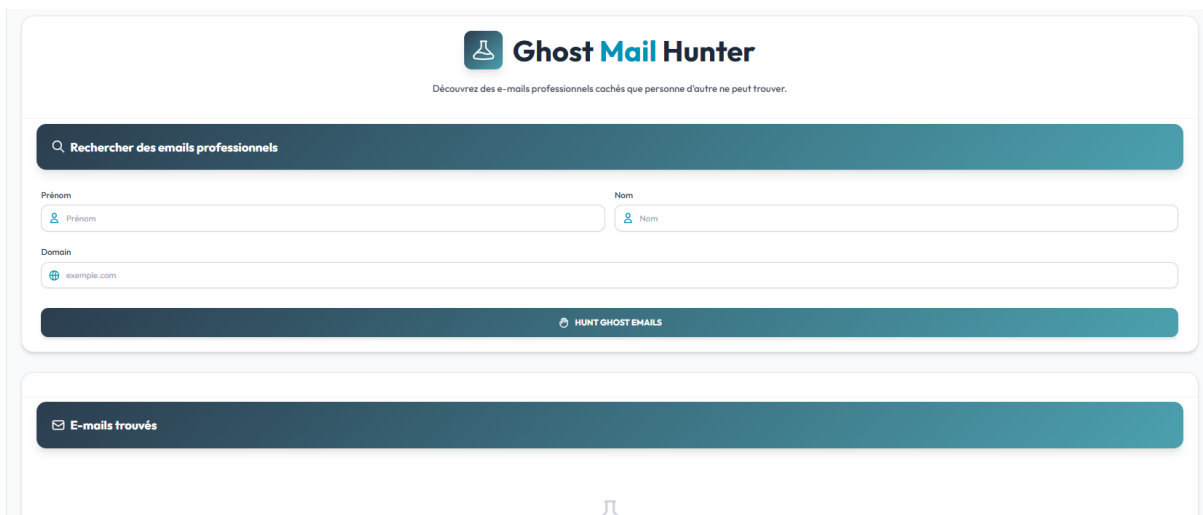


Figure 5.6. Page de recherche d'emails professionnels

La page Company Trace présente un outil de recherche d'emails d'entreprise permettant de découvrir tous les emails importants d'une société, notamment les adresses de contact principales et les emails stratégiques pour la prospection commerciale. L'interface propose un formulaire simple avec un champ "Domaine de l'entreprise" et un bouton "Search for Emails", accompagné d'une section "E-mails valides" qui affiche e-mail valide trouvées.

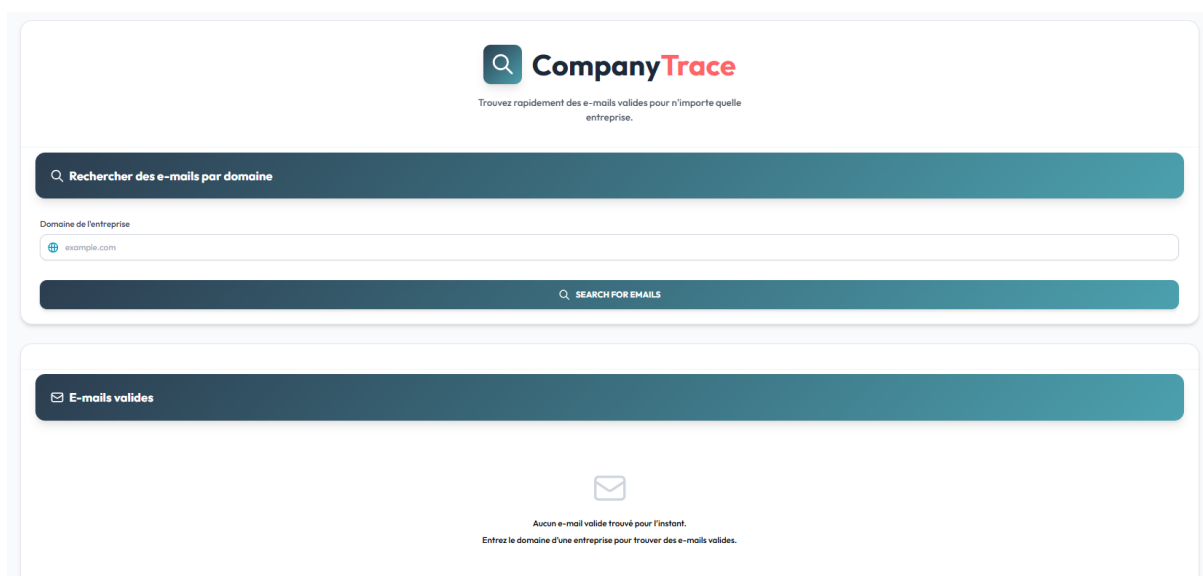


Figure 5.7. Page de recherche d'emails d'entreprise

g. Page de Recherche d'entreprises B2B

La page "Recherche d'entreprises B2B" présente un outil de prospection d'entreprises. Elle permet à l'utilisateur d'effectuer une recherche d'entreprises sur une carte interactive en filtrant par nom, catégorie, ville, score ou notation. Les résultats sont affichés à gauche sous forme de liste, tandis qu'une carte à droite localise les entreprises avec des marqueurs cliquables. Chaque entreprise peut être visualisée avec ses coordonnées, horaires et avis. Cette interface facilite l'identification rapide de prospects B2B pertinents selon leur position géographique et leur réputation.

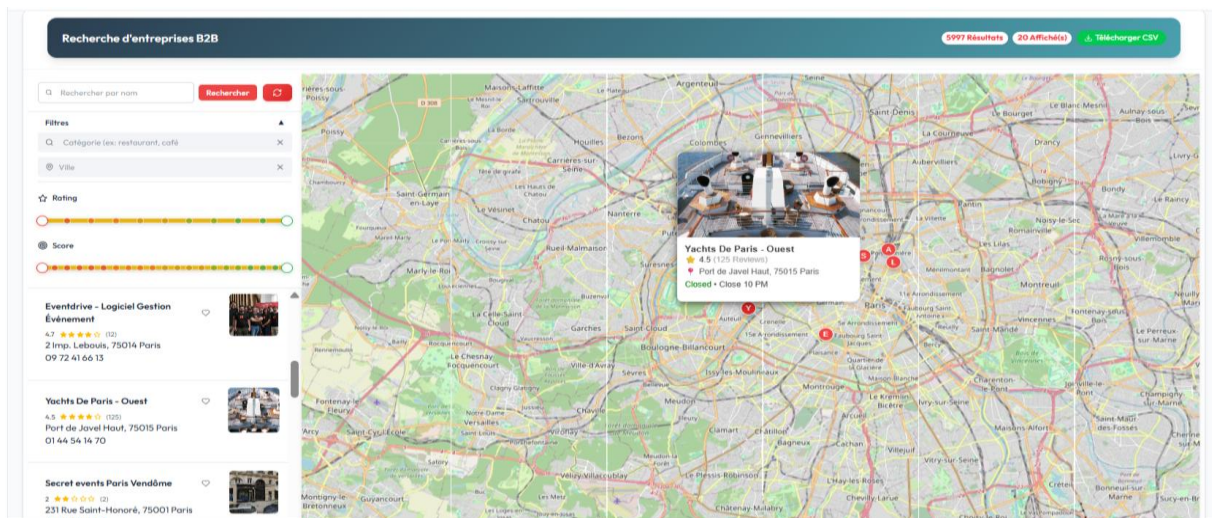


Figure 5.8. Page dev Recherche d'entreprises B2B

La fenêtre popup détaillée d'entreprise avec photo de couverture, note étoilée, nombre d'avis et score de qualité. Le popup affiche la catégorie d'activité, une description complète des services, et les informations de contact essentielles (adresse, téléphone, site web) ainsi qu'une section heures d'ouverture, permettant aux utilisateurs d'obtenir rapidement toutes les données nécessaires pour leur prospection commerciale directement depuis l'interface de recherche.

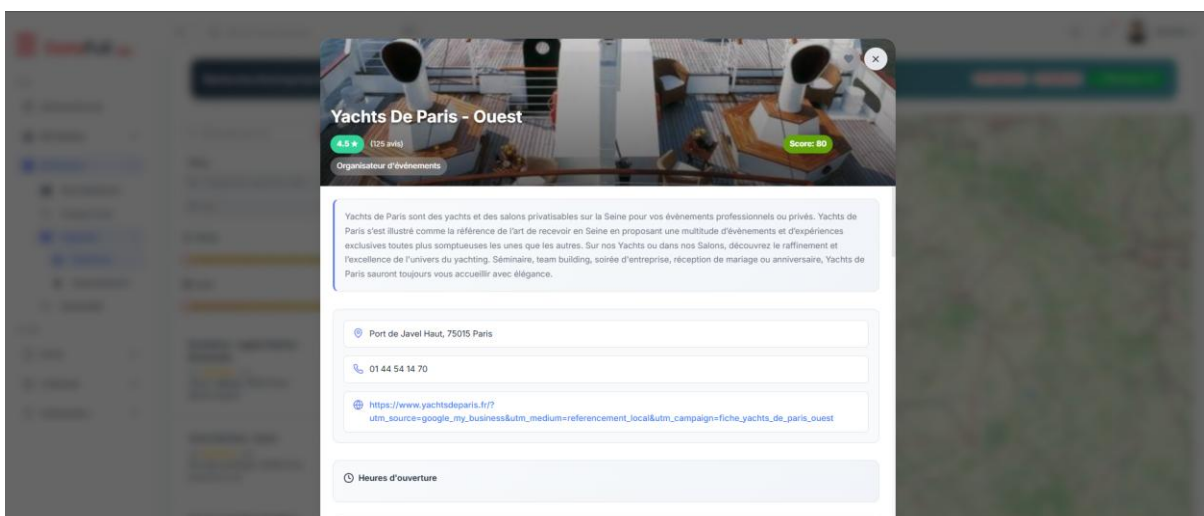


Figure 5.9. détaillée d'entreprise B2B

h. Page pour trouver les entreprises les mieux notées

Cette page présente un service de recherche et d'analyse Trustpilot permettant de trouver les entreprises les mieux notées, consulter les avis authentiques de millions de consommateurs, analyser la réputation des entreprises concurrentes, et identifier les opportunités de prospection en se basant sur les scores de satisfaction client et les commentaires vérifiés, avec possibilité d'exporter les données pour études de marché et prospection commerciale.

The screenshot displays the Trustpilot search results page. At the top, there's a search bar with the text 'Rechercher une entreprise par nom...' and a green 'Rechercher' button. Below the search bar, there are statistics: 1M+ Entreprises, 50M+ Avis clients, 99% Satisfaction, and 126 Résultats. The main section is titled 'Résultats de recherche' and shows 126 résultats trouvés. The results are listed as follows:

- Chez Laudio**: 4.9 rating, 0 avis. Address: 6 rue de la Source, 1753... Contact: 0546472432. Website: http://chez-laudio.fr. Trustpilot URL: http://chez-laudio.fr?utm_medium=company_profile&utm_source=trustpilot...
- Amplifon**: 4.0 rating, 130 avis. Address: 14 rue du port, 92000... Contact: 09 72 54 05 21. Website: http://amplifon.fr. Trustpilot URL: http://amplifon.fr?utm_medium=company_profile&utm_source=trustpilot...
- elcie-healthy.com**: 2.8 rating, 0 avis. Address: 86 Rue Voltaire, 93100... Contact: 0185001030. Website: http://elcie-healthy.com. Trustpilot URL: http://elcie-healthy.com?utm_medium=company_profile&utm_source=trustpilot...
- myBinocle**: 2.9 rating, 206 avis. Address: 86 Rue Voltaire, 93100... Contact: 0185001030. Website: http://mybinocle.com. Trustpilot URL: http://mybinocle.com?utm_medium=company_profile&utm_source=trustpilot...
- Les Ateliers Refaela**: 4.7 rating, 0 avis. Address: 86 Rue Voltaire, 93100... Contact: 0185001030. Website: https://www.refaelagroup.fr. Trustpilot URL: https://www.refaelagroup.fr?utm_medium=company_profile&utm_source=tr...
- Juyar**: 4.7 rating, 15 avis. Address: 86 Rue Voltaire, 93100... Contact: 0185001030. Website: http://juyar.fr. Trustpilot URL: http://juyar.fr?utm_medium=company_profile&utm_source=trustpilot&utm_c...

Figure 5.10. Page pour trouver les entreprises les mieux notées

i. Page d'analyse de la réputation et de la fiabilité d'une entreprise

Cette page permet d'analyser la réputation et la fiabilité d'une entreprise cible avant de l'approcher commercialement, en consultant les avis clients authentiques et les statistiques de performance pour évaluer sa crédibilité. Elle aide à identifier les points forts et faiblesses de l'entreprise via la répartition des notes et les commentaires détaillés, permettant d'adapter son approche commerciale en fonction du niveau de satisfaction client. L'outil facilite également la veille concurrentielle en comparant les performances et la réputation des concurrents, et permet de qualifier les prospects en vérifiant leur sérieux et leur réactivité via le taux de réponse aux avis, optimisant ainsi les stratégies de prospection B2B.

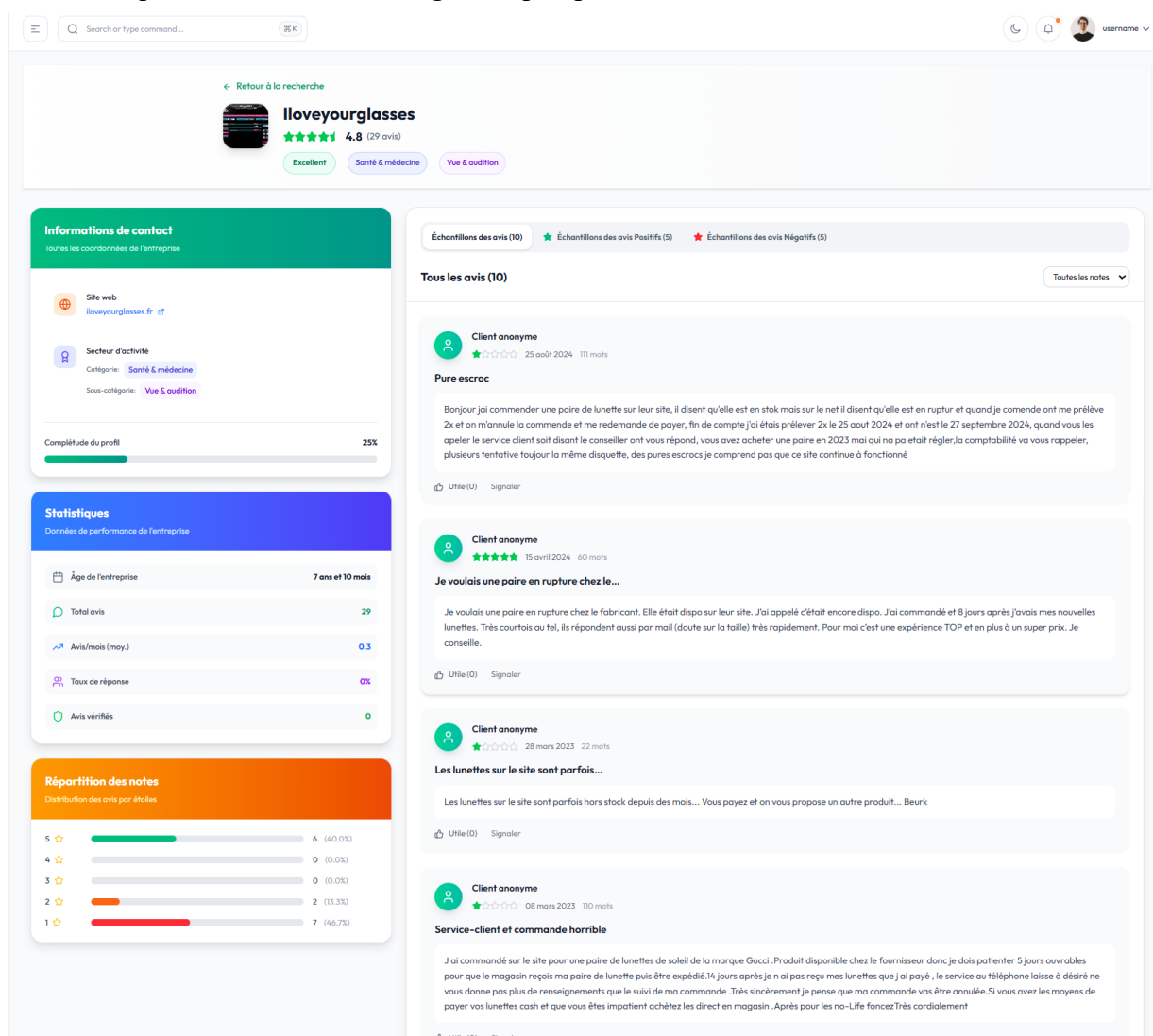


Figure 5.11. Page d'analyse de la réputation et de la fiabilité d'une entreprise

5.4.2. Application ETL

a. Tableau de bord ETL

Cette figure affiche le tableau de bord principal de l'application ETL. Il fournit une vue en temps réel de l'état du système, des tâches actives et de l'utilisation des ressources. Des boutons d'action rapide permettent aux utilisateurs de lancer de nouvelles tâches ou de téléverser directement des données.

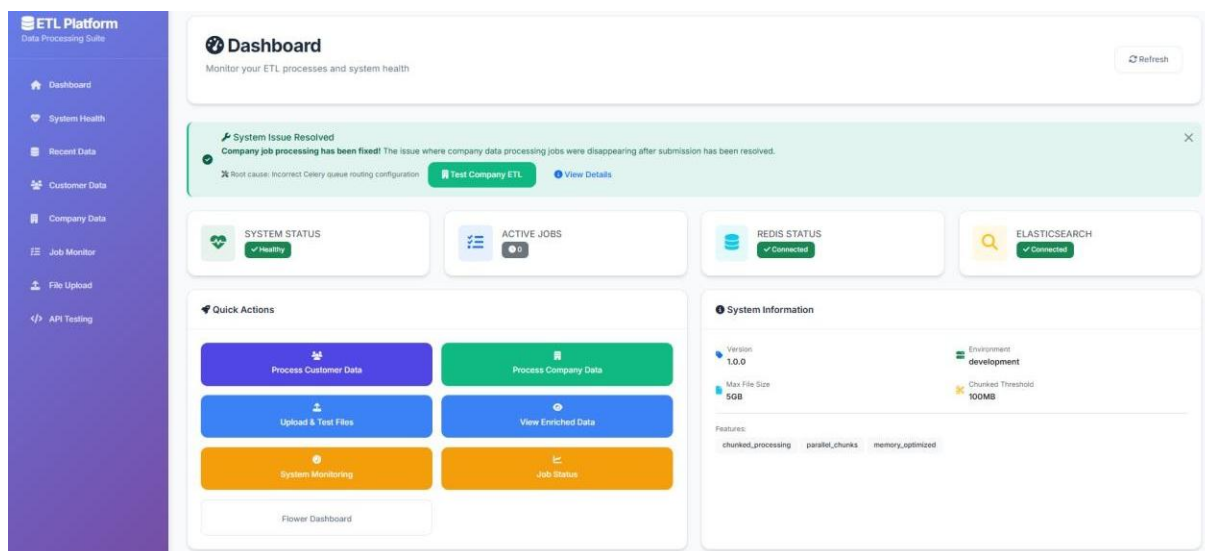


Figure 5.12. ETL Dashboard

b. Visualiseur de données récentes ETL

Cette figure montre le visualiseur de données récentes de l'application ETL. Il liste les ensembles de données récemment traités, avec des détails tels que les noms, les statuts, les horodatages et les métriques. Ce visualiseur permet des vérifications rapides et facilite le débogage des processus de traitement de données.

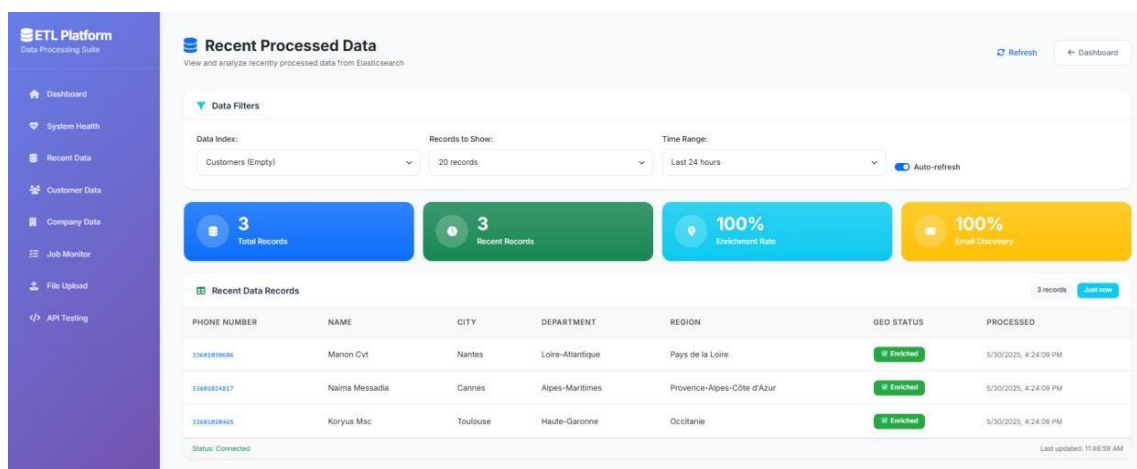


Figure 5.13. Visualiseur de données récentes ETL

c. Interface de test APIs

Cette figure illustre l'interface de test API de l'application ETL. Elle permet aux utilisateurs de tester des points de terminaison API en saisissant des paramètres et en lançant des requêtes. L'interface affiche les réponses, les codes de statut et les erreurs pour faciliter le débogage.

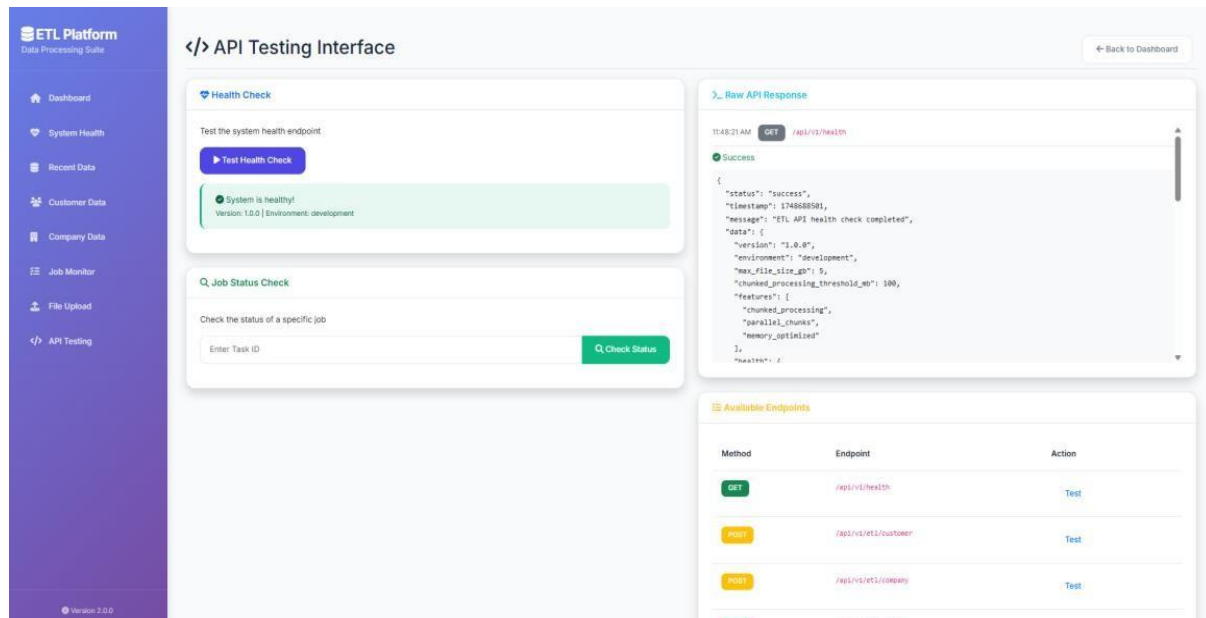


Figure 5.14. Interface de test APIs de ETL

d. Gestion et surveillance des tâches ETL

Cette figure illustre la section de gestion et de suivi des tâches de l'application ETL. Elle détaille le statut des tâches, leur avancement et les temps d'exécution. Les utilisateurs peuvent démarrer, arrêter ou redémarrer les tâches pour un suivi efficace.

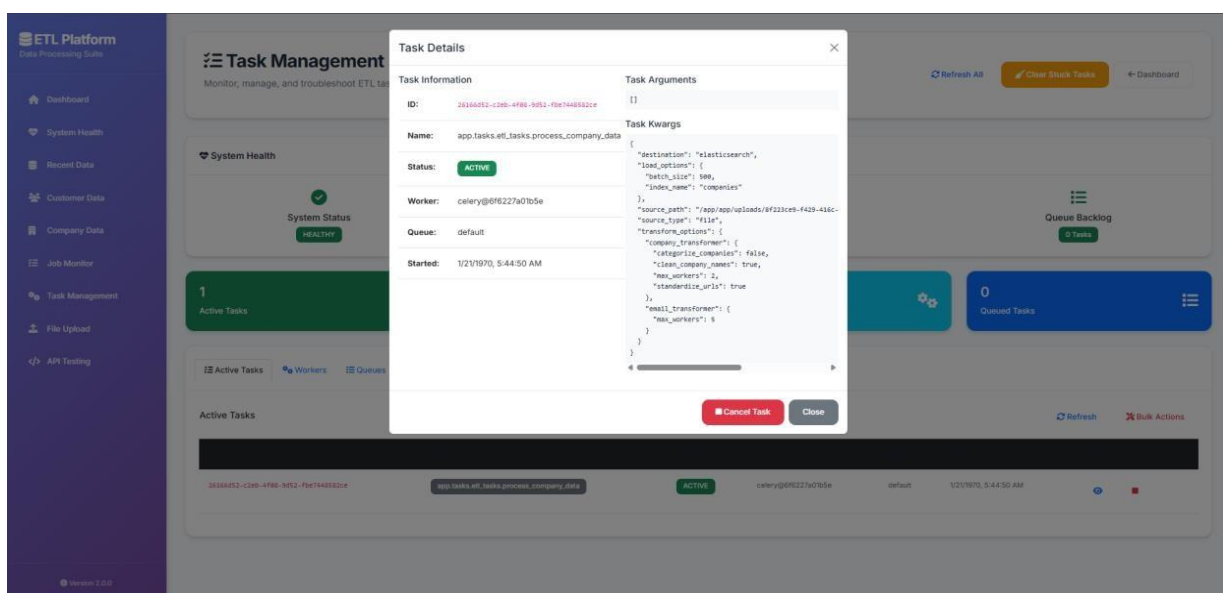


Figure 5.15. Gestion et surveillance des tâches ETL

5.5. Points à améliorer

- Module de gestion des utilisateurs : Finaliser le système de gestion des utilisateurs avec une authentification complète, la gestion des rôles et des interfaces dédiées aux utilisateurs est une priorité majeure.
- Finalisation de l'architecture microservices : Le système actuel nécessite l'ajout d'une passerelle API (API Gateway), d'un registre de services et de mécanismes de communication inter-services pour réaliser pleinement une architecture microservices robuste.
- Intégration de l'application ETL : L'application ETL, actuellement partiellement autonome, doit être intégrée pleinement au système principal. Son design modulaire peut être étendu pour supporter davantage de formats de données, règles de transformation et traitements programmés.
- Mise en place du système de paiement : L'intégration d'un système de paiement sécurisé (ex. Stripe) est nécessaire pour monétiser les services de la plateforme et gérer les abonnements.
- Stockage centralisé et automatisation : La transition vers une chaîne de stockage unifiée utilisant S3, avec ingestion et récupération automatisées, améliorera la scalabilité et réduira les interventions manuelles.
- Supervision et journalisation : L'incorporation de logs structurés et d'outils de monitoring en temps réel (ex. AWS CloudWatch, Prometheus) renforcera la visibilité opérationnelle et la fiabilité.
- Améliorations UI/UX : Le frontend doit être davantage optimisé pour la réactivité, l'usage mobile et des interactions dynamiques basées sur le comportement utilisateur.

5.6. Leçons apprises

Ce stage m'a véritablement permis de sortir de ma zone de confort et de progresser, tant en tant qu'ingénieur qu'en tant que membre d'équipe.

J'ai développé une grande capacité d'adaptation : démarrer avec FastAPI et Postgres, puis basculer vers Spring Boot et Elasticsearch au fil de l'évolution du projet, m'a appris à faire des choix technologiques durables et réfléchis.

La mise en place d'un pipeline ETL complet m'a sensibilisé à l'importance d'un code propre, modulaire et d'une gestion rigoureuse des erreurs, car la fiabilité est essentielle lorsqu'on manipule des données réelles.

La gestion d'instances EC2 et l'optimisation des coûts en environnement cloud m'ont permis de mieux comprendre les enjeux de performance et d'efficacité. Je suis aujourd'hui à l'aise avec des déploiements en conditions réelles.

Enfin, évoluer au sein d'une équipe agile en mode Scrum m'a fait prendre conscience de la force de la collaboration : décomposer les problèmes, avancer par itérations et livrer régulièrement font une réelle différence.

Surtout, je ressors de cette expérience avec confiance, prêt à concevoir et piloter des systèmes complexes tout en apportant une vraie valeur ajoutée.

5.7. Perspectives d'avenir de DataPull

En regardant vers l'avenir, la plateforme DataPull est bien positionnée pour s'intégrer à des technologies émergentes susceptibles d'améliorer considérablement ses fonctionnalités. L'intégration de l'intelligence artificielle (IA) et du machine learning représente la voie de développement la plus prometteuse.

- **Évaluation des leads par l'IA (Lead Scoring automatisé) :** La mise en œuvre d'algorithmes de machine learning pourrait permettre d'automatiser le processus de qualification des leads, en analysant les données historiques de conversion pour prédire la probabilité d'engagement d'un prospect. Cela permettrait aux utilisateurs de mieux prioriser leurs actions de prospection.
- **Traitement automatique du langage naturel (NLP) :** L'intégration de capacités NLP renforcerait l'extraction de données à partir de sources non structurées, en classant et en étiquetant automatiquement les informations sur les entreprises afin d'en améliorer la recherche et l'organisation.
- **Analyse prédictive :** Des capacités analytiques avancées pourraient fournir aux utilisateurs des prévisions sur les tendances du marché, des recommandations sur le meilleur moment pour contacter les prospects, et des informations stratégiques sur les concurrents, basées sur l'analyse de données agrégées.
- **Opportunités d'expansion de marché :** L'architecture modulaire de la plateforme la rend particulièrement adaptée à une extension vers des marchés connexes et à de nouveaux cas d'usage. Une expansion géographique au-delà du marché français actuel pourrait exploiter la même infrastructure technique, tout en s'adaptant aux sources de données et réglementations locales.
- **Gestion de la relation client (CRM) :** L'intégration avec des systèmes de gestion de la relation client (CRM), des plateformes d'automatisation marketing et des outils de vente

pourrait positionner DataPull comme un élément central de l'écosystème technologique commercial des entreprises, quelle que soit leur taille.

- **Conformité réglementaire et éthique**

Les développements futurs devront impérativement tenir compte de l'évolution des réglementations en matière de protection des données, au-delà du RGPD. L'adoption des principes de privacy by design, la gestion avancée du consentement et des politiques d'utilisation des données transparentes seront indispensables à une croissance durable.

Les implications éthiques de la collecte et du traitement de données à grande échelle nécessitent une attention continue. L'établissement de directives claires, la mise en place de mécanismes d'opposition (opt-out) et la transparence dans le traitement des données seront essentiels pour maintenir la confiance des utilisateurs et respecter les obligations réglementaires.

5.8. Conclusion

Ce chapitre a présenté en détail la mise en œuvre des principaux composants de la plateforme DataPull, incluant le scraping de données, les applications web et le pipeline ETL. Nous avons exposé les solutions adoptées, les résultats obtenus et les défis rencontrés durant le développement. Ces réalisations constituent les bases solides pour des améliorations futures et des intégrations à venir, visant à renforcer la robustesse et la scalabilité de la plateforme.

Conclusion Générale et Perspectives

Le projet DataPull s'inscrit dans une démarche d'innovation technologique au service de la prospection commerciale. Il a permis de valider la faisabilité d'une plateforme orientée données, capable de collecter, structurer et restituer de l'information pertinente pour identifier de nouveaux leads. Ce travail, mené selon une approche itérative, a abouti à la réalisation d'un outil fonctionnel, évolutif et adapté aux besoins concrets du marché.

Tout au long du développement, des choix technologiques solides ont été effectués : architecture en microservices, pipeline ETL robuste, interface utilisateur intuitive et modulaire. Ces choix ont non seulement permis de garantir la performance et la maintenabilité du système, mais ont aussi constitué une base technique durable pour les évolutions futures. Les difficultés rencontrées, notamment en matière de traitement des données non structurées, de fiabilité des sources et de conception modulaire, ont été surmontées grâce à une veille continue, un travail collaboratif rigoureux et une capacité d'adaptation permanente.

Les perspectives d'amélioration sont nombreuses. À moyen terme, l'intégration de modèles d'intelligence artificielle, notamment pour le scoring prédictif des leads ou la recommandation de prospects, permettrait de transformer DataPull en un véritable assistant intelligent. Le recours à des algorithmes de machine learning et de traitement du langage naturel ouvrirait la voie à une exploitation plus fine et plus stratégique des données disponibles. Par ailleurs, l'automatisation de workflows et la mise en place de systèmes d'alerte intelligents augmenteraient considérablement la réactivité commerciale des utilisateurs.

En parallèle, une extension fonctionnelle de la plateforme pourrait être envisagée à travers l'intégration avec des CRM populaires, la création de connecteurs pour des sources tierces, ou encore le développement de modules de veille concurrentielle automatisée. Cela positionnerait DataPull comme un véritable hub central de l'intelligence commerciale.

Enfin, ce projet a représenté une étape déterminante sur le plan professionnel. Il a permis de mettre en pratique des compétences techniques et méthodologiques, tout en développant une réelle capacité à conduire un projet complexe dans un environnement proche de celui de l'entreprise. Ce stage a non seulement contribué à la montée en compétences des membres de l'équipe, mais a aussi confirmé l'intérêt d'une pédagogie par projet, à la croisée de l'innovation technologique et des besoins métiers.

Références

- [1] Site web Havet Digital : <https://havetdigital.fr/> [consulté le 6 mai 2025]
- [2] Présentation Iaweb.Dev : <https://havetdigital.fr/iaweb-dev/> [consulté le 6 mai 2025]
- [3] Concepts de nettoyage et enrichissement de données : <https://www.datacamp.com/tutorial/data-cleaning-in-python> [consulté le 10 mai 2025]
- [4] Principes du développement backend : <https://www.geeksforgeeks.org/backend-development/> [consulté le 8 mai 2025]
- [5] UI/UX Design Principles: <https://www.interaction-design.org/literature/topics/ui-ux-design> [last time visited on May 10, 2025]
- [6] GDPR : <https://gdpr-info.eu/> [last time visited on April 30, 2025]
- [7] API Integration Concepts: <https://www.redhat.com/en/topics/api/what-is-api-integration> [last time visited on May 10, 2025]
- [8] ETL Process Overview: <https://www.talend.com/resources/what-is-etl/> [last time visited on May 15, 2025]
- [9] Cloud Infrastructure (AWS) Concepts: <https://aws.amazon.com/what-is/cloud-computing/> [last time visited on May 15, 2025]
- [10] FastAPI vs Spring Boot vs Laravel Benchmark: <https://blog.okami101.io/2023/12/a-2024-benchmark-of-main-web-api-frameworks/#conclusion> [last time visited on June 2, 2025]
- [11] PostgreSQL vs Elasticsearch vs MongoDB Comparison: <https://db-engines.com/en/system/Elasticsearch%3BMongoDB%3BPostgreSQL> [last time visited on June 2, 2025]
- [12] Frontend Frameworks Comparison: <https://dev.to/rfornal/comparing-angular-react-vue-and-vanilla-js-37o9> [last time visited on June 6, 2025]
- [13] TechEmpower Framework Benchmarks: <https://www.techempower.com/benchmarks/> [last time visited on June 6, 2025]
- [14] Apache Airflow Performance Best Practices: <https://airflow.apache.org/docs/apache-airflow/stable/best-practices.html> [last time visited on June 6, 2025]
- [15] AWS Glue Monitoring Guide: <https://docs.aws.amazon.com/glue/latest/dg/monitoring-awsglue-with-cloudwatch.html> [last time visited on June 6, 2025]
- [16] Documentation UML : <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/> [consulté le 20 mars, 2025]
- [17] React Documentation: <https://react.dev/learn> [last time visited on May 22, 2025]
- [18] Spring Boot Documentation: <https://docs.spring.io/spring-boot/docs/current/reference/html/> [last time visited on May 20, 2025]
- [19] Documentation Elasticsearch : <https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html> [consulté le 25 mai 2025]
- [20] Documentation PostgreSQL : <https://www.postgresql.org/docs/> [consulté le 10 mai 2025]
- [21] Documentation Docker : <https://docs.docker.com/> [consulté le 25 mai 2025]
- [22] Documentation GitHub : <https://docs.github.com/> [consulté le 28 mai 2025]
- [23] Documentation Postman : <https://www.postman.com> [consulté le 28 mars 2025]
- [24] Documentation Jenkins : <https://www.jenkins.io/doc> [consulté le 18 mai 2025]
- [25] Documentation Lombok : <https://projectlombok.org/features/all> [consulté le 4 avril 2025]
- [26] Documentation Swagger : <https://swagger.io/docs/> [consulté le 2 avril 2025]
- [27] Documentation Junit : <https://junit.org/junit5/docs/current/user-guide/> [consulté le 28 juin 2025]
- [28] Python Documentation: <https://docs.python.org/3/> [last time visited on April 5, 2025]
- [29] FastAPI Documentation: <https://fastapi.tiangolo.com/> [last time visited on May 8, 2025]
- [30] Documentation AWS EC2 : <https://docs.aws.amazon.com/ec2/> [consulté le 20 mai 2025]
- [31] Documentation AWS S3 : <https://docs.aws.amazon.com/s3/> [consulté le 22 mai 2025]
- [32] Documentation Uvicorn : <https://www.uvicorn.org/> [consulté le 8 mai 2025]
- [33] Documentation Gunicorn : <https://docs.gunicorn.org/en/stable/> [consulté le 10 mai 2025]
- [34] Documentation Flask : <https://flask.palletsprojects.com/> [consulté le 25 mai 2025]
- [35] Colab Notebook : <https://colab.research.google.com/> [consulté le 12 mai 2025]
- [36] Documentation VS Code : <https://code.visualstudio.com/docs> [consulté le 25 mars 2025]
- [37] Documentation Redis : <https://redis.io/docs/> [consulté le 28 mai 2025]
- [38] Documentation Celery : <https://docs.celeryq.dev/> [consulté le 30 mai 2025]
- [39] SQL : <https://en.wikipedia.org/wiki/SQL> [consulté le 8 mai 2025]
- [40] Power BI : <https://learn.microsoft.com/en-us/power-bi/fundamentals/power-bi-overview> [consulté le 10 juin 2025]
- [41] Manuel Shell et Bash : <https://www.gnu.org/software/bash/manual/bash.html> [consulté le 18 avril 2025]
- [42] Documentation INPI : <https://www.inpi.fr/> [consulté le 28 mars 2025]
- [43] Documentation INSEE : <https://www.insee.fr/> [consulté le 30 mars 2025]
- [44] Documentation INPI FTP : https://www.inpi.fr/sites/default/files/Documentation%20technique%20SFTP%20Actes%20v1.0%20mars%202023_0.pdf [consulté le 5 avril 2025]
- [45] OpenDataSoft : <https://data.opendatasoft.com/explore/> [consulté le 8 avril 2025]
- [46] Documentation BeautifulSoup : <https://www.crummy.com/software/BeautifulSoup/bs4/doc/> [consulté le 12 avril 2025]
- [47] Documentation Selenium : <https://www.selenium.dev/documentation/> [consulté le 15 avril 2025]
- [48] Benchmarks EC2 : <https://cloudonaut.io/ec2-network-performance-of-t3/> [consulté le 22 avril 2025]
- [49] Documentation Keycloak : <https://www.keycloak.org/documentation> [consulté le 28 mai 2025]